

西南交通大學
Southwest Jiaotong University

智能嵌入式系统设计

-实验指导书-

陈大玲 何滨

西南交通大学计算机与人工智能学院

计算机科学与技术系

2026 年 3 月

目　录

实验一 实验开发环境及实验开发板的使用.....1
一、实验目的.....1
二、实验设备.....1
三、实验内容.....1
四、思考题.....28
实验二 GPIO 实验.....29
一、实验目的.....29
二、实验设备.....29
三、实验原理.....29
四、实验内容.....35
五、思考题.....37
实验三 外部中断实验.....38
一、实验目的.....38
二、实验设备.....38
三、实验原理.....38
四、实验内容.....42
五、思考题.....43
实验四 异步通信实验.....44
一、实验目的.....44
二、实验设备.....44
三、实验原理.....44
四、实验内容.....51
五、思考题.....52
实验五 DMA 实验.....53
一、实验目的.....53
二、实验设备.....53
三、实验原理.....53
四、实验内容.....56
五、思考题.....56
实验六 定时器实验.....57
一、实验目的.....57
二、实验设备.....57
三、实验原理.....57
四、实验内容.....64
五、思考题.....64
实验七 ADC 实验.....65
一、实验目的.....65
二、实验设备.....65

三、实验原理.....65
四、实验内容.....71
五、思考题.....71
实验八 嵌入式操作系统应用实验.....72
一、实验目的.....72
二、实验设备.....72
三、实验原理.....72
四、实验内容.....79
五、思考题.....79
嵌入式系统设计与应用综合设计.....80
一、实验目的.....80
二、实验设备及软件.....80
三、实验内容.....80
四、综合性实验设计要求.....80

实验一 实验开发环境及实验开发板的使用

一、实验目的

1.学习和掌握《嵌入式系统设计及应用》实验环境的配置，认识嵌入式系统实验板。

2.了解硬件编程工具 MDK5 的程序的安装、编程、编译及代码下载方法。

2.了解项目生成工具 Cube MX 程序的安装、项目创建、配置、工程自动创建方法。

3.了解简单汇编程序、C 程序的设计及软件延时计算器的使用，初步学习 MDK5 环境调试方法。

4.学习在嵌入式中输入输出指令的使用。

二、实验设备

1.硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

2.配套软件系统：

MDK5、Cube MX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验内容

(一)软件环境配置

1. MDK5 软件安装及配置

（1）MDK5 简介

MDK 源自德国的 KEIL 公司，是 RealViewMDK 的简称。在全球 MDK 被超过 10 万的嵌入式开发工程师使用。目前最新版本为：MDK536，该版本使用 uVision5IDE 集成开发环境，是目前针对 ARM 处理器，尤其是 CortexM 内核处理器的最佳开发工具。

MDK5 向后兼容 MDK4 和 MDK3 等，以前的项目同样可以在 MDK5 上进行开发(但是头文件方面得全部自己添加)，MDK5 同时加强了针对 Cortex-M 微控制器开发的支持，并且对传统的开发模式和界面进行升级，MDK5 由两个部分组成：MDKCore 和 SoftwarePacks。其中，SoftwarePacks 可以独立于工具链进行新芯片支持和中间库的升级。

在 MDK5 安装完成后，要让 MDK5 支持 STM32F103 的开发，我们还需要安装 STM32F1 的器件支持包： Keil.STM32F1xx_DFP.2.3.0.pack（STM32F1 的器件包）。

（2）安装 MDK5 及器件包

I. MDK5 的安装与配置

MDK-ARM

下载地址 www.keil.com/download/product/



<1>在 SofeWave 文件夹下，运行“MDK536.exe”安装软件，进入如图 1-1 所示界面，按照安装向导提示安装完成。

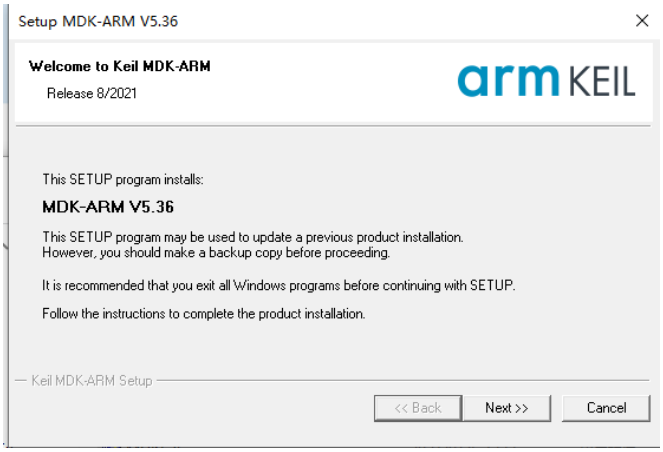


图 1-1 MDK536 安装首页

<2>以管理员模式启动桌面上 Keil 软件的快捷链接“KeiluVision5”，进入软件如图 1-2 所示，选择菜单“File”→“LicenseManagement”进入如图 1-3 配置界面，点击“GetLICviainternet...”按钮，获得自己计算机的 CID，运行 SofeWave\keil_C51And_MDK\ARM 目录下的“keil 通用注册.exe”，复制 CID 到图 1-4 所示通用注册软件中，点击“Generate”按钮，获得“NewLicenseIDCode（LIC）”，复制新 LIC 编码到图 1-3 的“NewLicenseIDCode（LIC）”框中，点击“AddLIC”按钮，列表中出现对应项，Keil 软件配置完成。

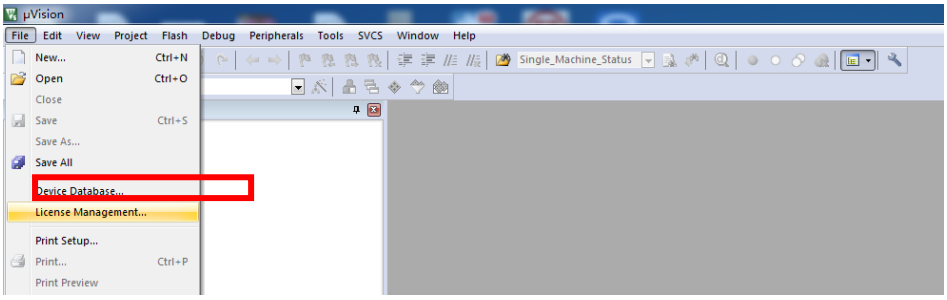


图 1-2 Keil 编译环境

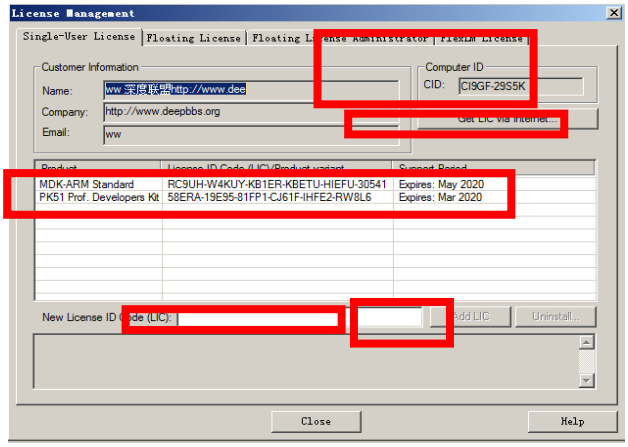


图 1-3 LicenseManagement 配置界面



图 1-4 Keil **通用注册**软件界面

II. 安装“器件包”，两种安装方法

下载文件直接安装 点击目录 SofeWave 下的“Keil.STM32F1xx_DFP.2.3.0.pack”按向导运行，安装 STM32F1 系列芯片的器件包，下载地址 www.keil.com/dd2/pack。如图 1-5。

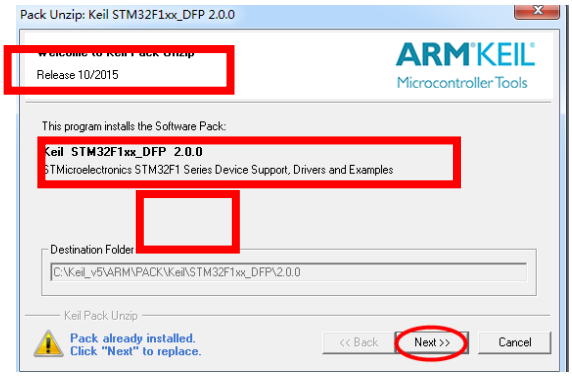


图 1-5 器件包安装界面

MDK 器件包在线安装 启动 Keil uVision5，如图 1-6。点击 Pack Installer，安装 STM32F1 系列芯片的器件包。

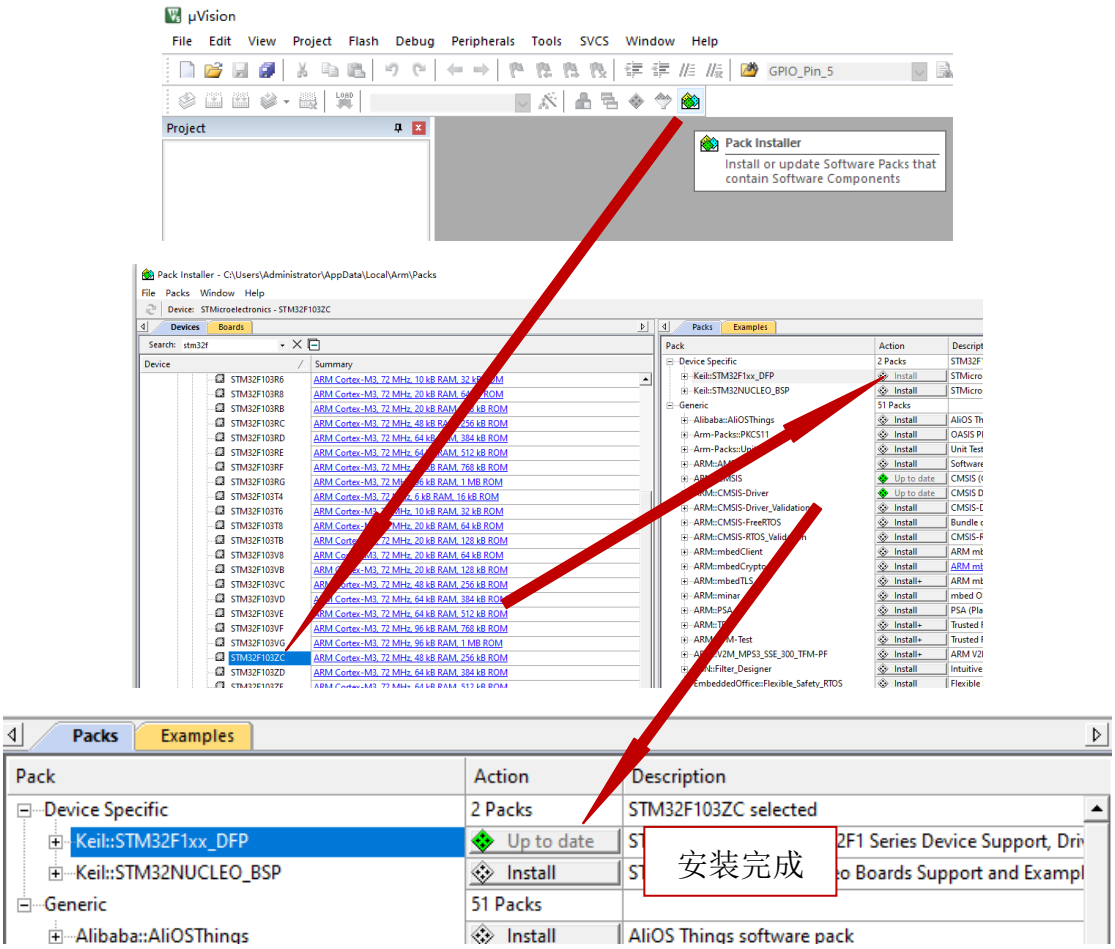


图 1-6 器件包在线安装

III. USB 驱动程序安装

根据安装向导提示，安装 USB 驱动程序“USBtoUARTDriver—CH340_CH341”。

IV. ST-Link 驱动程序安装

根据安装向导提示，安装 dpinst_amd64，如图 1-7。

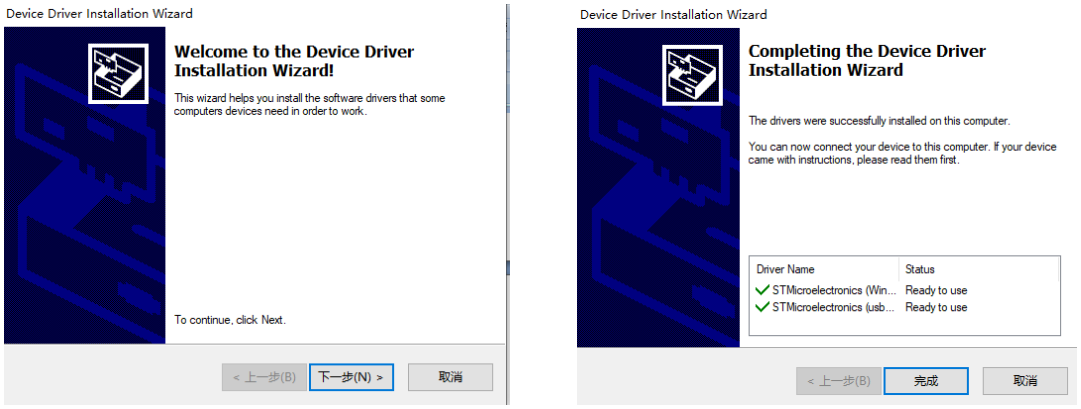


图 1-7 ST-Link 安装

V. 固件升级程序 新实验开发板使用DAP-Link下载方式，不用完成这一步

默认升级程序路径：C:\Keil_v5\ARM\STLink
固件升级程序名称：ST-LinkUpgrade.exe

此操作需要连接实验开发板完成。

DAP-Link下载方式的实验板不用完成这一步

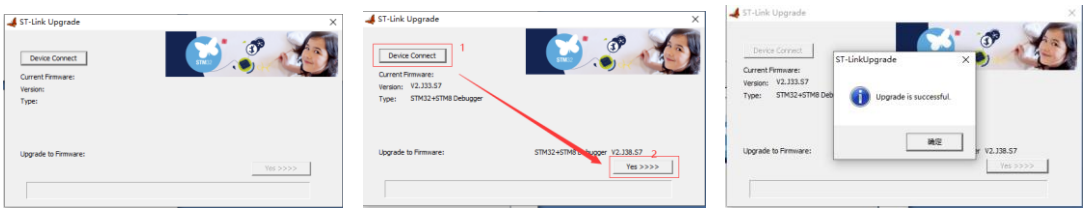
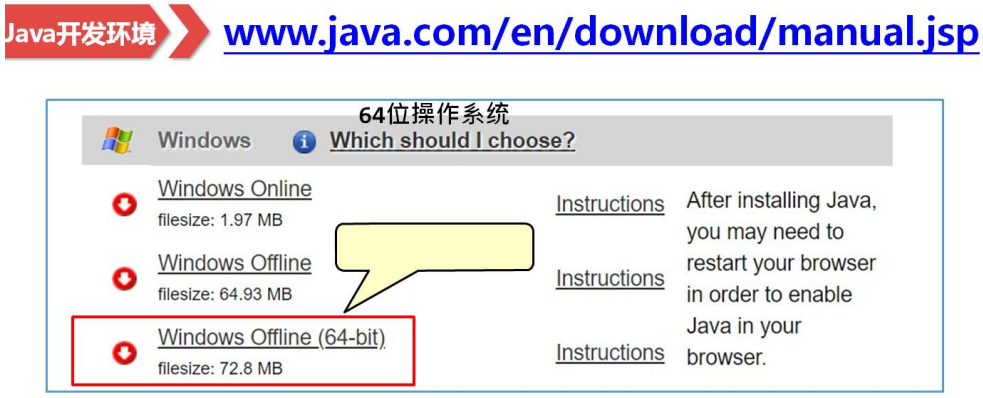


图 1-8 固件升级程序安装

2. Java 软件安装及配置



可以在网上下载，也可以用软件包中的软件安装，根据安装向导提示，安装 Java 程序“ jre-8u301-windows-x64”，安装过程使用默认配置。



图 1-9 Java 安装

3. CubeMX 软件安装及配置

(1) CubeMX 软件安装

可以在网上下载，也可以用软件包中的软件安装，根据安装向导提示，安装 CubeMX 程序“SetupSTM32CubeMX-6.3.0-Win”，安装过程使用默认配置。

注意：安装路径不能包含中文，建议使用默认的安装路径。

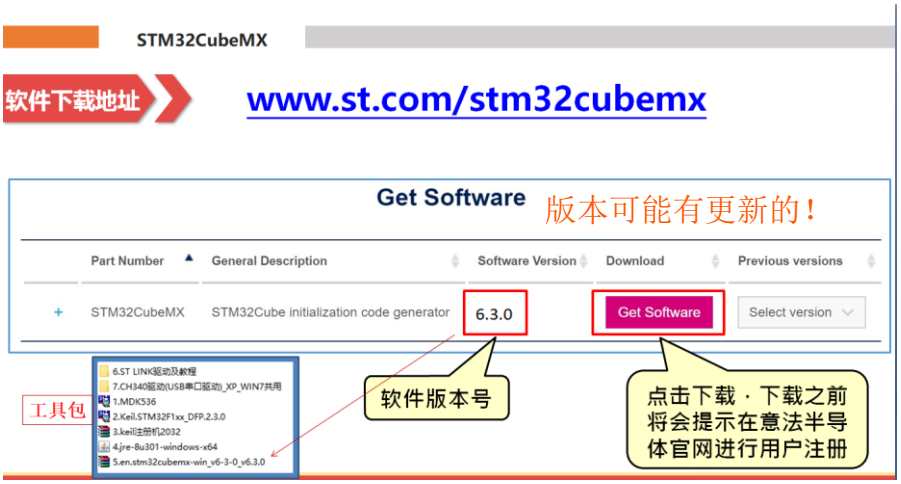
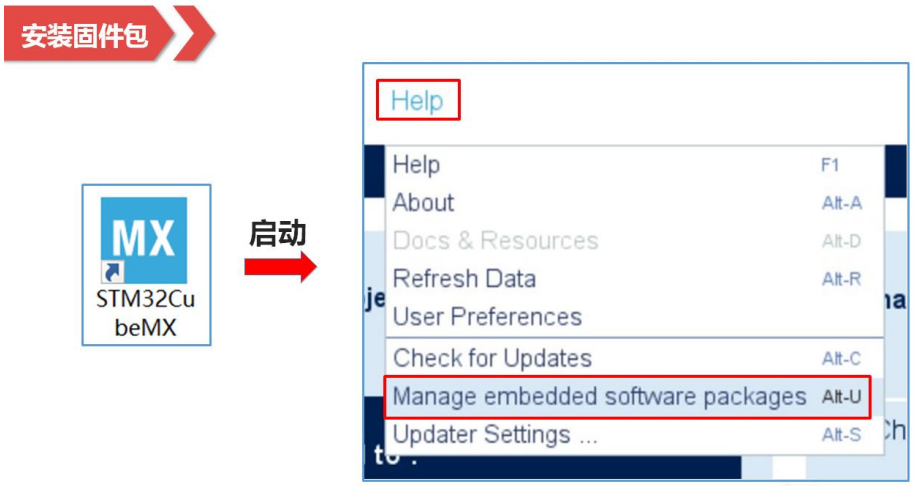


图 1-10 CubeMX 软件下载

(2) CubeMX 固件包在线安装



选择 STM32F1,勾选最新版 1.8.4，点击 Install Now，安装完成，复选框变为绿色。此步需要在网络环境下完成。

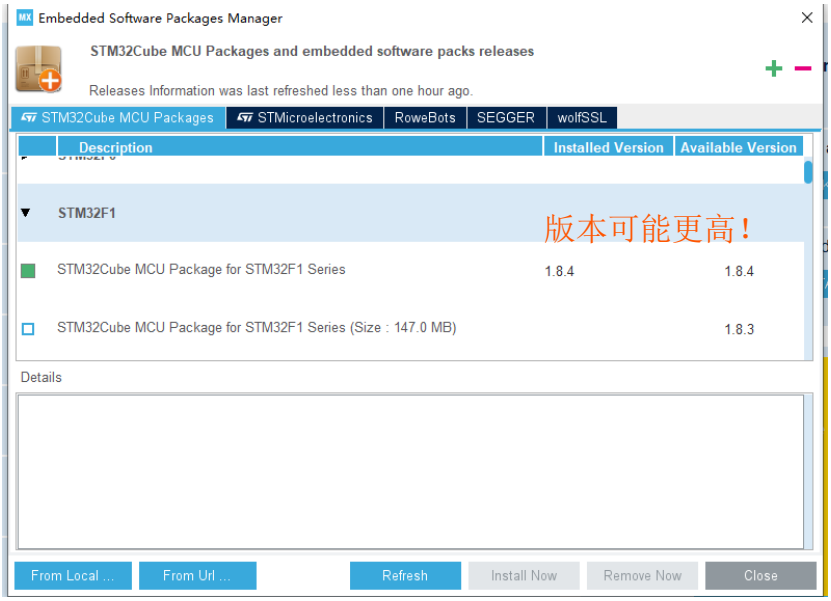


图 1-11 CubeMX 固件包在线安装

(二)软件开发环境测试

1、使用 CubeMX 快速创建工程,使用 Keil_V5 进行程序设计和调试。

CubeMX 具有**集成 MCU Finder 功能，便于用户进行芯片选型；图形化方式配置时钟、片内外设，自动生成初始化代码；自动生成工程框架，支持 MDK、EWARM 等多种开发环境；提高开发效率，聚焦业务层的设计。**CubeMX 开发流程如图 1-12 所示。

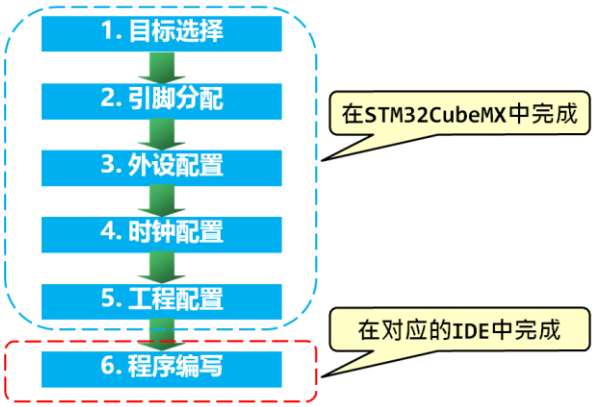


图 1-12 CubeMX 开发流程图

建议同学们在桌面上新建一个名为 ARM_CubeMX 的文件夹，用于存放本课程的例程代码。对于 STM32 微控制器的外设学习，可以再建立子文件夹：按照 外设名建立，如 GPIO、EXTI、TIMER、UART 等。注意：CubeMX 建立的工程必须存放在英文路径下。



图 1-13 CubeMX 启动界面简介

设计目标：开发板上 D1 灯以 1s 的频率闪烁。启动 CubeMX，在“New Project”栏选择“Start My Project From MCU”,选择 MCU，在“Part Number”框输入 STM32F103ZC,在右边的备选栏中选择STM32F103ZCTx,LQFP144封装,这是本学期实验开发板使用的MCU。如图 1-14。 鼠标双击所选择 MCU，进入配置主界面，如图 1-15.

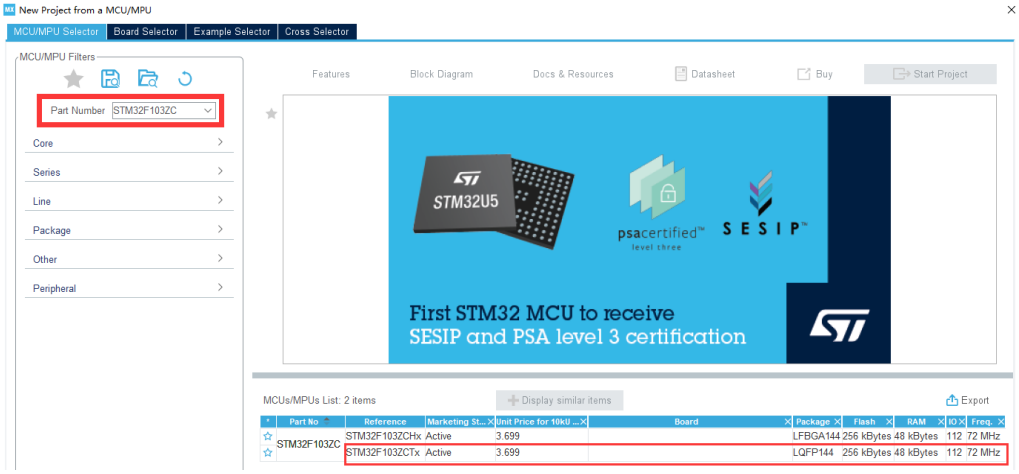


图 1-14 CubeMX 芯片选型

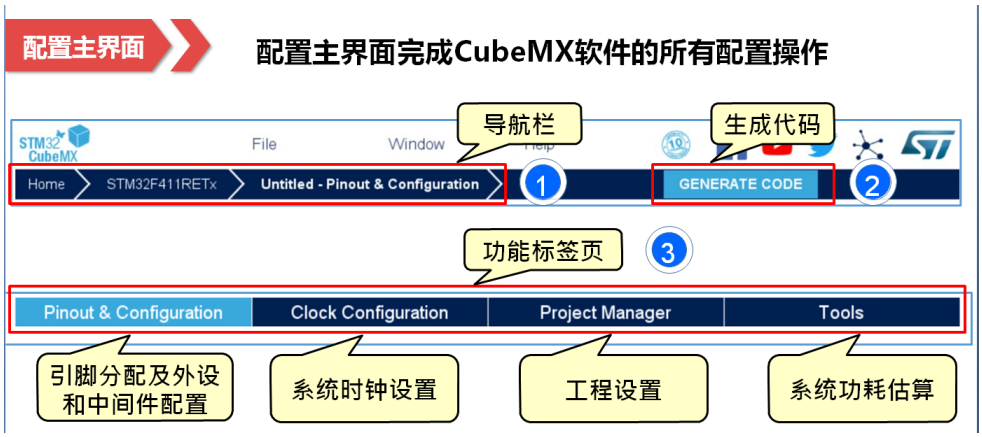


图 1-15 CubeMX 配置主界面

配置时钟模式，如图 1-16。

配置调试接口，如图 1-17。

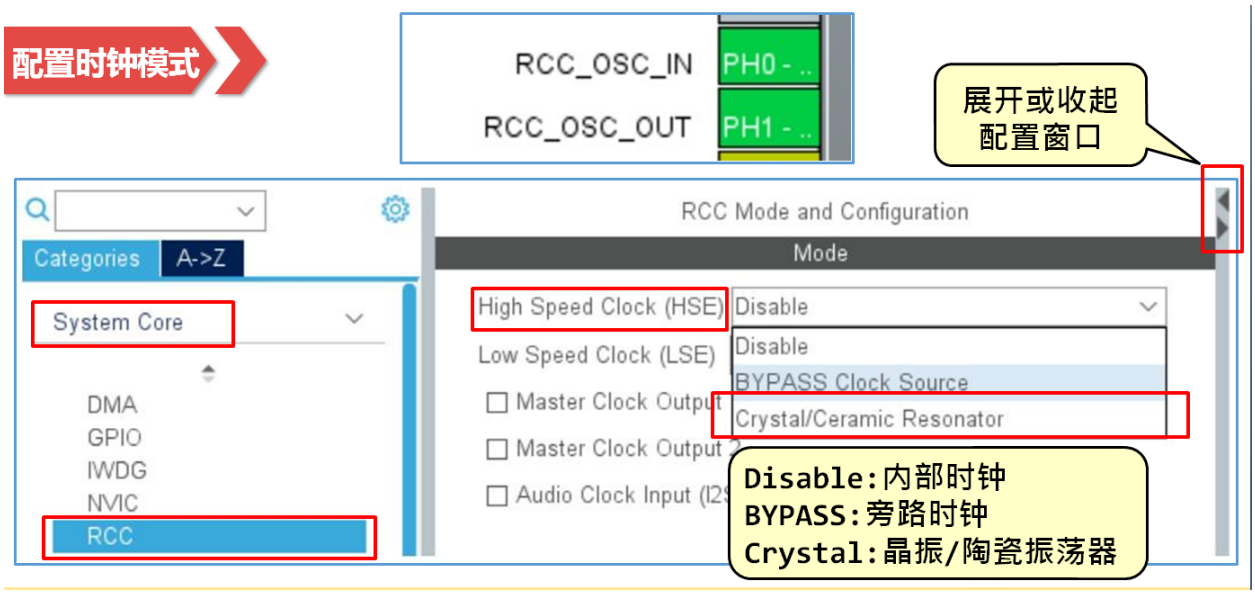


图 1-16 CubeMX 时钟模式配置界面

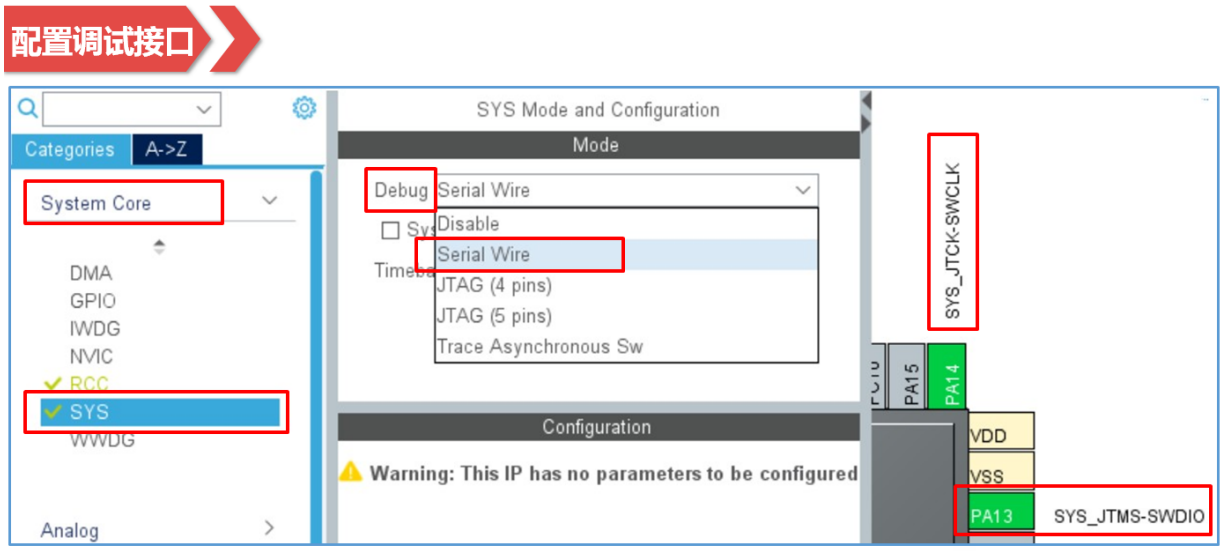


图 1-17 CubeMX 调试接口配置界面

配置 GPIO，根据设计要求，D1 电路如图 1-18，连结芯片 PF0 引脚，PF0 输出高电平，D1 点亮，PF0 输出低电平，D1 熄灭，即：用 PF0 的输出控制 D1 的亮灭。现配置 PF0 引脚为输出模式，在右下角搜索框输入 PF0，对应引脚开始闪烁，如图 1-19，鼠标单击 PF0，弹出引脚模式对话框，选择 GPIO_Output，在 GPIO 的 configuration 栏鼠标点击 PF0 栏，出现 PF0 configuration 窗口，模式设置为推挽输出模式“Output Push Pull”。

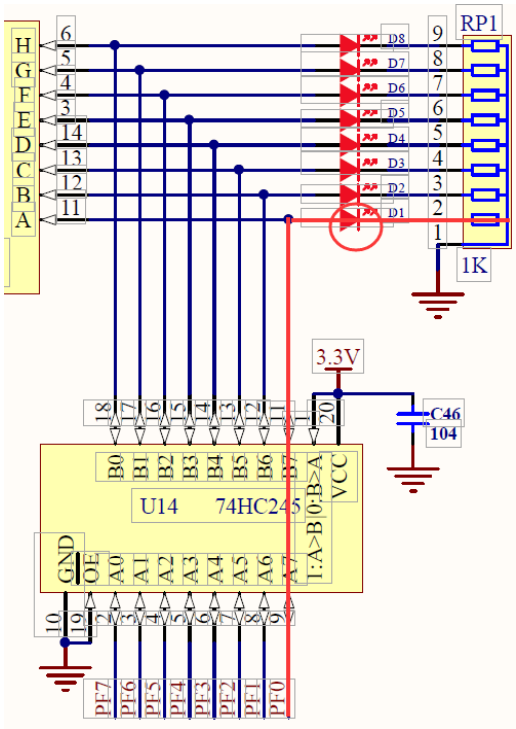


图 1-18 发光二极管 D1-D8 电路图



图 1-19 CubeMX GPIO 配置界面

配置时钟。选择 HSE 锁相环，配置 APB1 为 1MHZ，如图 1-20.

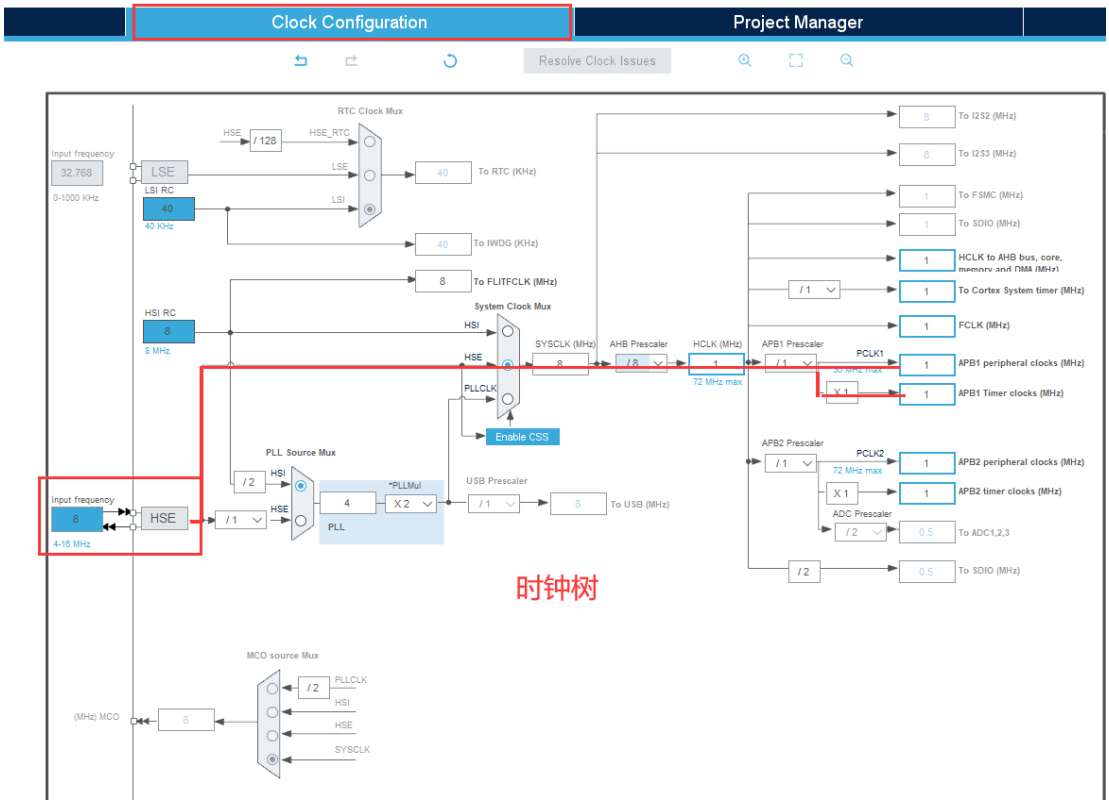


图 1-20 CubeMX 时钟树配置界面

配置工程管理。依次配置工程管理、代码生成、高级设置。

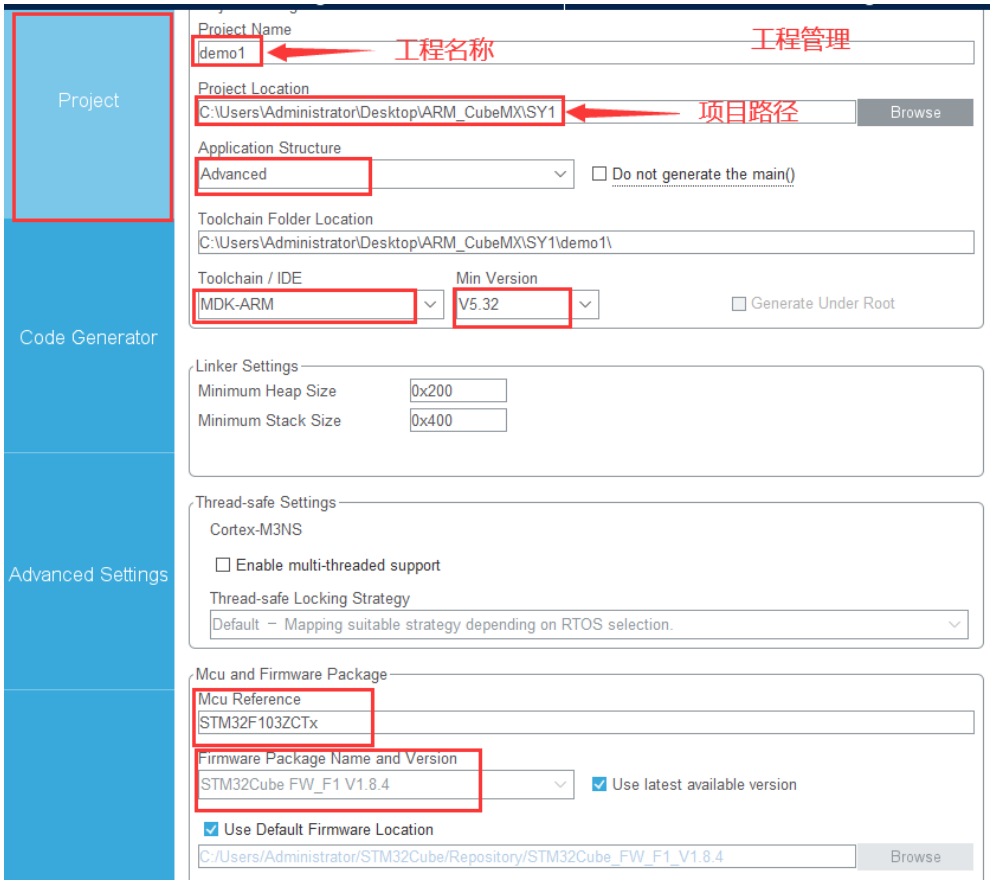


图 1-21 CubeMX 工程管理 Project 界面

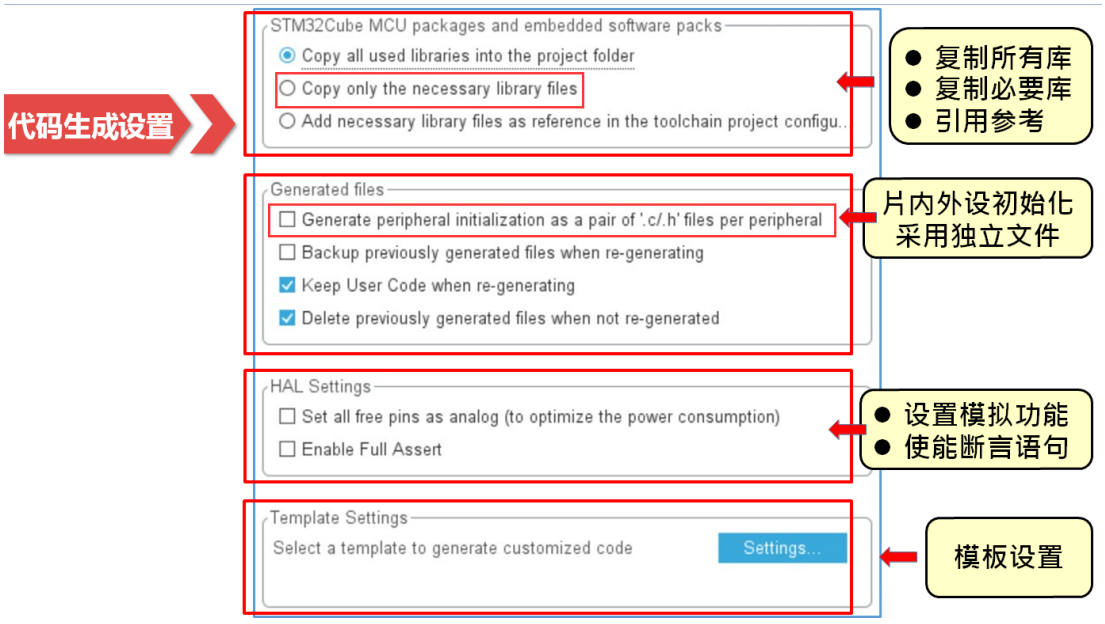


图 1-22 CubeMX 工程管理 Code Generator 界面

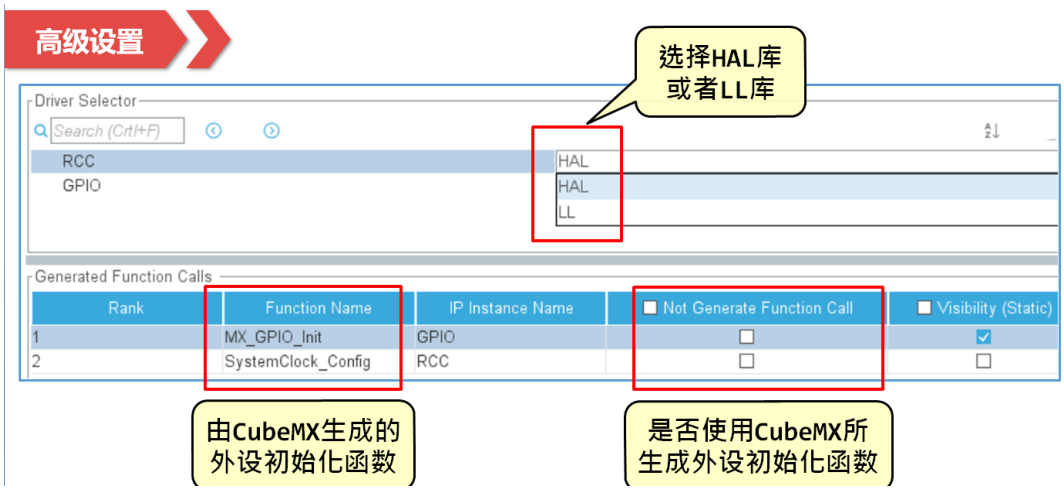


图 1-22 CubeMX 工程管理高级设置界面

配置完成，生成 MDK-ARM 集成开发环境的工程。

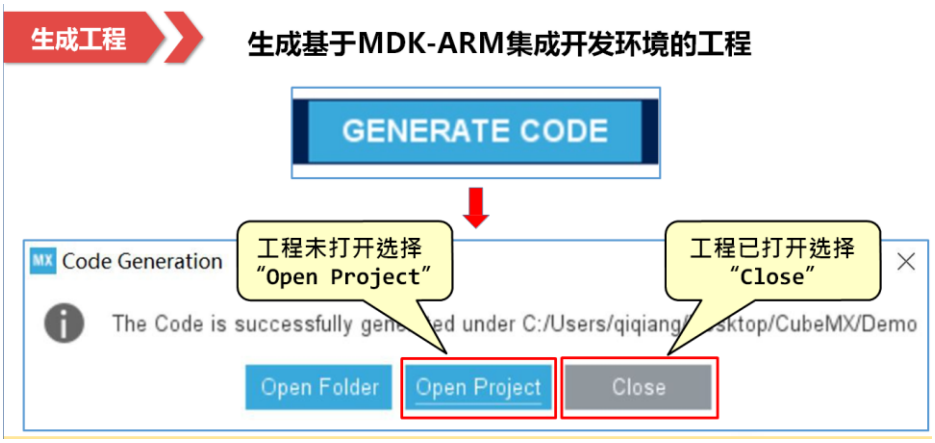


图 1-23 在 CubeMX 中创建 MDK 工程代码

打开 Keil_V5,如图 1-24，直接点击编译，输出窗口提示编译成功，0Warning，0Error,软件环境配置完成，输入代码，再次编译，USB 连结实验开发板，点击下载按钮，观察实验效果。

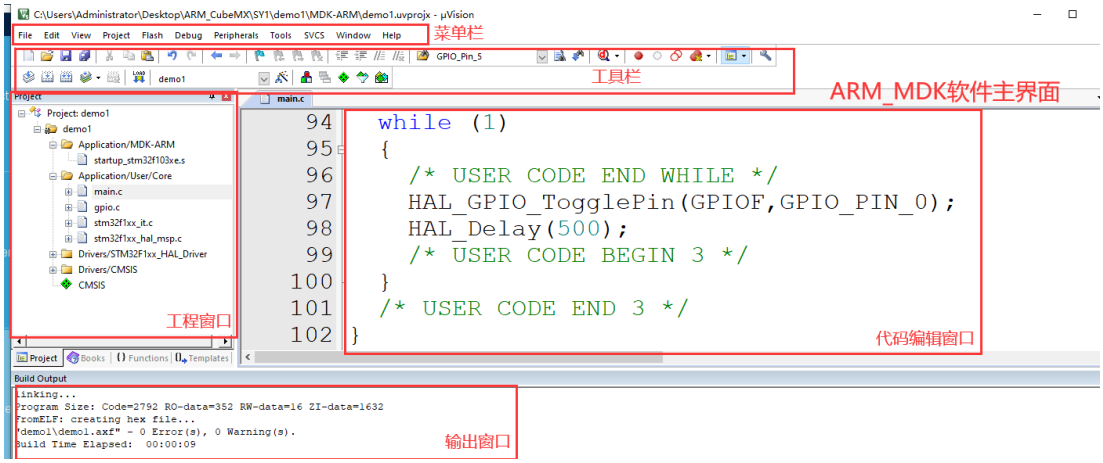
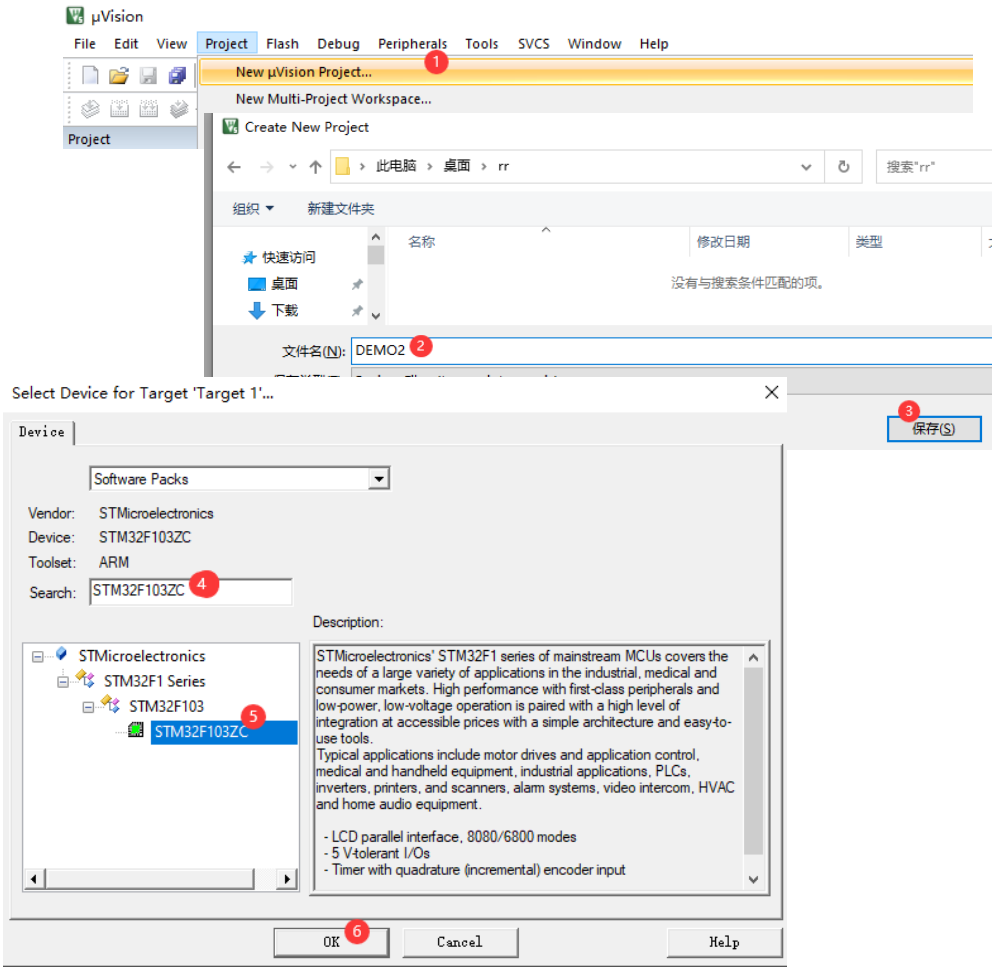
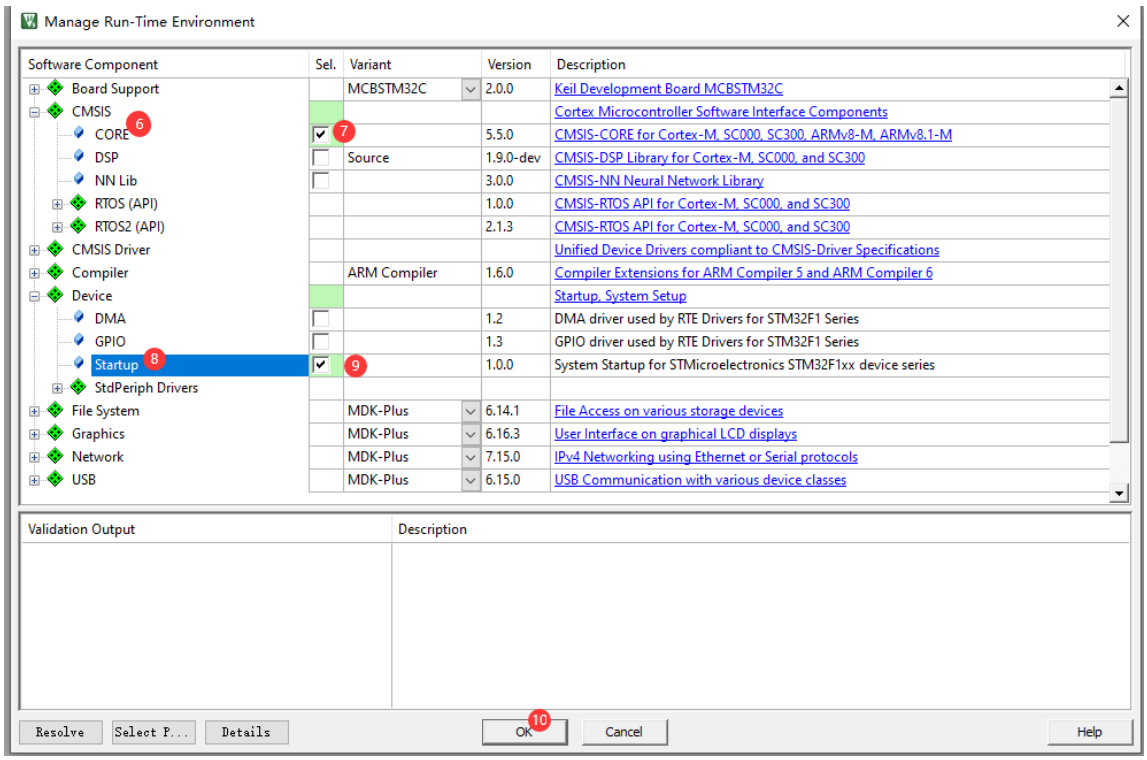


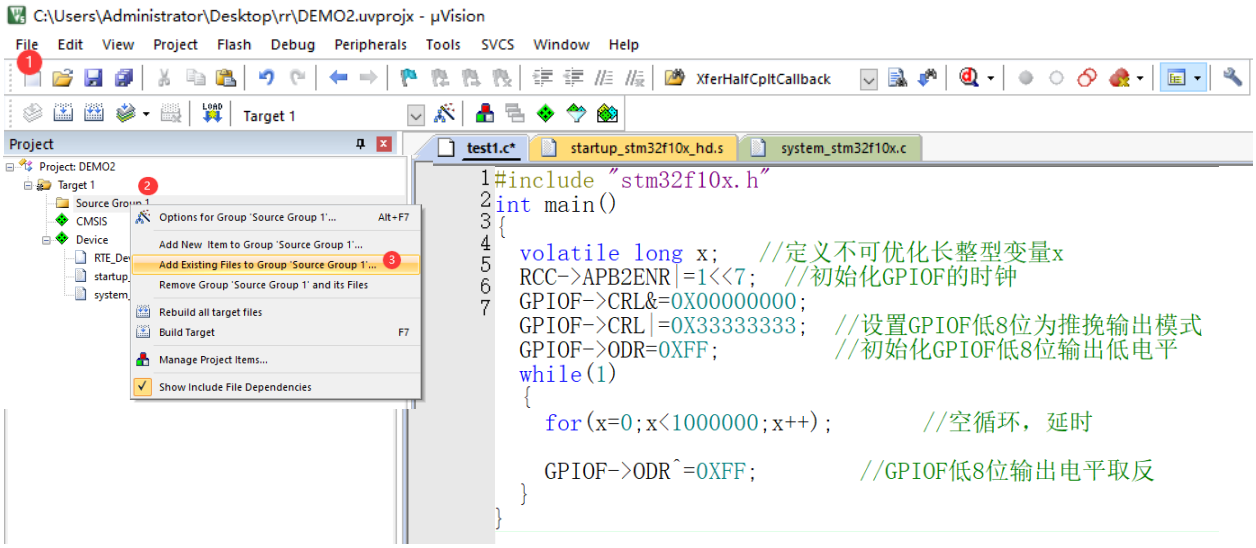
图 1-24 ARM-MDK 软件主界面

2、使用 Keil_V5 创建工程、程序设计和调试。

基于寄存器的开发模式，按照如图所示创建工程 DEMO2；







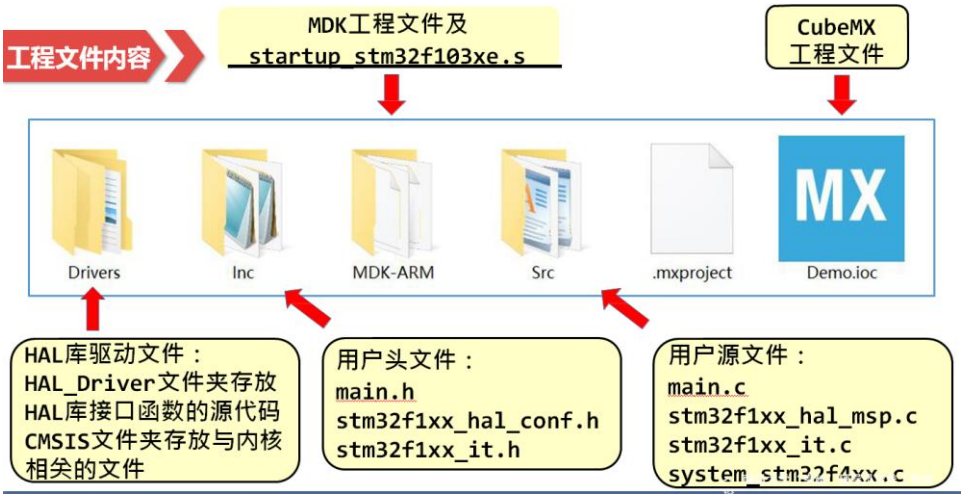
新建源程序，并保存为 test.c（**特别注意扩展名为 c**），并将该文件添加到 Source Group1，编写程序，保存，直接点击编译，输出窗口提示编译成功，0Warning，0Error，USB 连结实验开发板，点击下载按钮，观察实验效果（8 个跑马灯闪烁）。

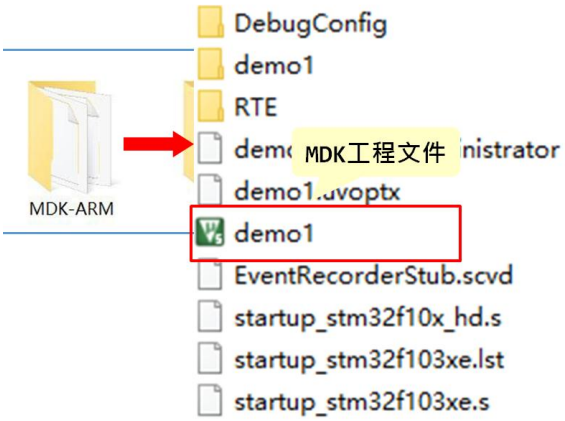
(三)软件开发工具的使用

1、MDK-ARM 的使用流程



（1）创建工程，使用 CubeMX 创建，MDK-ARM 打开工程，如图所示。

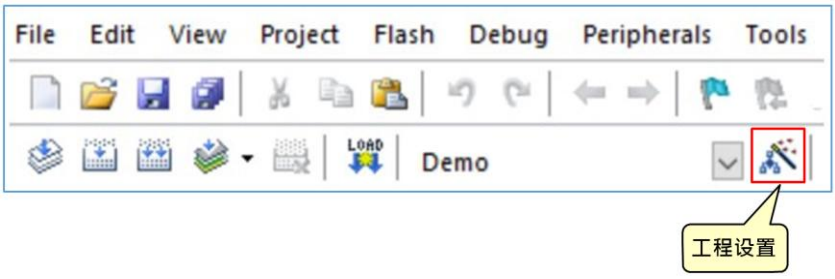


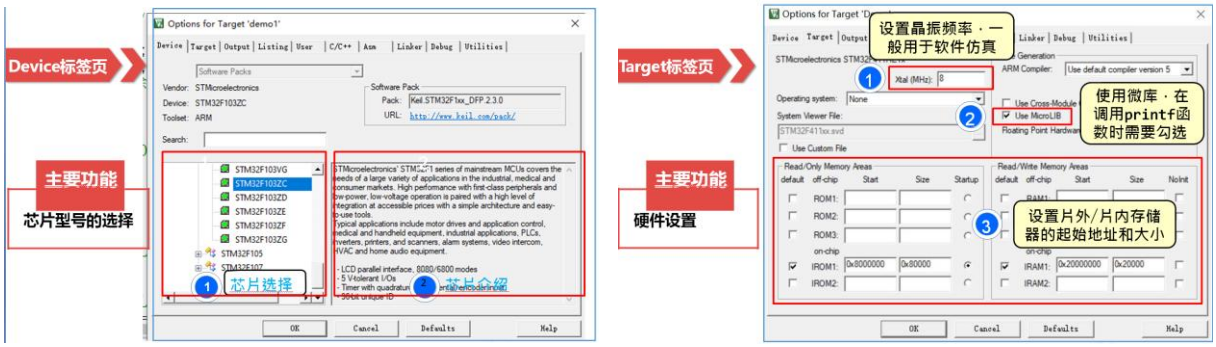


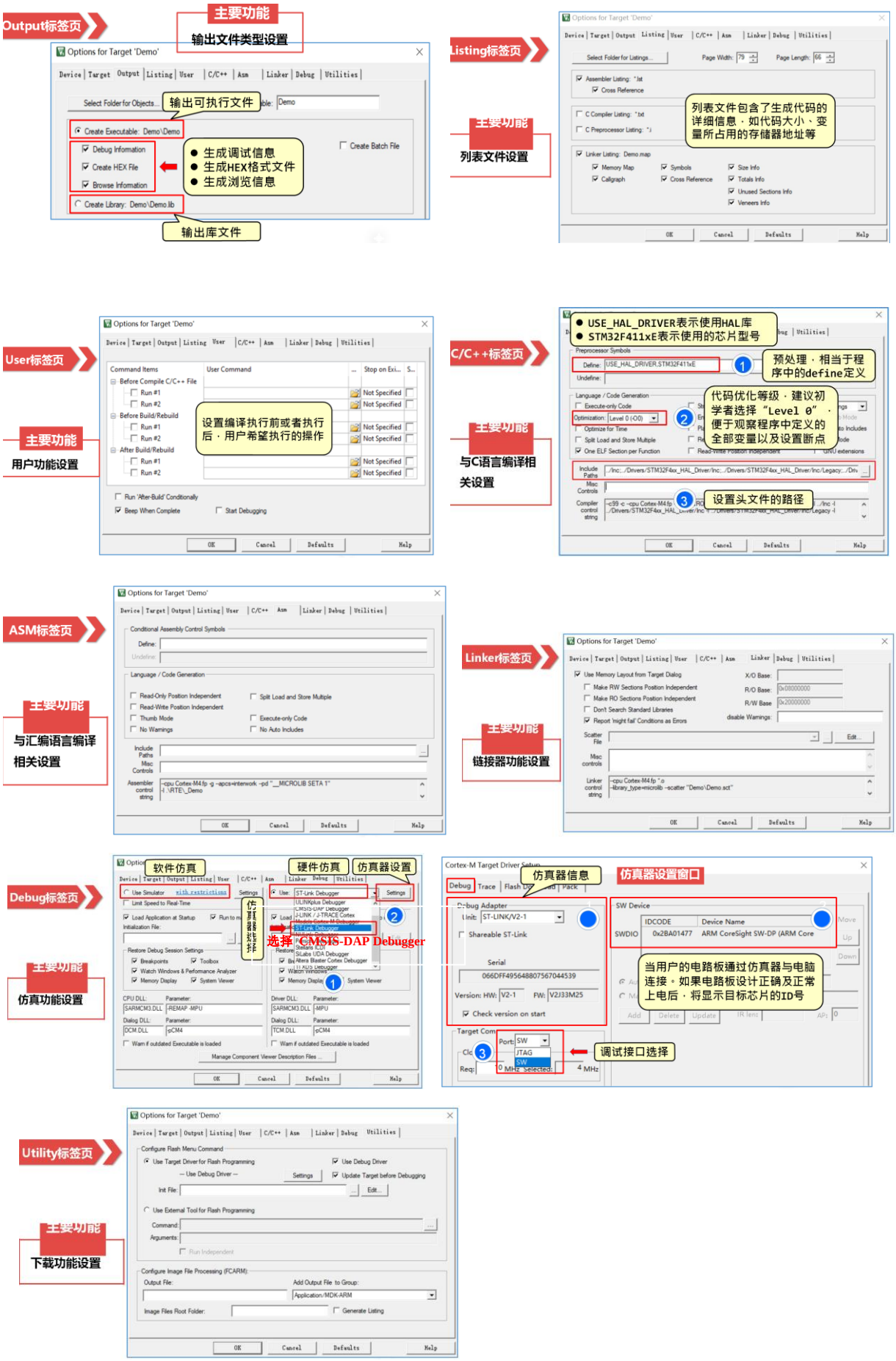
MDK-ARM 设计界面及工程框架如图。



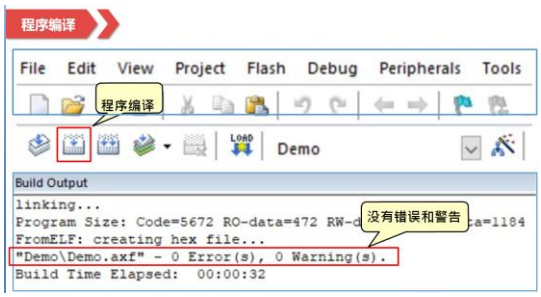
（2）工程设置。

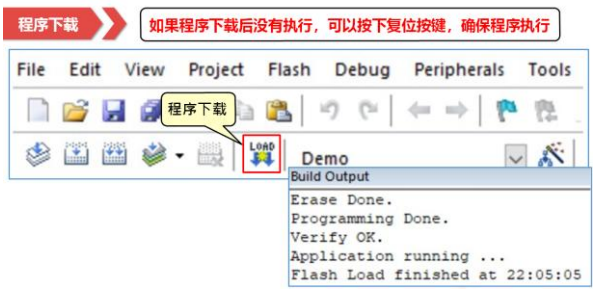





Output标签页
主要功能
输出文件类型设置
Options for Target 'Demo'
Device | Target | Output | Listing | User | C/C++ | Asm | Linker | Debug | Utilities
Select Folder for Objects...
输出可执行文件
Demo
Create Executable: Demo\Demo
Debug Infomation
Create HEX File
Browse Information
生成调试信息
生成HEX格式文件
生成浏览信息
Create Batch File
Create Library: Demo\Demo.lib
输出库文件
Listing标签页
主要功能
列表文件设置
Select Folder for Listings...
Page Width:
Page Length:
Assembler Listing: *.lst
Cross Reference
C Compiler Listing: *.txt
C Preprocessor Listing: *.i
Linker Listing: Demo.map
Memory Map
Symbols
Size Info
Callgraph
Cross Reference
Totals Info
Unused Sections Info
Veneers Info
列表文件包含了生成代码的详细信息，如代码大小、变量所占用的存储器地址等
OK
Cancel
Defaults
Help
User标签页
主要功能
用户功能设置
Command Items
User Command
Stop on Exi...
Before Compile C/C++ File
Before Build/Rebuild
After Build/Rebuild
Run #1
Run #2
Not Specified
设置编译执行前或者执行后，用户希望执行的操作
Run 'After-Build' Conditionally
Beep When Complete
Start Debugging
C/C++标签页
主要功能
与C语言编译相关设置
USE_HAL_DRIVER表示使用HAL库
STM32F411xE表示使用的芯片型号
Preprocessor Symbols
Define: USE_HAL_DRIVER,STM32F411xE
Undefine:
预处理，相当于程序中的define定义
Language / Code Generation
Execute-only Code
Optimization: Level 0 (-O0)
Optimize for Time
Split Load and Store Multiple
One ELF Section per Function
代码优化等级，建议初学者选择“Level 0”，便于观察程序中定义的全部变量以及设置断点
Include Paths
Misc Controls
Compiler control string
设置头文件的路径
ASM标签页
主要功能
与汇编语言编译相关设置
Conditional Assembly Control Symbols
Define:
Undefine:
Language / Code Generation
Read-Only Position Independent
Read-Write Position Independent
Thumb Mode
No Warnings
Split Load and Store Multiple
Execute-only Code
No Auto Includes
Include Paths
Misc Controls
Assembler control string
Linker标签页
主要功能
链接器功能设置
Use Memory Layout from Target Dialog
Make RW Sections Position Independent
Make RO Sections Position Independent
Don't Search Standard Libraries
Report 'might fail' Conditions as Errors
X/O Base:
R/O Base:
R/W Base
disable Warnings:
Scatter File
Edit...
Misc controls
Linker control string
Debug标签页
主要功能
仿真功能设置
软件仿真
硬件仿真
仿真器设置
Use Simulator
with restrictions
Use: ST-Link Debugger
Settings
Limit Speed to Real-Time
Load Application at Startup
Run to main()
Initialization File:
Restore Debug Session Settings
Breakpoints
Toolbox
Watch Windows & Performance Analyzer
Memory Display
System Viewer
CPU DLL:
Parameter:
SARMCM3.DLL
-REMAP -MPU
Dialog DLL:
DCM.DLL
pCM4
Warn if outdated Executable is loaded
Manage Component Viewer Description Files ...
Driver DLL:
SARMCM3.DLL
-MPU
TCM.DLL
选择“CMSIS-DAP Debugger”
Cortex-M Target Driver Setup
仿真器信息
仿真器设置窗口
Debug
Trace
Flash Download
Pack
Debug Adapter
Unit: ST-LINK/V2-1
Shareable ST-Link
Serial
066DFF495648807567044539
Version: HW: V2-1
FW: V2J33M25
Check version on start
SW Device
IDCODE
Device Name
SWDIO
0x2BA01477
ARM CoreSight SW-DP (ARM Core
当用户的电路板通过仿真器与电脑连接。如果电路板设计正确及正常上电后，将显示目标芯片的ID号
Target Com
Port: SW
JTAG
SW
Clo
Req:
MHz selected
4 MHz
调试接口选择
Add
Delete
Update
IR len:
Move
Up
Down
Utility标签页
主要功能
下载功能设置
Configure Flash Menu Command
Use Target Driver for Flash Programming
Use Debug Driver
Update Target before Debugging
Settings
Init File:
Edit...
Use External Tool for Flash Programming
Command:
Arguments:
Run Independent
Configure Image File Processing (FCARM):
Output File:
Add Output File to Group:
Application/MDK-ARM
Image Files Root Folder:
Generate Listing

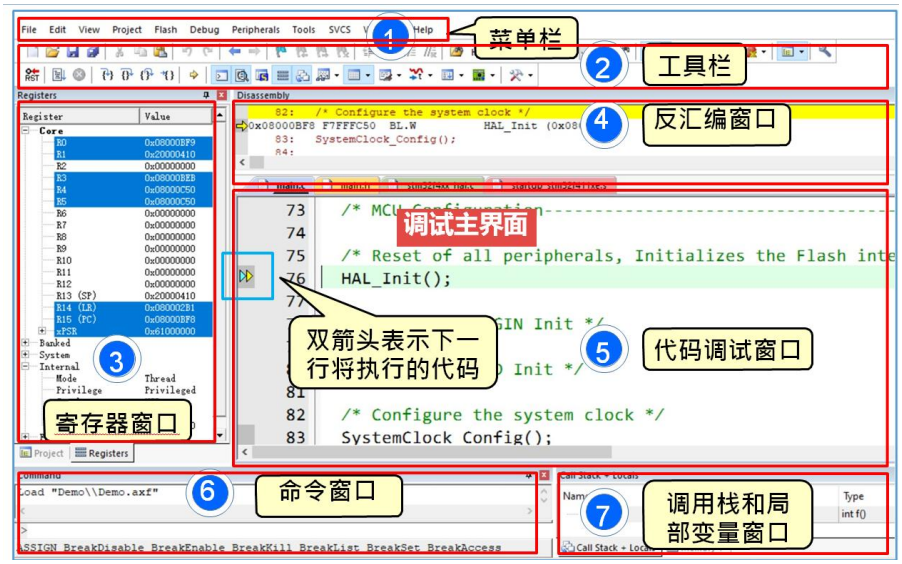
（3）编译与下载。



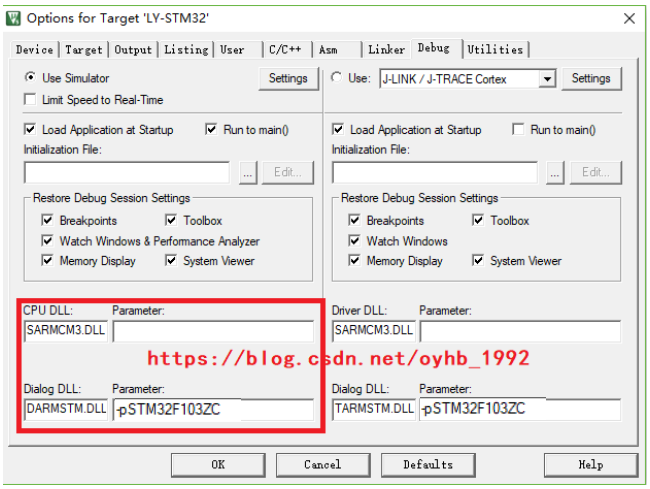


（4）编译与下载。





如果逻辑分析仪 Logic Analyzer 无法添加，或者无信号，则需要再工程设置中配置 debug 部分。



(四)硬件环境及工具测试

1、实验开发板硬件电路

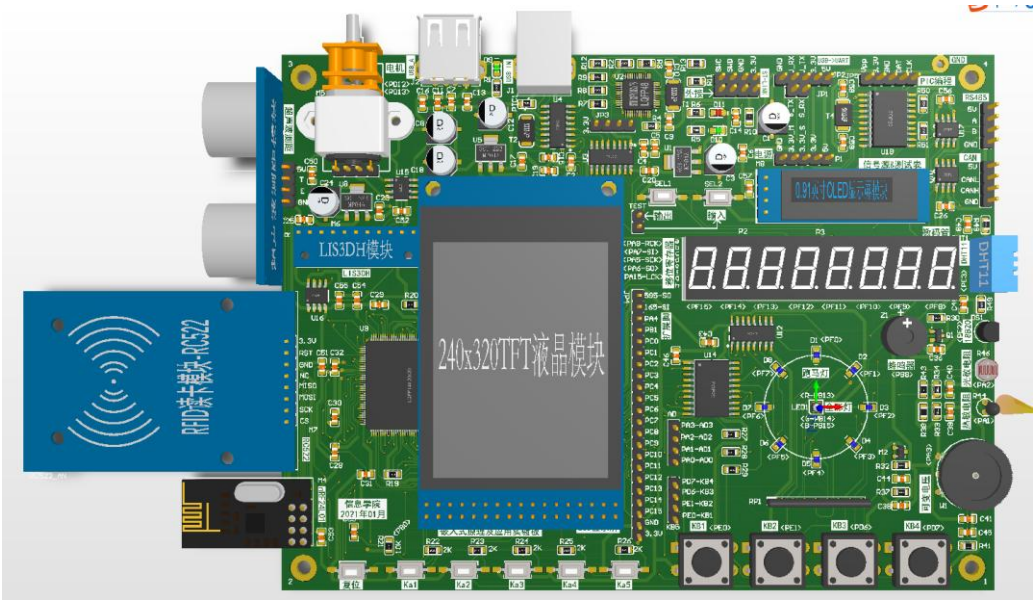
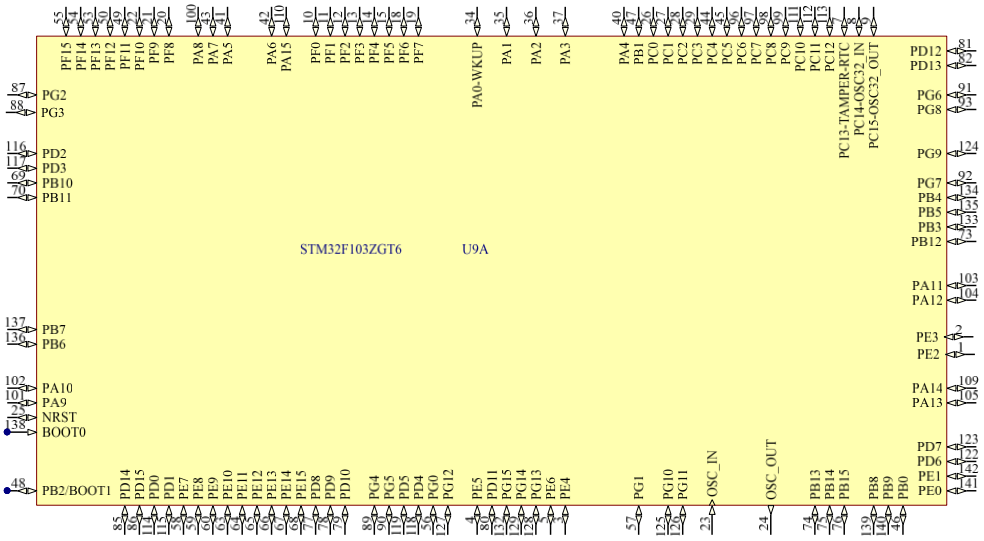


图 1-25 嵌入式实验开发板

（1）MCU

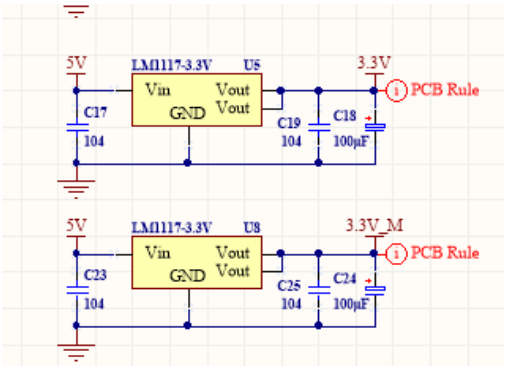
开发板选用的 MCU 是 STM32F103ZCT6，该芯片封装为 LQFP144，拥有的资源包括：

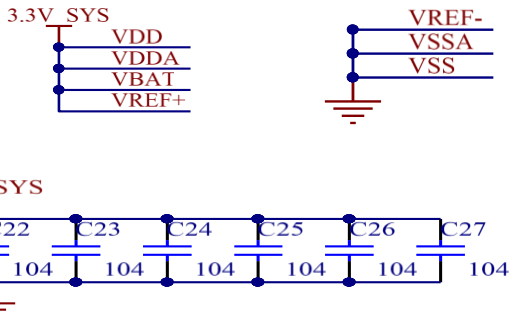


- 48KB SRAM、256KB FLASH、
- 2 个基本定时器、4 个通用定时器、2 个高级定时器
- 2 个 DMA 控制器（共 12 个通道）
- 3个 SPI、2 个 IIC、5 个串口、1 个 USB、1 个 CAN、1 个 SDIO 接口
- 3 个 12 位 ADC、2个 12 位 DAC、
- 1 个 FSMC 接口
- 112个通用 IO 口。
- 72Mhz

(2)电源电路

后备电源、CPU 电源和参考电源均接 3.3V，在设计中采用电解电容和瓷片电容抗干扰。

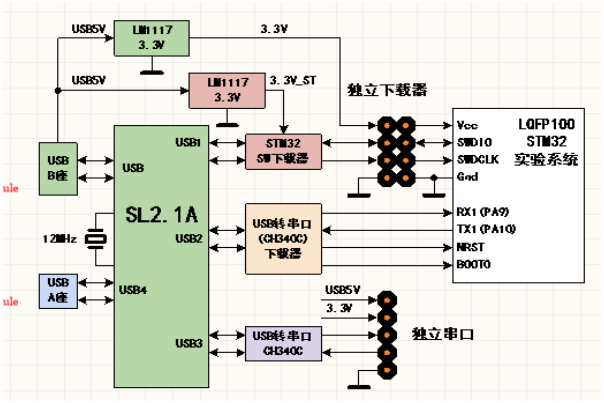


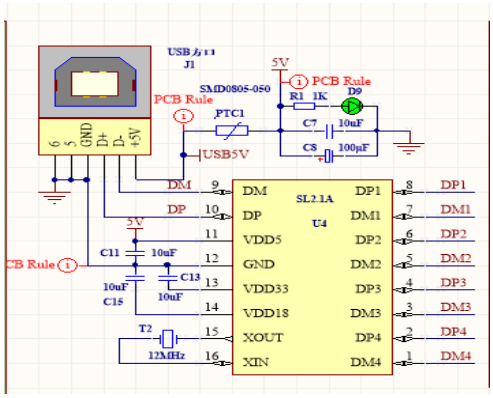


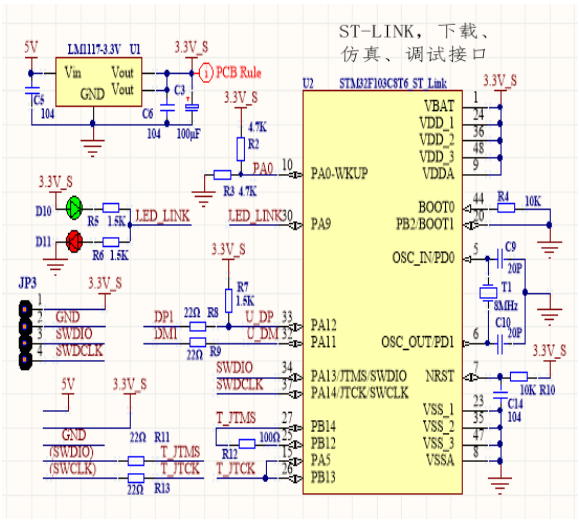
（3）USB、USB 转串口、SWD 下载

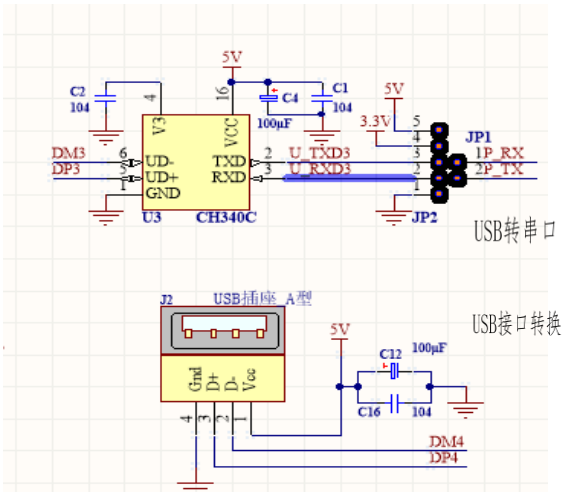
PA10 为 MCU 串口 1 的 RX 脚， PA9 为 MCU 串口 1 的 TX 脚，分别与 CH340G 的 TXD 和 RXD 相连接，即可实现串口通信。

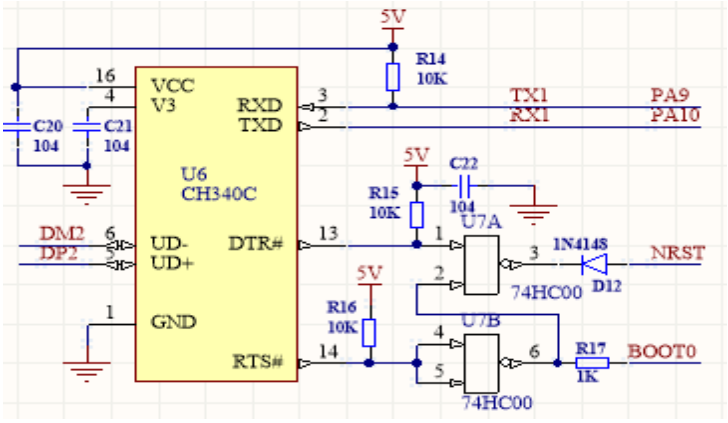
USB HUB，1路USB输入，四路USB输出。





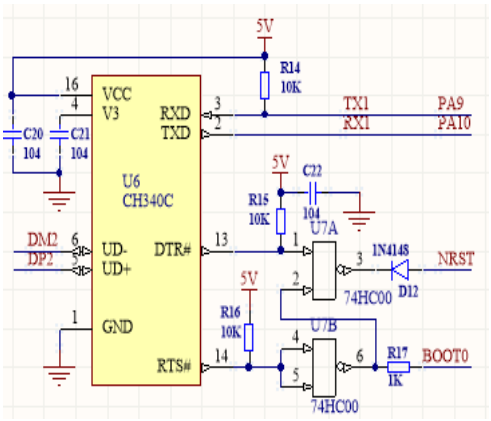






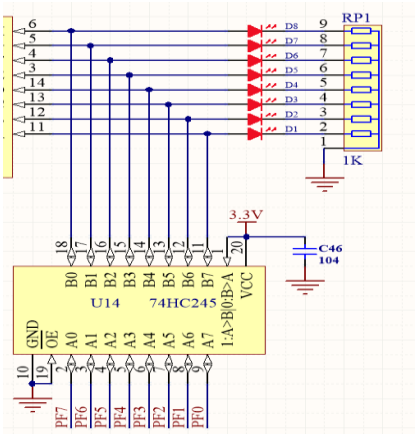
USB转串口， PA10为MCU串口1的RX脚，PA9为MCU串口1的TX脚，分别与CH340G的TXD和RXD相连接，即可实现串口通信。

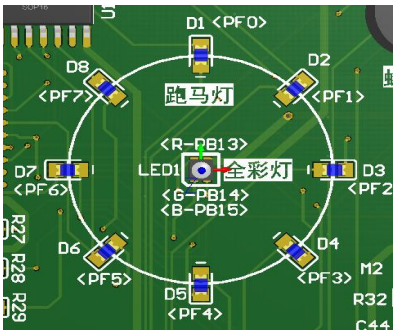
（4）复位电路



一键下载流程（程序控制）：
（1）、DTR=1，RTS=0，使NRST为低电平，MCU处于复位。
（2）、DTR=0，RTS=0，NRST为高电平，启动MCU，BOOT0=1，启动系统程序，用于串口下载。
（3）、 DTR=0，RTS=1，NRST为高电平，启动MCU，BOOT0=0，启动用户程序。

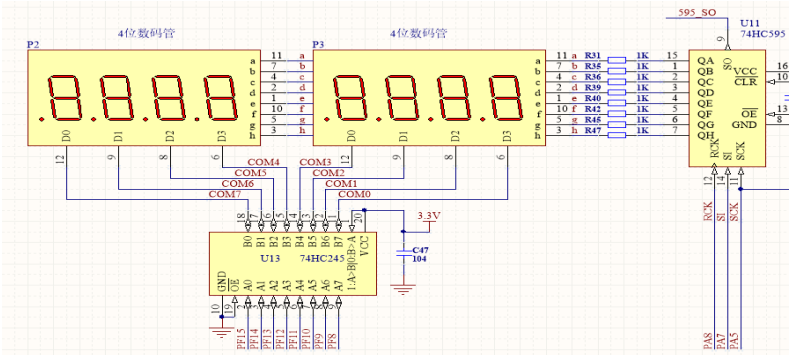
（5）旋转LED灯（跑马灯）



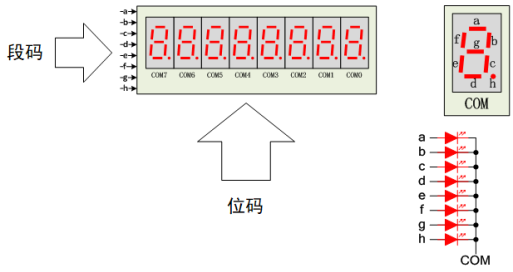


74HC245驱动，1K电阻限流。发光二极管D0-D7分别对应MCU引脚PF0-PF7。

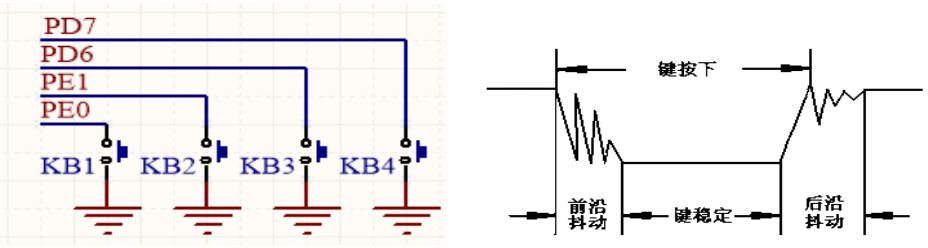
（6）数码管显示电路



数码管的显示数字（段码）由串行移位寄存器输出控制PA8,PA7,PA5。
数码管选择控制（位码）由PF8-PF15控制。

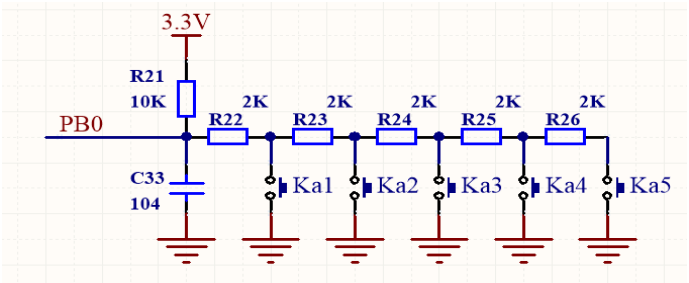


（7）独立按键



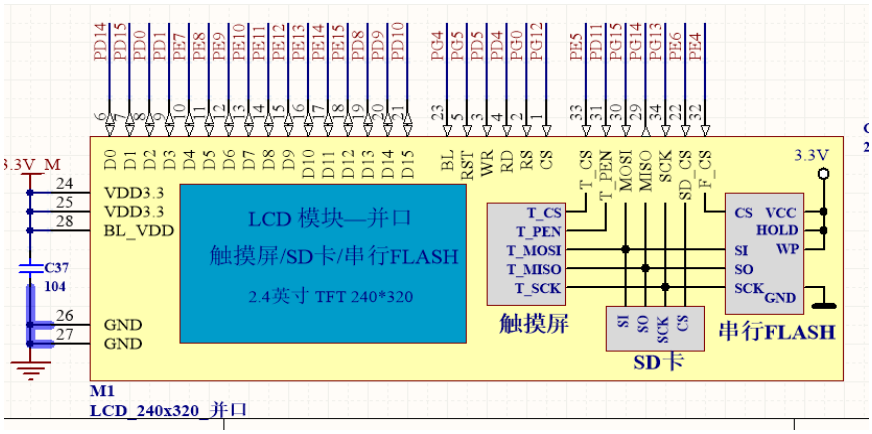
KB1、KB2、KB3、KB4 可用作普通按键输入或外部中断输入，分别连接在 PE0、PE1、PD6 和 PD7 上，并使用外部上拉电阻。在作为按键输入时，要注意消除按键的抖动。

（8）AD 按键

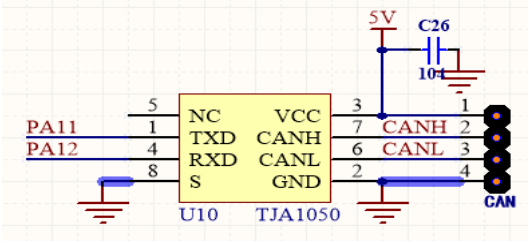


Ka1、Ka2、Ka3、Ka4、Ka5 可用作普通按键输入，按键按下后在 PB0 端产生的电压值不同，根据不同的电压值来判断是哪个按键按下。在作为按键输入时，要注意消除按键的抖动。

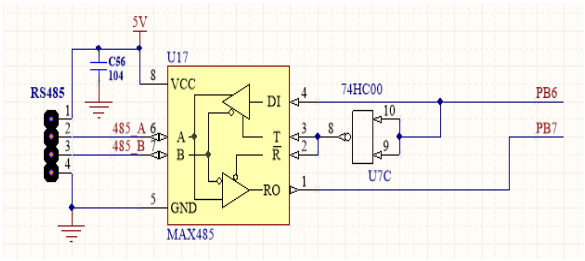
（9）显示屏



（10）CAN 接口、485 接口

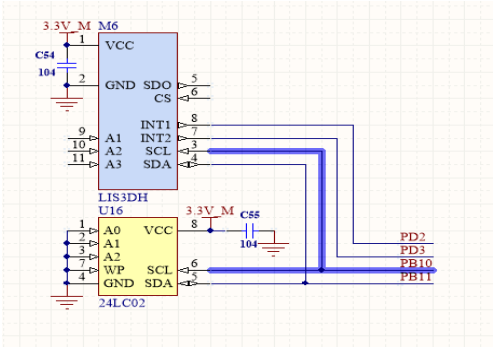


CAN 接口



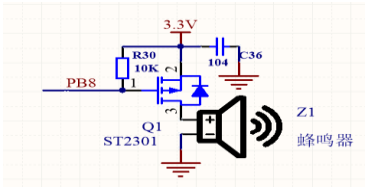
485 接口

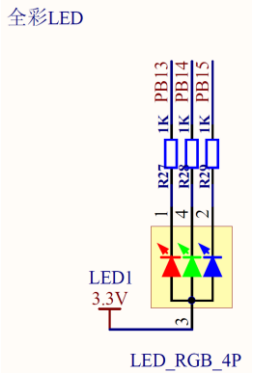
（11）IIC 接口



串行EEROM，2Kbit，
IIC接口

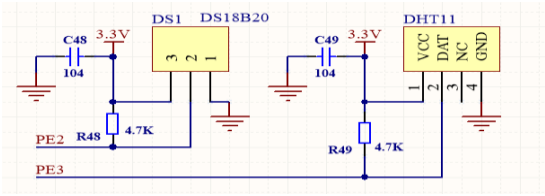
（12）蜂鸣器、全彩灯 RGB





有源蜂鸣器是指自带了震荡电路的蜂鸣器，这种蜂鸣器一接上电就会自己震荡发声。无源蜂鸣器，则需要外加一定频率（2~5Khz）的驱动信号，才会发声。这里我们选择使用无源蜂鸣器，方便大家使用。图中 Q1 是用来扩流，R38 则是一个下拉电阻，避免 MCU 复位的时候，蜂鸣器可能发声的现象。控制信号直接连接在 MCU 的 PB8 上面，PB8 可以做 PWM 输出（TIM4_CH3），所以大家如果想玩高级点（如：控制蜂鸣器“唱歌”），就可以使用 PWM 来控制蜂鸣器。

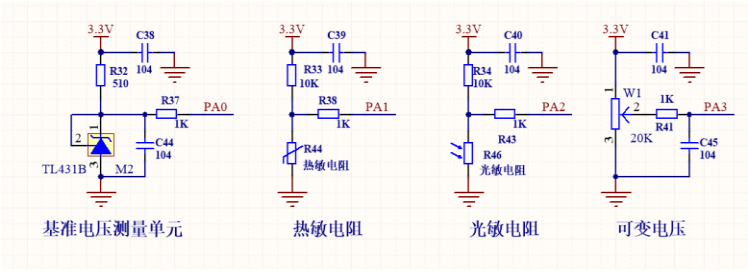
（13）温湿度传感器



DS18B20 是常用的数字温度传感器，其输出的是数字信号。特点：体积小、硬件开销低、抗干扰能力强、误差小。技术性能：单总线、多点组网、可寄生电源。 DHT11 是一款有已校准数字信号输出的温湿度传感器。 温度误差 2%，湿度误差 5%。量程湿度 20-90%RH，

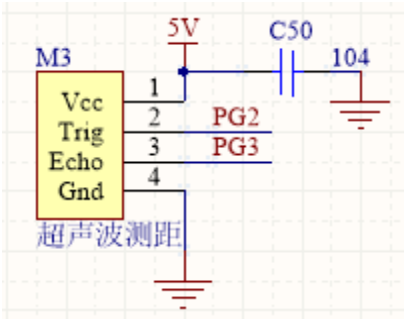
温度 0~50℃。

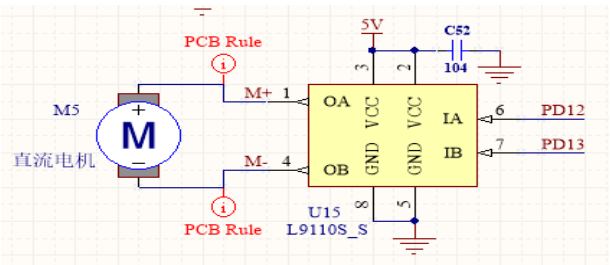
（14）AD 采样传感器



W1 为 0-20K 可调变位器，使 PA3 可采样 0-3.3V 电压，TL431 位 2.5V 电压基准，M4 为光敏传感器，其实就是一个光敏二极管，周围环境越亮，电流越大，反之电流越小，即可等效为一个电阻，M5 为热敏电阻，温度变化导致 M5 的阻值变化。

（15）超声波、电机模块

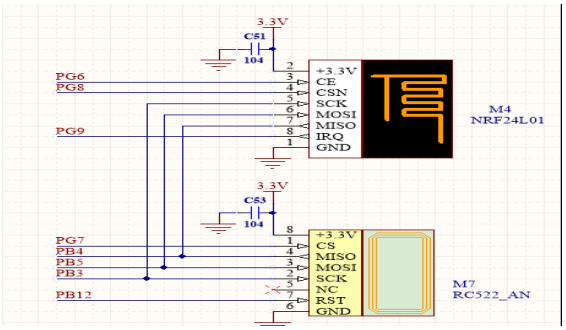




超声波测距模块是用来测量距离的一种产品，通过发送和就接收超声波，利用时间差和声音传播速度，计算出模块到前方障碍物的距离。

L9110S 是为控制和驱动电机设计的两通道推挽式功率。两个输出通道，每通道能通过 800mA 的持续电流，峰值电流能力可达 1.5A。

（16）2.4G 无线模块、RFID 模块

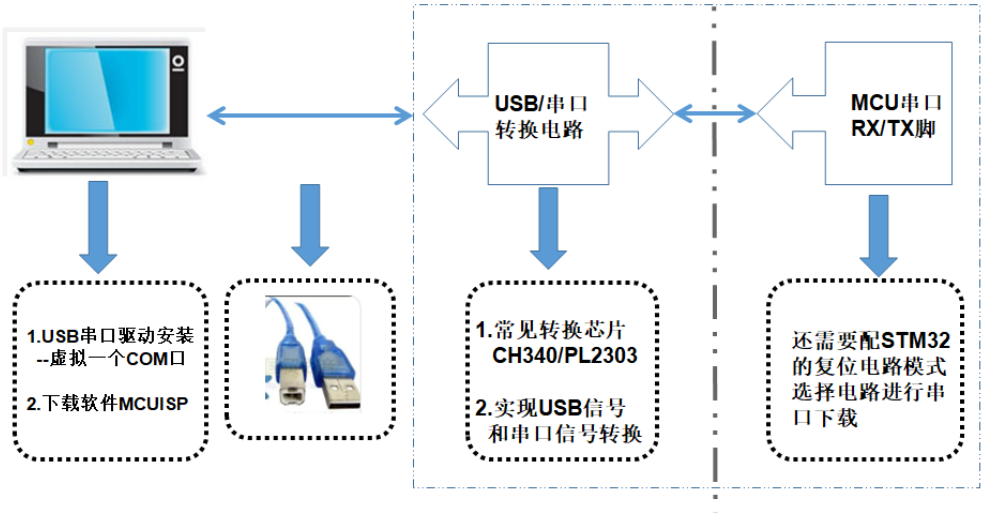


NRF24L01 为常用的无线模块，最大传输速度可以达到 2Mbps，传输距离最大可以到 30 米左右（空旷地，无干扰）.

MFRC522 是高度集成的非接触式（13.56MHz）读写卡芯片，此发送模块利用调制和调节的原理，并将它们完全集成到各种非接触式通信方法和协议中.

2、实验开发板测试

（1）硬件连接

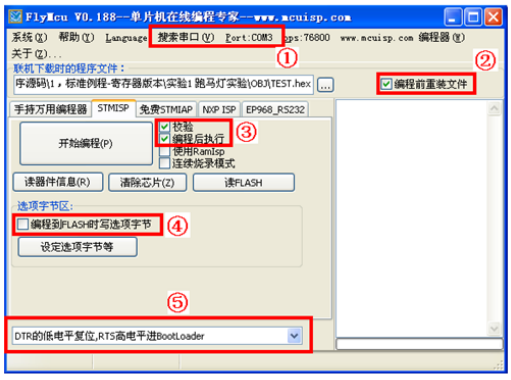


◆ 注意：

STM32的ISP下载，只能使用串口1，也就是对应串口发送接收引脚PA9,PA10。不能使用其他串口 （例如串口2：PA2,PA3)用来ISP下载。

（2）ISP 程序下载

启动工具包 FlyMCU，下载“2026 嵌入式实验开发板测试程序.hex”文件至开发板.同学们拿到的开发板已经写入该程序，可以直接测试。



下载工具配置:mcuisp(FlyMcu)

①搜索串口，选择虚拟出来USB串口：
COMx:空闲USB-SERIAL CH34
②勾选上：
“编程前重装文件”
③勾选上：
“校验”
“编程后执行”
④选项字节区 的“编程到FLASH时写选项直接”
不要勾上
⑤左下方：
选择第最后一个

（3）开发板测试

<a>按下“复位”键，观察并记录实验板的现象。

<b>按下“KB1”键，观察并记录实验板的现象。

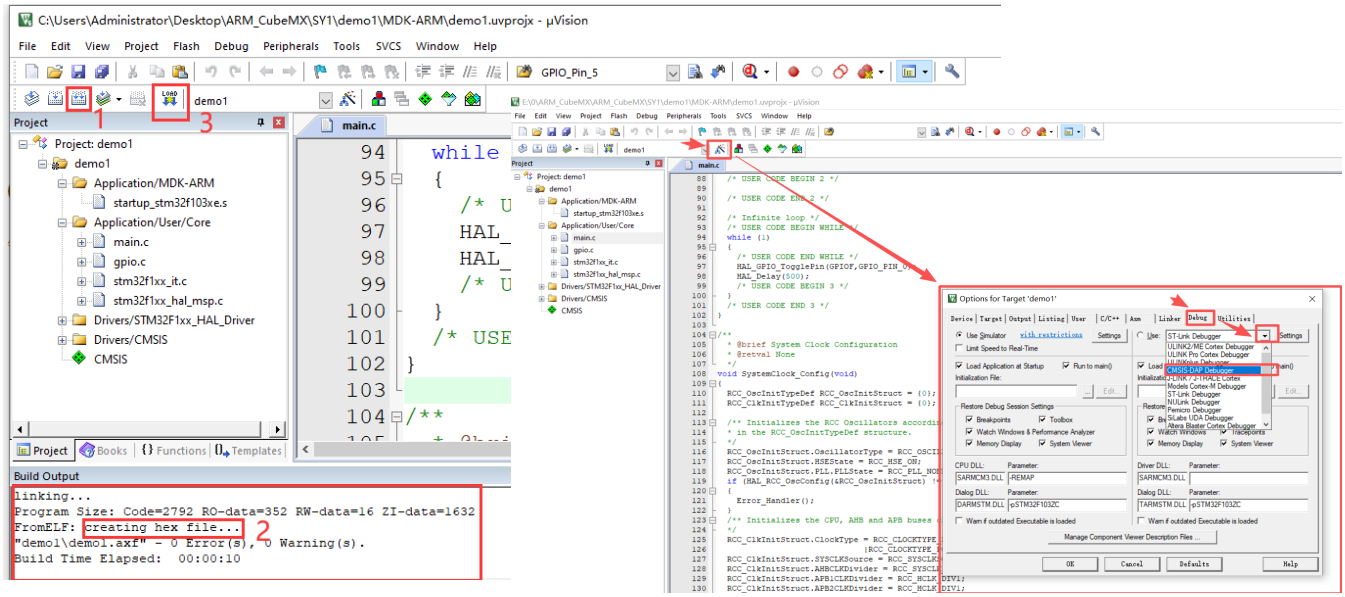
<c>按下“KB2”键，观察并记录实验板的现象。

<d>按下“KB3”键，观察并记录实验板的现象。

<e>按下“KB4”键，观察并记录实验板的状态。

(五)项目开发设计

1、启动 Keil uVision5,打开第（二）步中设计的工程，点击“编译”按钮，输出窗口出现“creating hex file…”字样，编译成功，连结实验板，设置下载方式为“CMSIS-DAPDebugger”,点击下载按钮，观察D1是否闪烁，如果没有按一下“复位”键。



电脑连结实验板，点击工具栏中“Debug 仿真”按钮  ，进入仿真界面，可以发现，多出了一个工具条，这就是 Debug 工具条，这个工具条在我们仿真的时候是非常有用的，下面简单介绍一下 Debug 工具条相关按钮的功能。

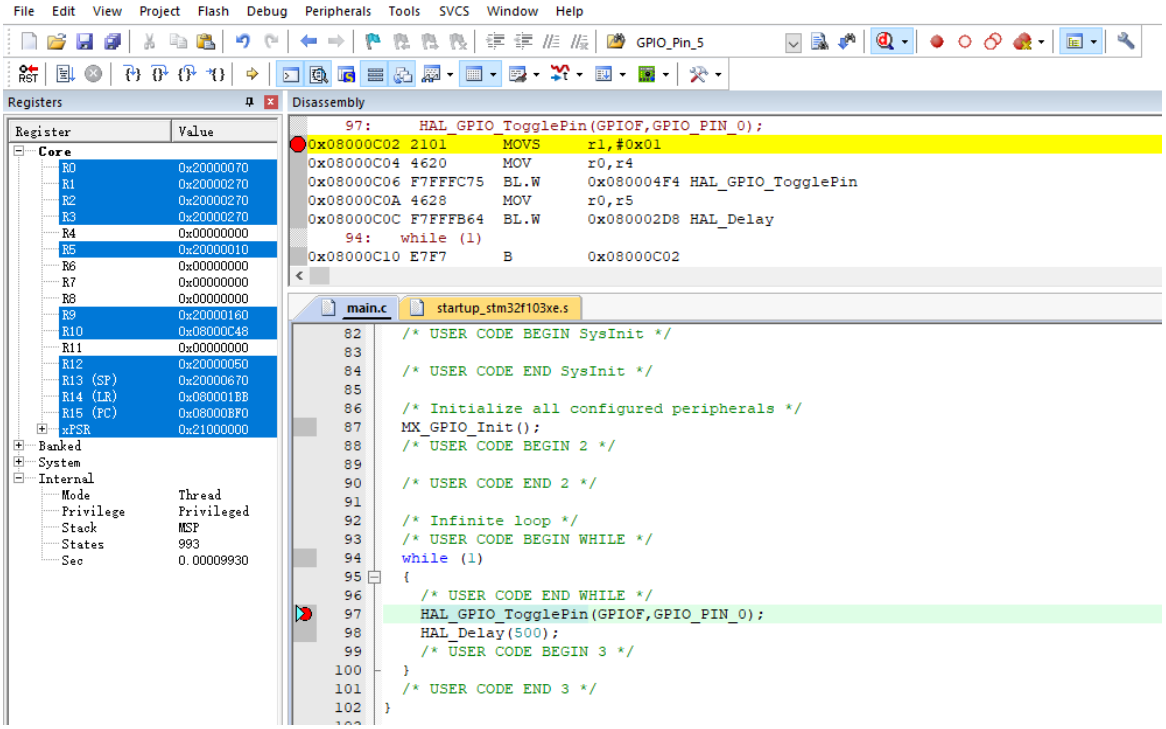




图 1-6 Debug 工具条

复位：其功能等同于硬件上按复位按钮。相当于实现了一次硬复位。按下该按钮之后，代码会重新从头开始执行。

执行到断点处：该按钮用来快速执行到断点处，有时候你并不需要观看每步是怎么执行的，而是想快速的执行到程序的某个地方看结果，这个按钮就可以实现这样的功能，前提是你在查看的地方设置了断点。

停止运行：此按钮在程序一直执行的时候会变为有效，通过按该按钮，就可以使程序停止下来，进入到单步调试状态。

执行进去：该按钮用来实现执行到某个函数里面去的功能，在没有函数的情况下，是等同于执行过去按钮的。

执行过去：在碰到有函数的地方，通过该按钮就可以单步执行过这个函数，而不进入这个函数单步执行。

执行出去：该按钮是在进入了函数单步调试的时候，有时候你可能不必再执行该函数的剩余部分了，通过该按钮就直接一步执行完函数余下的部分，并跳出函数，回到函数被调用的位置。

执行到光标处：该按钮可以迅速的使程序运行到光标处，其实是挺像执行到断点处按钮功能，但是两者是有区别的，断点可以有多个，但是光标所在处只有一个。

汇编窗口：通过该按钮，就可以查看汇编代码，这对分析程序很有用。

堆栈局部变量窗口：通过该按钮，显示 CallStack+Locals 窗口，显示当前函数的局部变量及其值，方便查看。

观察窗口：MDK5 提供 2 个观察窗口（下拉选择），该按钮按下，会弹出一个显示变量的窗口，输入你所想要观察的变量/表达式，即可查看其值，是很常用的一个调试窗口。

内存查看窗口：MDK5 提供 4 个内存查看窗口（下拉选择），该按钮按下，会弹出一个内存查看窗口，可以在里面输入你要查看的内存地址，然后观察这一片内存的变化情况。是很常用的一个调试窗口

串口打印窗口：MDK5 提供 4 个串口打印窗口（下拉选择），该按钮按下，会弹出一个

类似串口调试助手界面的窗口，用来显示从串口打印出来的内容。

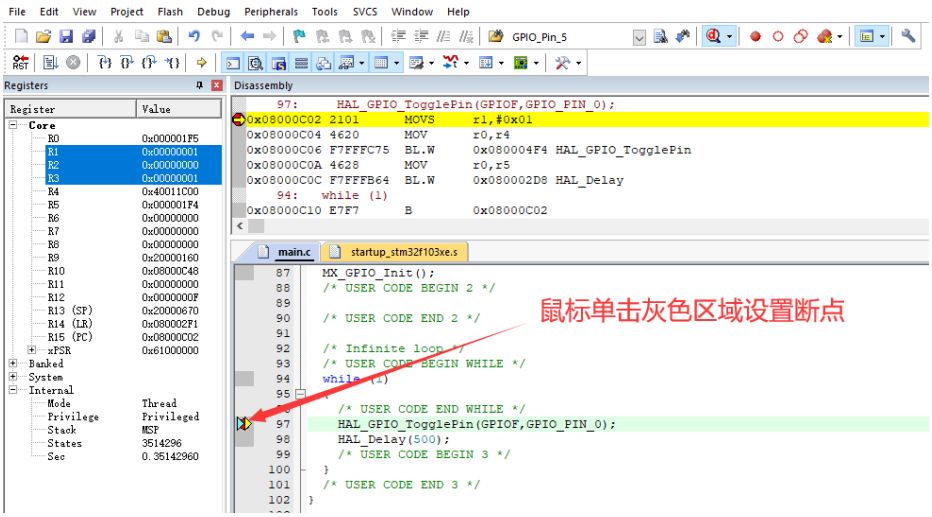
逻辑分析窗口：该图标下面有 3 个选项（下拉选择），我们一般用第一个，也就是逻辑分析窗口(LogicAnalyzer)，点击即可调出该窗口，通过 SETUP 按钮新建一些 IO 口，就可以观察这些 IO 口的电平变化情况，以多种形式显示出来，比较直观。

系统查看窗口：该按钮可以提供各种外设寄存器的查看窗口（通过下拉选择），选择对应外设，即可调出该外设的相关寄存器表，并显示这些寄存器的值，方便查看设置的是否正确。

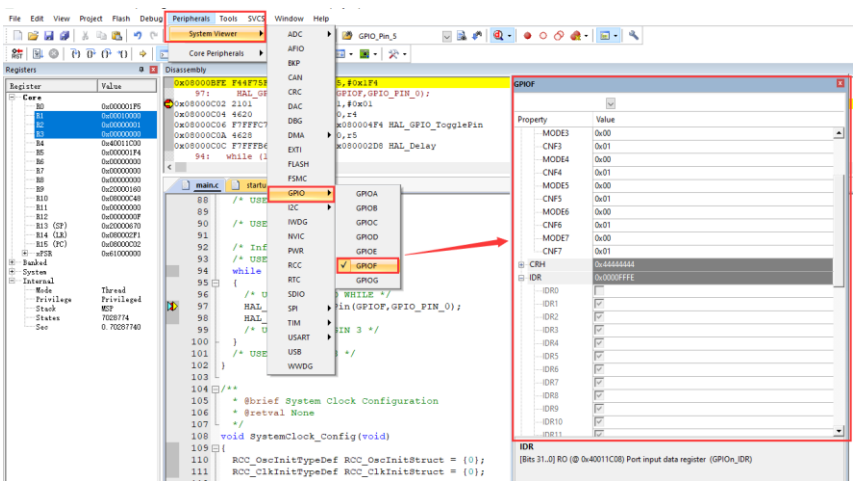
Debug 工具条上的其他几个按钮用的比较少，我们这里就不介绍了。以上介绍的是比较常用的，当然也不是每次都用得着这么多，具体看你程序调试的时候有没有必要观看这些东西，来决定要不要看。

仿真调试：

<a>设置断点，点击 run 按钮  ，观察实验板现象，再次点击 run 按钮…。



<b>启动 GPIOF 的观察窗口，点击 run 按钮，观察 GPIOF 各个寄存器的值，再次…。



<c>点击**单步运行**按钮  ，观察实验板现象,观察 GPIOF 各个寄存器的值，再次点击**单步运行**按钮…。

2、设计项目工程，D1、D2 循环交替点亮，调整延时间隔，直至双灯 D1、D2 均点亮。

3、设计项目工程，全彩灯 RGB 点亮,设计呈现呼吸灯显示效果。

4、设计项目工程，全彩灯 RGB 点亮,设计呈现全彩渐变的显示效果。

四、思考题

1、软件环境安装及配置过程中遇到的问题及解决方法？

2、基于实验开发板的外设接口及功能，在允许添加外接模块的情况下，设计一套应用于某工程的嵌入式系统，集数据采集、传输、存储、处理、控制、显示等于一体，你想设计那些功能？

3、实验收获？

实验二 GPIO 实验

一、实验目的

1. 了解 STM32 时钟系统和系统时钟。

2. 学习 GPIO 输入输出模式，掌握 GPIO 口配置寄存器的设置方法、掌握 GPIO 口 HAL 库的设置方法、掌握 GPIO 口库函数的设置方法。

3. 学会 GPIO 输入和输出数据,掌握基于 GPIO 控制跑马灯、数码管、蜂鸣器、电机的设计与应用。

4. 学习和掌握于串入并出、并入串出等外围电路的使用方法。

二、实验设备

1. 硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

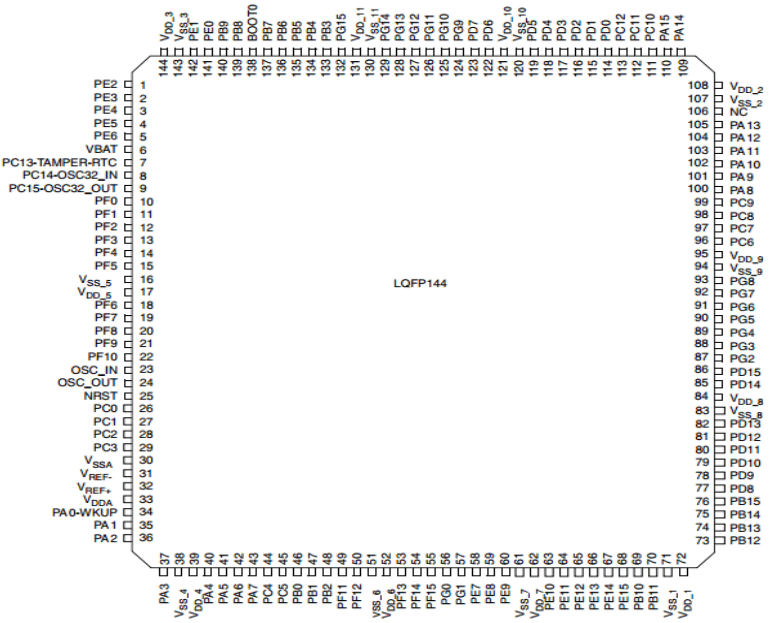
2. 配套软件系统：

MDK5、CubeMX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验原理

GPIO 端口内部是由一个个寄存器组成，一个引脚对应一个寄存器或多个引脚对应一个多位的寄存器，改变寄存器中的数据就可以改变外设的工作方式。与 51 单片机相比，STM32 拥有更多的 I/O 引脚，其驱动能力更强，控制方式更多更灵活，功能也更强大。

本课程使用 MCU **STM32F103ZET6** 一共有 144 个引脚，这些引脚共分为六大类：



电源引脚：VDD、VSS、VREF+、VREF-、VDDA、VSSA、、VBAT；

晶振引脚：PC14、PC15 和 OSC_IN、OSC_OUT；

复位引脚：NRST；

BOOT 引脚：BOOT0、BOOT1；

程序下载引脚：PA13、PA14、PA15、PB3 和 PB4；

GPIO 引脚：共有 7 组 GPIO，分别为：

PA 组：PA0~PA15；

PB 组：PB0~ PB15；

PC 组：PC0~ PC15；

PD 组：PD0~ PD15；

PE 组：PE0~PE15；

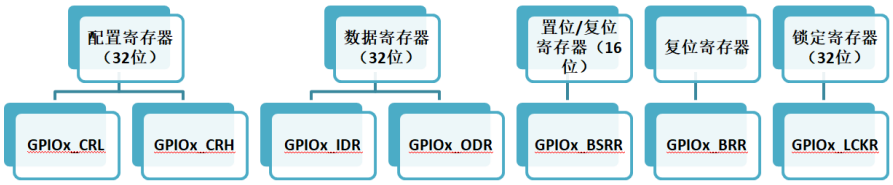
PF 组：PF0~PF15；

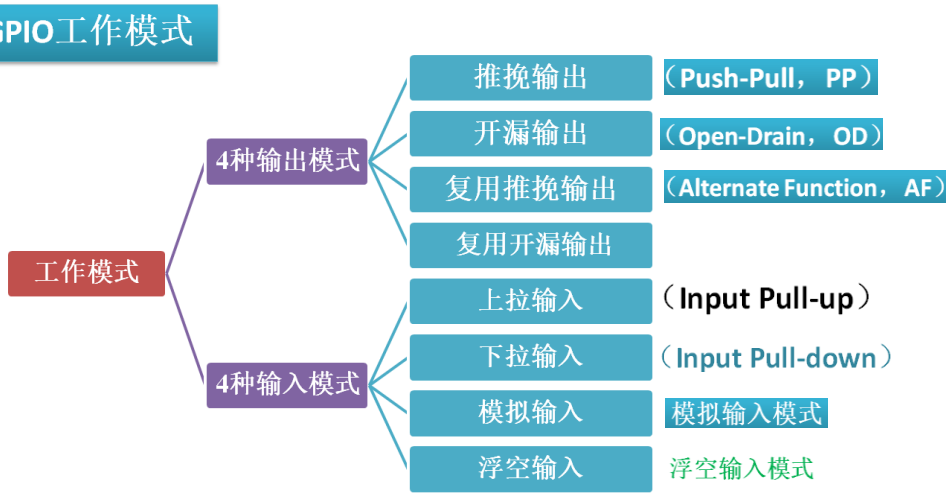
PG 组：PG0~PG15；

大多数引脚还通过复用技术兼具其他专用功能。

1、GPIO 相关配置寄存器

STM32F103ZET6 一共有 7 组 GPIO 口，每组 GPIO 端口(Px)都由 7 个寄存器组成，负责控制该端口的 16 个引脚 Px0 ~ Px15；每个引脚提供多种输入输出工作模式，根据应用需要配置相应配置寄存器。





（1）端口配置寄存器(GPIOx_CRL，GPIOx_CRH)

GPIOx_CRL

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Px0（例PA0）

Px15（例PA15）

GPIOx_CRH

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

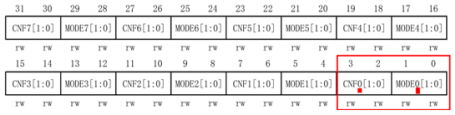


表18　输出模式位

MODE[1:0]	意义
00	保留
01	最大输出速度为10MHz
10	最大输出速度为2MHz
11	最大输出速度为50MHz

表17　端口位配置表

配置模式		CNF1	CNF0	MODE1	MODE0	PxODR寄存器
通用输出	推挽(Push-Pull)	0	0	01 10 11 见表18		0 或 1
	开漏(Open-Drain)		1			0 或 1
复用功能输出	推挽(Push-Pull)	1	0			不使用
	开漏(Open-Drain)		1			不使用
输入	模拟输入	0	0	00		不使用
	浮空输入		1			不使用
	下拉输入	1	0			0
	上拉输入					1

STM32F103 系列微控制器的 I/O 引脚的输出速度有 3 种选择：2MHz、10MHz 和 50MHz，一般常用的外设如 LED、蜂鸣器等建议采用 2MHz 的输出速度，而用作 IIC、SPI 等复用功能的输出引脚时，则尽量选择高响应速度如 10MHz 或 50MHz。

(2)数据输入寄存器（GPIOx_IDR）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位31:16	保留，始终读为0。
位15:0	**IDRy[15:0]**：端口输入数据(y = 0…15) (Port input data) 这些位为只读并只能以字(16位)的形式读出。读出的值为对应I/O口的状态。

（3）数据输出寄存器 GPIOx_ODR

Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy:** Port output data (y= 0 .. 15)

These bits can be read and written by software and can be accessed in Word mode only.

Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. G).

GPIOx_ODR数据输出寄存器的高16位为保留位，没有用到；低16位对应I/O端口相应的引脚，可进行置位或复位操作。

(4)端口位清除寄存器（GPIOx_BRR）

端口位清除寄存器**GPIOx_BRR**

Port bit reset register (GPIOx_BRR) (x=A..G)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved

Bits 15:0 **BRy:** Port x Reset bit y (y= 0 .. 15)

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Reset the corresponding ODRx bit

端口位清除寄存器**GPIOx_BRR**可以对端口数据输出寄存器**GPIOx_ODR**中的每位进行复位操作，即**GPIOx_BRR**只能改变引脚状态为低电平。

◆对**GPIOx_BRR**的低16位的某位置“1”，则对应的I/O端口引脚置输出低电平“0”；

◆对**GPIOx_BRR**的低16位的某位置“0”，则对应的I/O端口引脚状态不变。

(5)端口位设置/清除寄存器（GPIOx_BSRR）

置位/复位寄存器**GPIOx_BSRR**

Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy:** Port x Reset *bit y (y= 0 .. 15)*

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Reset the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy:** Port x *Set bit y (y= 0 .. 15)*

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Set the corresponding ODRx bit

◆ 对**GPIOx_BSRR**的低16位的某位置“1”，则对应的I/O端口引脚输出高电平“1”；

◆ 对**GPIOx_BSRR**的低16位的某位置“0”，则对应的I/O端口引脚状态不变；

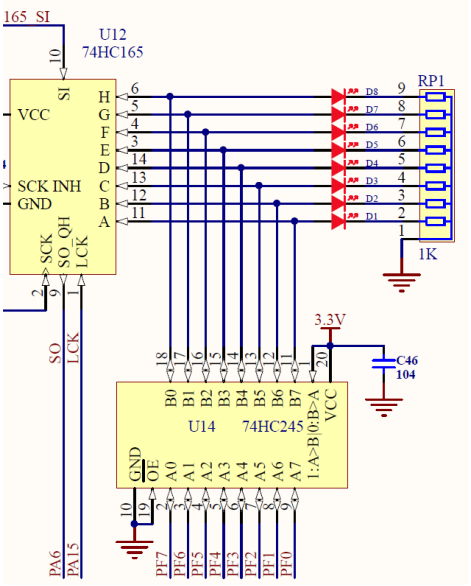
◆ 对**GPIOx_BSRR**的高16位的某位置“1”，则对应的I/O端口引脚输出低电平“0”；

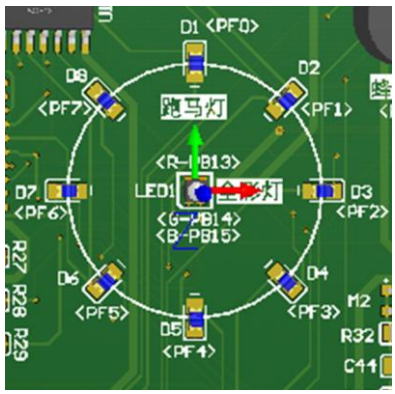
◆ 对**GPIOx_BSRR**的高16位的某位置“0”，则对应的I/O端口引脚状态不变；

GPIOx_BSRR寄存器的高16位称为清除寄存器，低16位称为置位寄存器。

2、基于寄存器的 GPIO 配置：首先使能 IO 口时钟，配置寄存器 RCC_APB2ENR；初始化 IO 口模式。配置寄存器 GPIOx_CRH/CRL；操作 IO 口，输出高低电平，配置寄存器 GPIOX_ODR 或者 BSRR/BRR。应用举例：8 个 LED 灯闪烁，使用 CubeMX 创建工程，只设置芯片选型、系统时钟、调试端口；GPIO 配置在程序中通过寄存器配置完成。

```
int main(void)
{
        //系统时钟设置在使用CubeMX创建工程时完成
  RCC->APB2ENR|=1<<7;                  //使能PORTF时钟
  GPIOF->CRL&=0X00000000;
  GPIOF->CRL|=0X33333333; //PD.0-PD7 推挽输出
  GPIOF->ODR=0xe000;                   //清除显示
  while(1)
  {
        GPIOF->ODR^=0xff;    //取反
        HAL_Delay (500);
  }
}
```





3、基于 HAL 库的 GPIO 开发：HAL 库接口函数的源码在源文件 stm32f1xx_hal_gpio.c 中，其对应的头文件 stm32f1xx_hal_gpio.h 声明了 GPIO 所有的库函数，共 8 种。

类型	函数原型	功能描述
初始化及复位函数	HAL_GPIO_Init()	GPIO初始化函数
	HAL_GPIO_DeInit()	复位选定的端口引脚到初始状态
引脚功能操作函数	HAL_GPIO_ReadPin()	读取选定的端口引脚的电平状态
	HAL_GPIO_WritePin()	设置选定的端口引脚输出高电平或低电平
	HAL_GPIO_TogglePin()	设置选定端口引脚的电平状态翻转
	HAL_GPIO_LockPin()	当端口引脚电平状态改变时保持锁定时的值
	HAL_GPIO_EXTI_IRQHandler()	外部中断处理函数
	HAL_GPIO_EXTI_Callback()	中断回调函数

GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)，功能是读取输入数据寄存器 IDR 的值与指定引脚进行按位与操作，结果若不为 0，则返回高电平；结果若为 0，则返回低电平。GPIOx 为引脚的端口号，取值为 GPIOA~GPIOG，表示指向 GPIO 结构体的指针，用于访问端口引脚的寄存器。

GPIO_Pin为常量
GPIO_Pin_0表示0x0001；
GPIO_Pin_15表示0x8000。

GPIO_PinState表示引脚电平状态，为枚举变量，

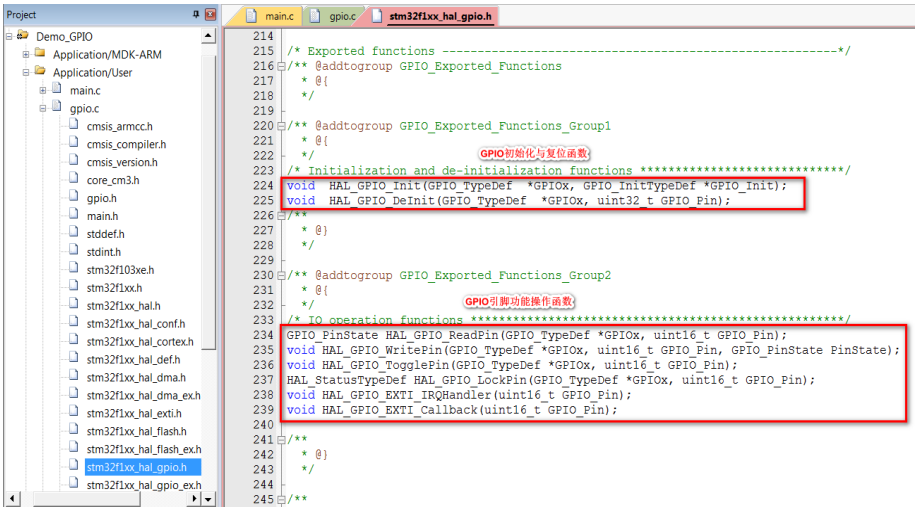
```
typedef enum
{
  GPIO_PIN_RESET = 0u,
  GPIO_PIN_SET
} GPIO_PinState;
```

取值范围为GPIO_PIN_SET=1或者GPIO_PIN_RESET=0。

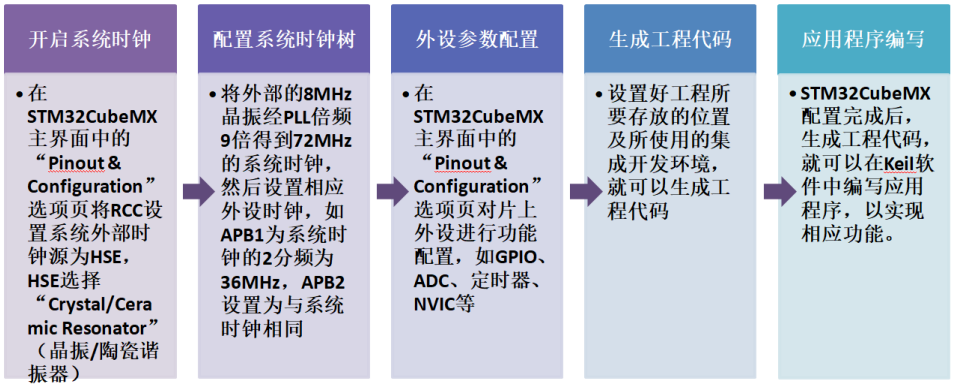
引脚号	引脚定义寄存器地址
GPIO_PIN_0	((uint16_t) 0x0001
GPIO_PIN_1	(uint16_t) 0x0002
GPIO_PIN_2	0x0004
GPIO_PIN_3	0x0008
GPIO_PIN_4	0x0010
GPIO_PIN_5	0x0020
GPIO_PIN_6	0x0040
GPIO_PIN_7	0x0080
GPIO_PIN_8	0x0100
GPIO_PIN_9	0x0200
GPIO_PIN_10	0x0400
GPIO_PIN_11	0x080
GPIO_PIN_12	0x1000
GPIO_PIN_13	0x2000
GPIO_PIN_14	0x4000
GPIO_PIN_15	0x8000
GPIO_PIN_All	0xFFFF

void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)，函数功能：将指定引脚的电平状态写入置位/复位寄存器 BSRR 中，该寄存器为 32 位，其中高 16 位控制端口 16 个引脚（0~15）输出低电平，低 16 位控制端口 16 个引脚输出高电平。若设置的引脚电平状态不等于低电平，则将该引脚设置的状态位写入 BSRR 寄存器对应的低 16 位，否则，将该 GPIO_Pin 左移 16 位写入 BSRR 寄存器对应的高 16 位中。

void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin),函数功能：取反 GPIO 空指定位，读出输出数据寄存器 ODR 的值与指定引脚进行按位与操作，结果若为真，表明原来为高电平，则写入 BRR 寄存器（BRR 寄存器写入“1”有效，写入“0”不影响 ODR 的状态）；若结果为假，表明原来为低电平，则控制 BSRR 寄存器低 16 位部分做置位（置“1”）操作。



基于 HAL 库的开发流程：



四、实验内容

学习嵌入式开发板硬件电路，学习嵌入式开发板 IO 口配置。嵌入式开发板硬件电路按键及数码管部分配置如下表，学习和掌握基于寄存器/基于 HAL 库开发模式，选择某种开发模式实现如下功能。

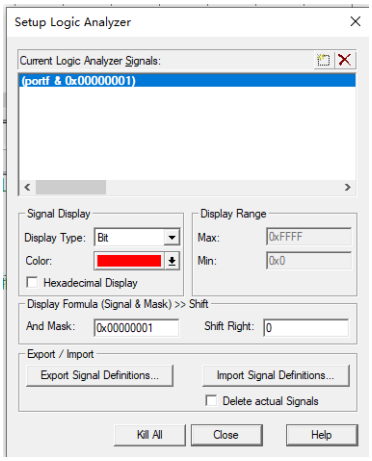
表 2-1　按键及数码管 IO 口配置模式

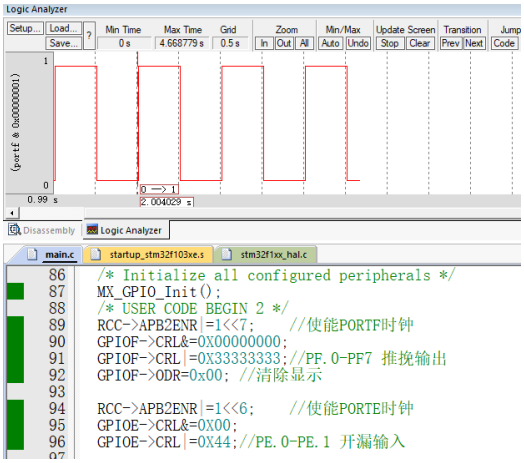
序号	基本端口	端口模式	基本功能/扩展功能	备注
1	PF0	推挽输出	跑马灯-D1	高电平点亮
2	PF1	推挽输出	跑马灯-D2	高电平点亮
3	PF2	推挽输出	跑马灯-D3	高电平点亮
4	PF3	推挽输出	跑马灯-D4	高电平点亮
5	PF4	推挽输出	跑马灯-D5	高电平点亮
6	PF5	推挽输出	跑马灯-D6	高电平点亮
7	PF6	推挽输出	跑马灯-D7	高电平点亮
8	PF7	推挽输出	跑马灯-D8	高电平点亮
9	PF8	推挽输出	数码管位码	低电平选中
10	PF9	推挽输出	数码管位码	低电平选中
11	PF10	推挽输出	数码管位码	低电平选中
12	PF11	推挽输出	数码管位码	低电平选中
13	PF12	推挽输出	数码管位码	低电平选中
14	PF13	推挽输出	数码管位码	低电平选中
15	PF14	推挽输出	数码管位码	低电平选中
16	PF15	推挽输出	数码管位码	低电平选中
17	PA5	推挽输出	串行发送脉冲	脉冲
18	PA7	推挽输出	数码管位码串行发送	0/1 数据位
19	PA8	推挽输出	595 锁存脉冲	脉冲
20	PB13	推挽输出	全彩灯-R 灯	低电平点亮
21	PB14	推挽输出	全彩灯-G 灯	低电平点亮
22	PB15	推挽输出	全彩灯-B 灯	低电平点亮
23	PE0	输入	KB1	按下为低电平
24	PE1	输入	KB2	按下为低电平
25	PD6	输入	KB3	按下为低电平
26	PD7	输入	KB4	按下为低电平
27	PB8	推挽输出	蜂鸣器	一定频率方波

1. 设计 GPIO 实验项目 1，功能：当按键 KB1 按下时，实验板上全彩发光二极管周边的发光二极管全亮，当按键 KB2 按下时跑马灯 D0 闪亮。

（1） 设计项目，编译、下载到开发板，按键 KB1\KB2 观察；

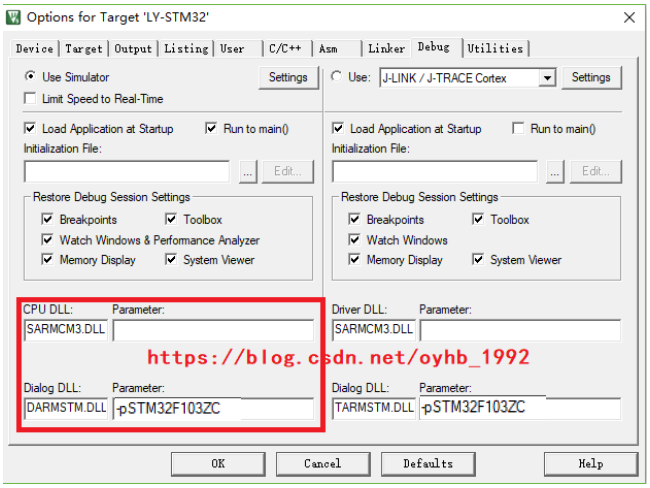
（2） 进行仿真调试，先按  开始仿真，接着按  ，显示逻辑分析窗口，点击 Setup，新建信号 PORTF.0。接着点击  开始运行。运行一段时间之后，按  按钮，暂停仿真回到逻辑分析窗口，可以看到如图所示的波形。





这里注意 Gird 要调节到 500ms 左右比较合适，可以通过 Zoom 里面的 In 按钮来放大波形，通过 Out 按钮来缩小波形，或者按 All 显示全部波形。从上图中可以看到 PORTF.0 循环输出高低电平，周期可以通过中间那根红线来测量。至此，我们的软件仿真已经顺利通过。在软件仿真没有问题了之后，我们就可以把代码下载到开发板上，看看运行结果是否与我们仿真的一致。

如果逻辑分析仪 Logic Analyzer 无法添加，或者无信号，则需要再工程设置中配置 debug 部分。



2、编写程序，实现顺时针方向旋转的跑马灯，跑马灯延时 0.2 秒。

3、编写程序，当按键 KB1 按下时，跑马灯按顺时针方向旋转，按下 KB2 时，跑马灯按逆时针方向旋转。

4、编写程序

（1）KB1 按下时，全彩 LED 中的发光二极管按红灯 100ms，绿灯 200ms，蓝灯 700ms，然后循环显示。

（2）KB2 按下时，全彩 LED 中的发光二极管按红灯 700ms，绿灯 100ms，蓝灯 200ms，然后循环显示。

（3）KB3 按下时，全彩 LED 中的发光二极管按红灯 200ms，绿灯 700ms，蓝灯 100ms，然后循环显示。

把时间缩小 10 倍、观察彩色 LED 颜色的变化；再缩小 10 被，再观察变化。

5、编写程序，控制蜂鸣器鸣叫，尝试改变蜂鸣器的声音高低。

6、编写程序，在数码管上显示自己的学号。

五、思考题

1、考虑如何实现长按键和按键双击识别，如何消除按键抖动？

2、全彩 LED 的颜色是怎么产生的，分析其原理。

3、蜂鸣器鸣叫是不同的频率发出不同的声音，请编程验证你能听见声音的频率范围。

实验三 外部中断实验

一、实验目的

1.学习中断的原理，掌握占先优先级和非占先优先级的区别。

2.了解实验板外部中断线路的连接方式。

3.掌握外部中断编程方法。

二、实验设备

1.硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

2.配套软件系统：

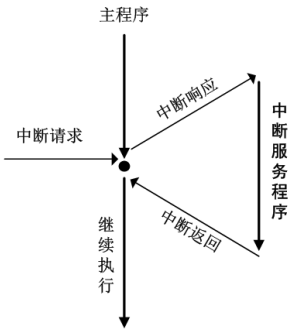
CubeMX、MDK5、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验原理

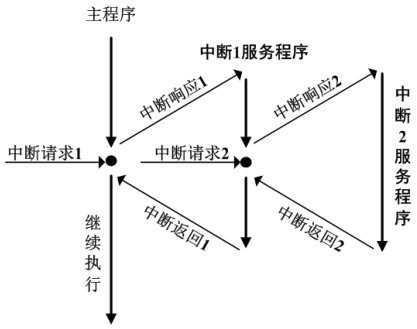
1、中断的概念

计算机在执行程序过程中，当出现异常情况（断电等）或特殊请求（数据传输等）时，计算机暂停现行程序的运行，转向对这些异常情况或特殊请求进行处理，处理完毕后再返回到现行程序的中断处，继续执行原程序，这就是“中断”。中断在嵌入式系统占有极其重要的地位，中断机制使得系统能更有效更合理地发挥效能和提高效率。

中断处理流程：中断请求、中断响应、中断服务和中断返回



单重中断的中断处理流程



多重中断的中断处理流程

ARM 公司设计的 Cortex-M3 内核可支持 256 个中断，15 个内核中断，240 个外部中断中断；具有 256 级的可编程中断优先级设置，即除了 3 个固定的高优先级（Reset、NMI、硬件失效），其他中断和异常的优先级可以由用户进行设置。**STM32F10x 系列**产品有 84 个中断通道，包括 16 个内核中断和 68 个可屏蔽中断；STM32F103 系列芯片只有 60 个可屏蔽中断；STM32F107 系列有 68 个可屏蔽中断。

2、中断向量表

中断向量表

当发生了异常或中断，内核要想响应这些异常或中断，就需要知道这些异常或中断的服务程序的入口地址，再由入口地址找到相应的中断服务程序，由中断入口地址组成的表称作中断向量表。

入口地址一般存放在程序存储器（ROM），默认情况下，Cortex-M3内核的中断向量表从零地址处开始，且每个向量占用4个字节。

向量编号	向量入口地址	说明
-	0x00000000	MSP的初始值
1	0x00000004	复位向量（PC初始值）
2	0x00000008	NMI异常服务程序的入口地址
3	0x0000000C	硬Fault异常服务程序的入口地址
…	…	其它中断服务程序的入口地址

外部中断向量表(部分)：

名称	说明	地址
EXTI0	EXTI 线 0 中断	0x00000058
EXTI1	EXTI 线 1 中断	0x0000005C
EXTI2	EXTI 线 2 中断	0x00000060
EXTI3	EXTI 线 3 中断	0x00000064
EXTI4	EXTI 线 4 中断	0x00000068
EXTI9_5	EXTI 线[9:5]中断	0x0000009C
EXTI15_10	EXTI 线[15:10]中断	0x000000E0
PVD	连接到 EXTI 的电源电压检查中断	0x00000044
RTC	实时时钟（RTC）全局中断	0x00000054
USB 唤醒	连接到 EXTI 的从 USB 唤醒中断	0x000000E8

3、中断优先级

void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)

NVIC_PriorityGroup	抢占优先级取值范围	响应优先级取值范围	描述
NVIC_PriorityGroup_0	0	0～15	抢占优先级占0位，响应优先级占4位
NVIC_PriorityGroup_1	0，1	0～7	抢占优先级占1位，响应优先级占3位
NVIC_PriorityGroup_2	0，1，2，3	0，1，2，3	抢占优先级占2位，响应优先级占2位
NVIC_PriorityGroup_3	0，1，2，3，4，5，6，7	0，1	抢占优先级占3位，响应优先级占1位
NVIC_PriorityGroup_4	0～15	0	抢占优先级占4位，响应优先级占0位

STM32 使用 Cortex-M3 的 8 位优先级寄存器中的 4 位来配置中断优先级，即 STM32 中的 NVIC 只支持 16 级中断优先级的管理，共 4 位，抢占优先级和响应优先级所占位数均不同，取值范围不同。高抢先优先级的中断可以打断低抢先优先级的中断服务，构成中断嵌套； 中断优先级的数值越小，优先级级别越高； 中断优先级判断：先判断抢占优先级的

大小，如果抢占优先级相同，则比较响应优先级的大小，若抢占优先级和响应优先级均相同，则根据中断向量表中的顺序来决定。

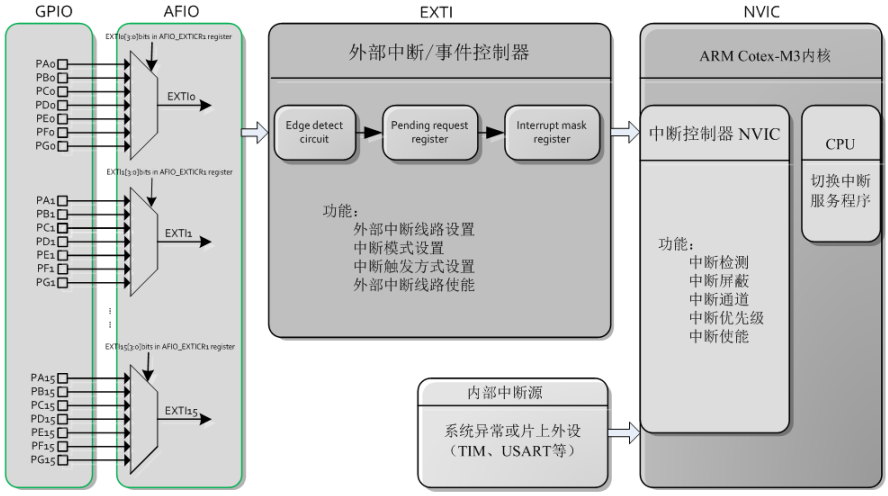
4、中断服务程序

STM32 将中断服务程序统一放在外设库 stm32f10x_it.c 文件中，其中的每个中断服务函数都只有函数名，函数体都是空的，需要用户自己编写相应的函数体，但**中断服务程序的函数名是不能更改**。

5、STM32 外部中断

STM32 芯片之外的外设的中断（I/O 端口）由 EXTI 和 NVIC 共同负责，即 STM32 的每一个 GPIO 引脚都可以配置成一个外部中断源，EXTI（External interrupt/event controller，外部中断/事件控制器）**支持 19 个外部中断/事件请求，STM32 中每一个 GPIO 都可以触发一个外部中断、PVC 输出、RTC 闹钟事件、USB 唤醒事件。**每个中断/事件都有独立的触发和屏蔽设置，具有中断模式和事件模式两种设置模式。

STM32的内部中断处理机制

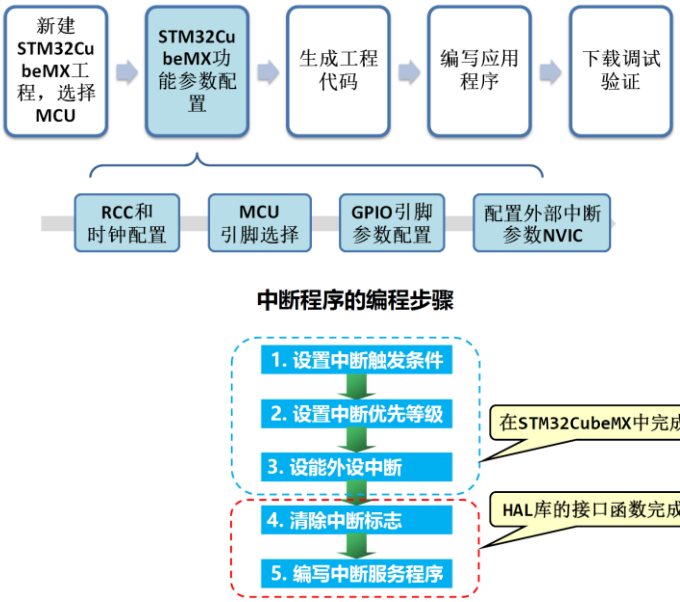


GPIO 的中断是以组为单位的，同组的外部中断公用一条外部中断线。GPIO 的中断是以组为单位的，同组的外部中断公用一条外部中断线。

中断标志	中断服务函数
EXTI0	void EXTI0_IRQHandler(void)
EXTI1	void EXTI1_IRQHandler(void)
EXTI2	void EXTI2_IRQHandler(void)
EXTI3	void EXTI3_IRQHandler(void)
EXTI4	void EXTI4_IRQHandler(void)
EXTI5-9	void EXTI9_5_IRQHandler(void)
EXTI10-15	void EXTI15_10_IRQHandler(void)

- **EXTI0-EXTI4**这5个外部中断有着各自独立的中断服务函数，如**EXTI3**的中断服务函数为 void EXTI3_IRQHandler(void);
- **EXTI5-9**共用一个中断服务函数 void EXTI9_5_IRQHandler(void);
- **EXTI10-15**共用一个中断服务函数 void EXTI15_10_IRQHandler(void);

6、外部中断开发流程



程序可参考 stm32f1xx_it.c 文件，void EXTI0_IRQHandler(void)是外部中断 0 的服务函数，负责 KB1 按键的中断检测；void EXTI1_IRQHandler(void)是外部中断 1 的服务函数，负责 KB2 按键的中断检测；void EXTI15_10_IRQHandler(void)是外部中断 6,7 的服务函数，负责 KB3，KB4 按键的中断检测；下面我们列出中断线 0、1 的相关配置代码：

```
void EXTIX_Init(void)   //中断初始化，如果使用 CubeMX，则在系统在 GPIO 的初始化中自动完成
{
    /*Configure GPIO pins : PE0 PE1 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;      //外部中断源引脚选择
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;    //下降沿触发
  GPIO_InitStruct.Pull = GPIO_PULLUP;      //上拉输入
  HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI0_IRQn, 3, 3);    //外部中断 0 抢占优先级为 3，响应优先级为 3
  HAL_NVIC_EnableIRQ(EXTI0_IRQn);    //使能外部中断 0

  HAL_NVIC_SetPriority(EXTI1_IRQn, 3, 2);      //外部中断 1 抢占优先级为 3，响应优先级为 2
  HAL_NVIC_EnableIRQ(EXTI1_IRQn);    //使能外部中断 1
}

void EXTI0_IRQHandler(void) //  外部中断 0 服务程序
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}

void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)    //外部中断通用处理函数
```

```
{
  /* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)   //检查中断标志
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);     //清除中断标志
    HAL_GPIO_EXTI_Callback(GPIO_Pin);           //执行中断回调函数
  }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    //用户自定义中断回调函数的处理任务
}
```

7、实验开发板外部中断接口电路

学习嵌入式开发板硬件电路外部中断部分，学习嵌入式开发板 IO 口配置，见表 3-1。

表 3-1　按键及外部中断输入口配置

序号	基本端口	端口模式	基本功能/扩展功能	备注
1	PE0	输入	KB1（外部中断输入）	按下为低电平
2	PE1	输入	KB2（外部中断输入）	按下为低电平
3	PD6	输入	KB3（外部中断输入）	按下为低电平
4	PD7	输入	KB4（外部中断输入）	按下为低电平

四、实验内容

1、编写程序，设置主程序：跑马灯以 0.2s 的速度旋转；将 KB1 设置为外部中断，下降沿触发，按下 KB1 则全彩灯的 R 灯闪烁 5 次。编译、下载程序到开发板，观察实验现象；按下 KB1，观察实验现象。

2、编写程序，设置主程序：跑马灯以 0.2s 的速度旋转；将 KB1，KB2 设置为外部中断，下降沿触发，按下 KB1 则 R 闪烁 5 次，按下 KB2 则 G 闪烁 5 次。编译、下载程序到开发板，观察实验现象；分别先后按下 KB1、KB2，观察实验现象。

3、设置主程序：跑马灯以 0.2s 的速度旋转；将 KB1，KB2 设置为外部中断，下降沿触发，按下 KB1 则 R 闪烁 5 次，按下 KB2 则 G 闪烁 5 次，设置 KB1 外部中断抢占先优先级设置为 1，KB2 外部中断抢占先优先级设置为 2。编译、下载程序到开发板，先按下 KB2，G 在闪烁过程中，再按下 KB1，观察实验现象。

4、设置 KB1 和 KB2 外部中断的抢占先优先级均为 1，KB1 外部中断的响应优先级为 1，KB2 外部中断响应优先级为 2。实验时先按下 KB2，LED1 在闪烁过程中，再按下 KB1，观察实验现象。

5、用短线连接 PE0 和 PE1，当 KB1 和 KB2 外部中断优先级设置不一致，按下 KB1 或 KB2 时，观察实验现象。

6、应用设计：

主程序为 RGB 循环点亮，外部中断 KB1,蜂鸣器播放门铃音，外部中断 KB2,电机顺时针旋转开门；KB2 的优先级高于 KB1。

五、思考题

1、中断服务程序中可否调用 HAL_Delay()函数，为什么？

2、列举生活中可能用到外部中断的事件。

实验四 异步通信实验

一、实验目的

1. 学习异步通信接口的原理，掌握串行口初始化方法。

2. 了解实验板串行线路的连接方式。

3. 掌握串行口通信编程方法。

二、实验设备

1. 硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

2. 配套软件系统：

MDK5、CubeMX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验原理

通信在嵌入式系统中的应用十分广泛，嵌入式微处理器的通信接口类型有很多，如 UART,SPI,I^2C,USB 和 CAN 等，其中 UART 是应用最方便、最普遍的一种通信接口。目前大多数微控制器都集成有 UART 接口，时嵌入式系统的常用外设。

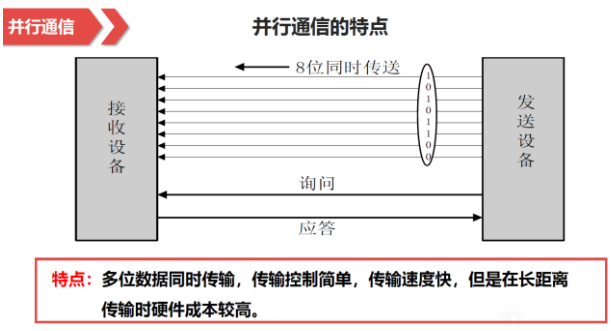
STM32 在其内部配备了功能强大的 USART 接口,支持 UART 功能,还支持同步单向通信、智能卡协议、调整解调器操作等。

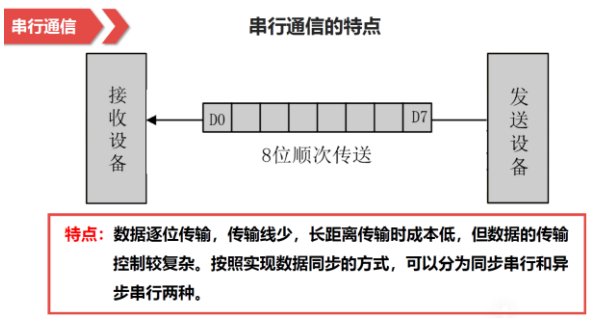
1、串行通信

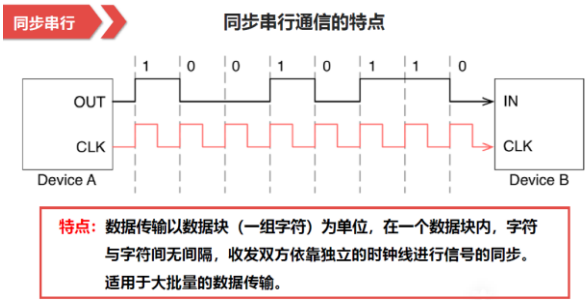
将计算机技术和通信技术相结合，完成计算机与外部设备或计算机与计算机之间的信息交换。按照数据传输方式的不同，可以分为串行通信和并行通信两大类。

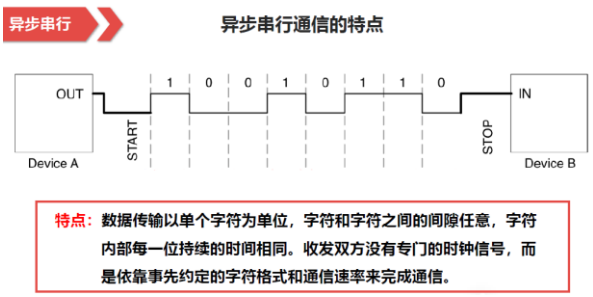
串行通信（Serial Transmission）：**是指通过一根数据线或少量数据线（少于 8 根）将数据一位一位地按顺序依次传送。**

并行通信（Parallel Transmission）：**是指用多条数据线同时传送多位数据，通常以 8 位、16 位、32 位等数据位传送。**







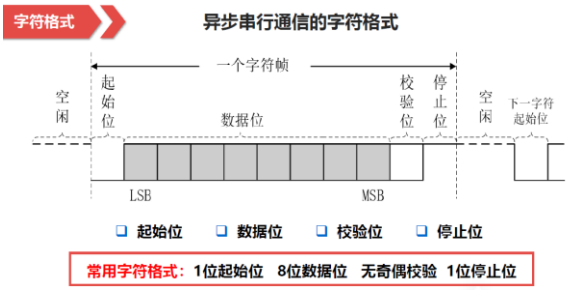


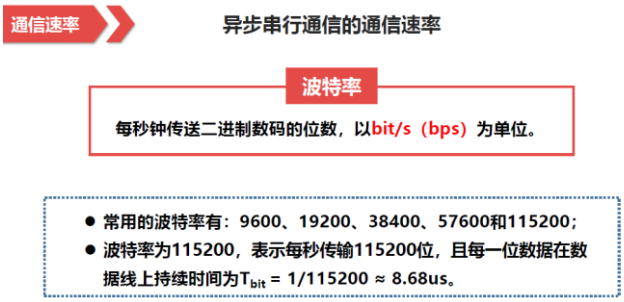
串行通信是目前最为流行的一种通信方式，如 U 盘以及采用 USB 接口的设备、工业上常用的 RS-485 和 RS-232 等，都是串行通信设备。计算机网络中所使用的传输方式均是串行传输，单片机与外设之间也大多采用各类串行接口，比如 USART、USB、IIC、SPI 等。

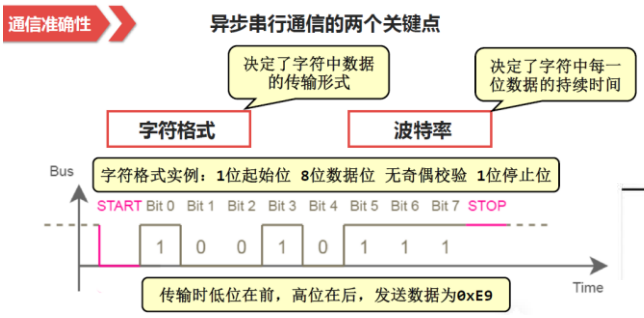
按照数据同步的方式分为：同步串行通信、异步串行通信。

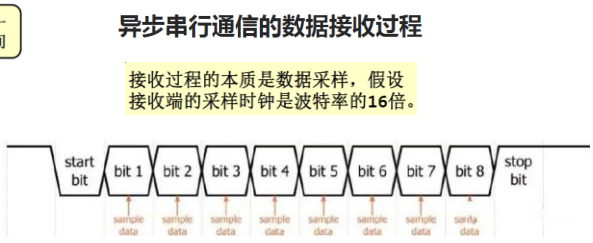
同步通信是连续串行传送数据的通信方式，要求收发双方的时钟必须保持严格的同步。特点：输速率较高。

异步通信在发送的有效数据中增加一些用于同步的控制位，比如开始位和停止位等，数据以字符为单位组成数据帧进行传送，收发双方需约定数据的传输速率。特点：传输效率较低；通信设备实现简单、成本低。









异步串行通信的数据接收过程：

①接收过程由起始位的下降沿启动；

②接收端等待 8 个时钟周期，以便建立一个接近比特周期中间的采样点；

③接收端等待 16 个时钟周期，使其进入第一个数据位周期的中点；

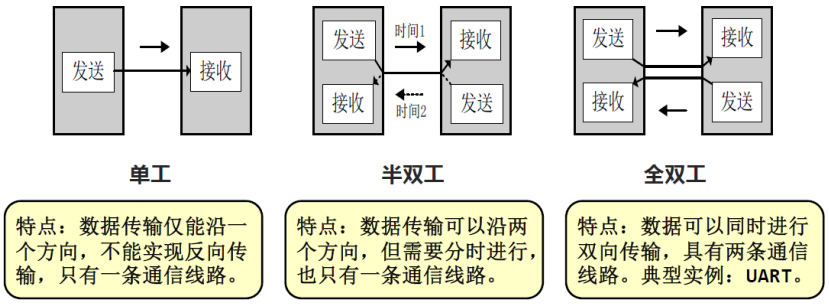
④第一个数据位被采样并存储在接收寄存器中；

⑤串口模块在采样第二个数据位之前等待另外 16 个时钟周期；

⑥重复此过程，直到所有数据位都被采样和存储；

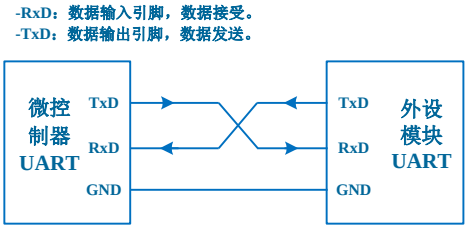
⑦由停止位的上升沿使数据线返回到空闲状态。

2、串口通信的数据传输方向



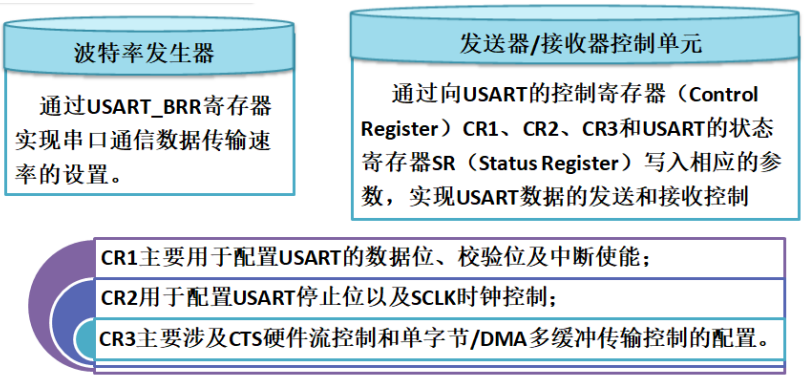
3、异步串行通信接口 UART

UART 至少需要两根数据线用于通信双方进行数据双向同时传输，最简单的 UART 接口由 TxD、RxD、GND 共 3 根线组成。其中，TxD 用于发送数据，RxD 用于接收数据，GND 为信号地线，通过交叉连接实现两个芯片间的串口通信。



4、STM32 的 USART 模块

（1）USART 内部功能实现由三部分构成：数据收发寄存器单元、波特率发生器、发送器/接收器控制单元。

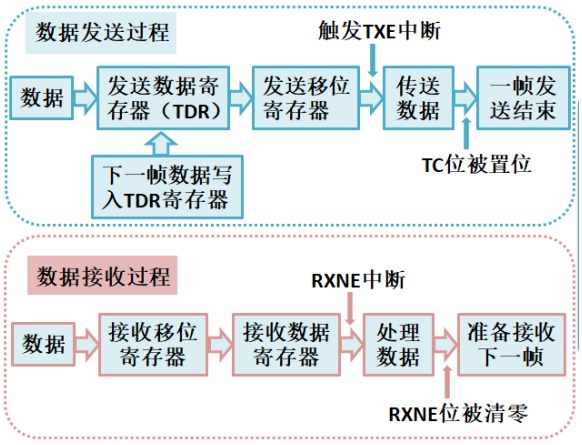


USART 状态寄存器 SR 各 bit 位所代表的含义：

bit位	寄存器名称	功能描述
31-10	保留，硬件强制设置为“0”	
9	CTS	CTS标志 0：nCTS状态线上没有变化；1：nCTS状态线上发生变化。
8	LBD	LIN断开检测标志（LIN break detection flag） 0：没有检测到LIN断开；1：检测到LIN断开。
7	TXE	发送数据寄存器空(Transmit data register empty) 0：数据还没有被转移到移位寄存器；1：数据已经被转移到移位寄存器。
6	TC	发送完成（Transmission complete） 0：发送还未完成；1：发送完成。
5	RXNE	读数据寄存器非空(Read data register not empty) 0：数据没有收到；1：收到数据，可以读出。

bit位	寄存器名称	功能描述
4	IDLE	监测到总线空闲（IDLE line detected） 0：没有检测到空闲总线；1：检测到空闲总线。
3	ORE	过载错误（Overrun error） 0：没有过载错误；1：检测到过载错误
2	NE	噪声错误标志（Noise error flag） 0：没有检测到噪声；1：检测到噪声。
1	FE	帧错误（Framing error） 0：没有检测到帧错误；1：检测到帧错误或者break符。
0	PE	校验错误（Parity error） 0：没有奇偶校验错误；1：奇偶校验错误。

数据收发寄存器单元组成：发送数据寄存器（TDR）、接收数据寄存器（RDR）、发送移位寄存器、接收移位寄存器。



STM32F103 大容量产品有 5 个串行通信接口，USART4 和 USART5 为 2 个通用异步收发器，不支持同步模式，只有异步通信功能。**USART 异步模式至少需要两个引脚：Rx（接收数据引脚）和 Tx（发送数据引脚），使用了复用 I/O 功能。**

总线	USARTx	复用功能	USART1_REMAP=0	USART1_REMAP=1	GPIO配置
APB2总线	USART1	USART1_TX	PA9	PB6	复用推挽输出
		USART1_RX	PA10	PB7	浮空输入
APB1总线	USART2	USART2_TX	PA2	PD5	复用推挽输出
		USART2_RX	PA3	PD6	浮空输入
	USART3	USART3_TX	PB10	PC10	复用推挽输出
		USART3_RX	PB11	PC11	浮空输入

（2）USART 的编程模式

轮询模式：CPU 不断地查询 I/O 设备是否准备就绪，如果准备就绪就发送，否则提示超时错误；会占用 CPU 的大量时间，效率低。通过定期或循环查询 USART 状态寄存器 SR 各位的状态，编写应用程序对接收或发送的数据进行处理。

中断方式：通过中断请求线，在 I/O 设备准备就绪时向 CPU 发出中断请求，CPU 中止正在进行的工作转向处理 I/O 设备的中断事件，中断方式相比轮询方式效率较高。

USART中断事件及中断使能标志位

中断标志	中断使能位	STM32中的定义	中断事件
TXE	TXEIE	UART_IT_TXE	发送数据寄存器空
CTS	CTSIE	UART_IT_CTS	CTS标志
TC	TCIE	UART_IT_TC	发送完成
RXNE	RXNEIE	UART_IT_RXNE	接收数据就绪（可读）
ORE	TXNEIE		检测到数据溢出
IDLE	IDLEIE	UART_IT_IDLE	检测到空闲线路标志
PE	PEIE	UART_IT_PE	奇偶检验错误标志
LBD	LBDIE	UART_IT_LBD	断开标志
NE或ORT或PE	EIE	UART_IT_ERR	噪声标志，溢出错误和帧错误

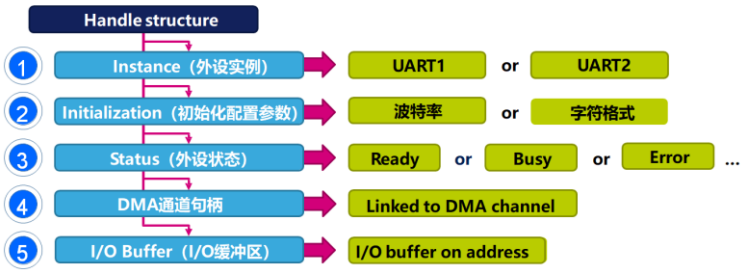
DMA 方式：直接存储器传送，不经过 CPU 直接在内存和外设之间进行批量数据交换，适用于高速大批量成组数据的传输；满足高速 I/O 设备的传输要求，有利于提高 CPU 的利用率。

类型		函数及功能描述
引脚功能操作函数	轮询方式	HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout); 功能描述：串口轮询模式发送函数，使用超时管理机制
		HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout); 功能描述：串口轮询模式接收函数，使用超时管理机制
	中断方式	HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口中断模式发送函数
		HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口中断模式接收函数
	DMA方式	HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口DMA模式发送函数
		HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size); 功能描述：串口DMA模式接收函数
		HAL_UART_DMAPause(UART_HandleTypeDef *huart); 功能描述：串口DMA模式暂停函数
		HAL_UART_DMAStop(UART_HandleTypeDef *huart); 功能描述：串口DMA模式停止函数
初始化及复位函数		HAL_UART_Init(UART_HandleTypeDef *huart) 功能描述：UART初始化函数
		HAL_UART_DeInit(UART_HandleTypeDef *huart) 功能描述：UART复位函数
中断处理函数（回调函数）		void HAL_UART_IRQHandler(UART_HandleTypeDef *huart); 功能描述：UART中断处理函数
		void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据发送完成时的回调函数，用户在该函数内编写中断处理程序
		void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据发送一半时的回调函数
		void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据接收完成时的回调函数，用户在该函数内编写中断处理程序
		void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart); 功能描述：UART数据接收一半时的回调函数
		void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart); 功能描述：UART传输出现错误时调用的回调函数
外设状态函数		HAL_UART_GetState(UART_HandleTypeDef *huart); 功能描述：获取串口状态函数
		HAL_UART_GetError(UART_HandleTypeDef *huart); 功能描述：获取串口错误函数

（3）USART 的串口数据结构、接口函数及应用

UART 模块的 HAL 库相关接口函数定义在源文件 stm32f1xx_hal_uart.c 中，在 stm32f1xx_hal_uart.h 头文件中可以查看相关库函数声明以及相关的结构体定义。

外设句柄数据类型的组成



串口句柄定义

```
typedef struct __UART_HandleTypeDef
{
  USART_TypeDef                 *Instance;      // 串口寄存器的基地址定义
  UART_InitTypeDef              Init;           // 串口初始化数据类型
  uint8_t                       *pTxBuffPtr;    // 串口发送缓冲区首地址
  uint16_t                      TxXferSize;     // 串口待发送数据个数
  __IO uint16_t                 TxXferCount;    // 串口发送数据计数器
  uint8_t                       *pRxBuffPtr;    // 串口接收缓冲区首地址
  uint16_t                      RxXferSize;     // 串口待接收数据个数
  __IO uint16_t                 RxXferCount;    // 串口接收数据计数器
  DMA_HandleTypeDef             *hdmatx;        // 串口发送的 DMA 通道句柄定义
  DMA_HandleTypeDef             *hdmarx;        // 串口接收的 DMA 通道句柄定义
  HAL_LockTypeDef               Lock;           // 保护锁类型定义
  __IO HAL_UART_StateTypeDef    gState;         // 串口全局状态和发送状态信息
  __IO HAL_UART_StateTypeDef    RxState;        // 串口接收状态信息
  __IO uint32_t                 ErrorCode;      // 串口错误代码
} UART_HandleTypeDef;
```

串口初始化

结构体类型，包括7个成员变量

```
typedef struct
{
  uint32_t BaudRate;        // 设置通信波特率
  uint32_t WordLength;      // 设置通信字符中数据位的位数
  uint32_t StopBits;        // 设置通信字符中停止位的位数
  uint32_t Parity;          // 设置奇偶校验模式
  uint32_t Mode;            // 设置接收或发送模式是否使能或禁能
  uint32_t HwFlowCtl;       // 设置硬件流控是否使能或禁能
  uint32_t OverSampling;    // 设置采样频率和信号传输频率的比例
} UART_InitTypeDef;
```

串口初始化函数

将通信参数存入句柄结构

```
void MX_USART2_UART_Init (void)
{
  huart2.Instance          = USART2;                // 配置串口 2
  huart2.Init.BaudRate     = 115200;                // 波特率 115200
  huart2.Init.WordLength   = UART_WORDLENGTH_8B;    // 数据位为 8 位
  huart2.Init.StopBits     = UART_STOPBITS_1;       // 停止位为 1 位
  huart2.Init.Parity       = UART_PARITY_NONE;      // 无奇偶校验
  huart2.Init.Mode         = UART_MODE_TX_RX;       // 使能接收和发送模式
  huart2.Init.HwFlowCtl    = UART_HWCONTROL_NONE;   // 无硬件流控
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;  // 16 倍过采样
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
}
```

执行串口初始化操作

主要函数：

接口函数：HAL_UART_Receive	
函数原型	HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
功能描述	在轮询方式下接收一定数量的数据
入口参数1	huart：串口句柄的地址
入口参数2	pData：存放接收数据的首地址
入口参数3	Size：待接收数据的个数
入口参数4	Timeout：超时等待时间，以ms为单位，HAL_MAX_DELAY表示无限等待
返回值	HAL状态值：HAL_OK表示接收成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用；HAL_TIMEOUT表示接收超时
注意事项	1. 该函数连续接收数据，在接收过程中通过判断RXNE标志来接收新的数据 2. 如果在超时时间内没有完成接收，则不再接收数据，返回超时标志 3. 该函数由用户调用

接口函数：HAL_UART_Transmit	
函数原型	HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
功能描述	在轮询方式下发送一定数量的数据
入口参数1	huart：串口句柄的地址
入口参数2	pData：待发送数据的首地址
入口参数3	Size：待发送数据的个数
入口参数4	Timeout：超时等待时间，以ms为单位，HAL_MAX_DELAY表示无限等待
返回值	HAL状态值：HAL_OK表示发送成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用；HAL_TIMEOUT表示发送超时
注意事项	1. 该函数连续发送数据，发送过程中通过判断TXE标志来发送下一个数据，通过判断TC标志来结束数据的发送 2. 如果在等待时间内没有完成发送，则不再发送，返回超时标志 3. 该函数由用户调用

1 串口中断方式发送函数：HAL_UART_Transmit_IT	
函数原型	HAL_StatusTypeDef HAL_UART_Transmit_IT (UART_HandleTypeDef *huart,uint8_t *pData, uint16_t Size)
功能描述	在中断方式下发送一定数量的数据（发送过程）
入口参数1	huart：串口句柄的地址（每发送一个数据进入一次中断）
入口参数2	pData：待发送数据的首地址（在中断中根据发送数据的个数来判断数据是否发送完成）
入口参数3	Size ：待发送数据的个数
返回值	HAL状态值：HAL_OK表示发送成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用
注意事项	1. 函数将使能串口发送中断 2. 函数将置位TXEIE和TCIE，使能发送数据寄存器空中断和发送完成中断。完成指定数量的数据发送后，将会关闭发送中断，即清零TXEIE和TCIE。因此用户采用中断方式连续发送数据时，需要重复调用该函数，以便重新开启发送中断 3. 当指定数量的数据发送完成后，将调用发送中断回调函数HAL_UART_TxCpltCallback进行后续处理 4. 该函数由用户调用。

2 串口中断方式接收函数：HAL_UART_Receive_IT	
函数原型	HAL_StatusTypeDef HAL_UART_Receive_IT (UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
功能描述	在中断方式下接收一定数量的数据（接收过程）
入口参数1	huart：串口句柄的地址（每接收一个数据进入一次中断）
入口参数2	pData：存放接收数据的首地址（在中断中根据接收数据的个数来判断数据是否接收完成）
入口参数3	Size ：待接收数据的个数
返回值	HAL状态值：HAL_OK表示接收成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用
注意事项	1. 函数将使能串口接收中断 2. 函数将置位RXNEIE，使能接收数据寄存器非空中断RXNE。完成指定数量的数据接收后，将会关闭接收中断，即清零RXNEIE。因此用户采用中断方式连续接收数据时，要重复调用该函数，以重新开启接收中断 3. 当指定数量的数据接收完成后，将调用接收中断回调函数HAL_UART_RxCpltCallback进行后续处理 4. 该函数由用户调用

3 串口中断通用处理函数：HAL_UART_IRQHandler

接口函数：HAL_UART_IRQHandler	
函数原型	void HAL_UART_IRQHandler(UART_HandleTypeDef *huart)
功能描述	作为所有串口中断发生后的通用处理函数
入口参数	huart：串口句柄的地址
返回值	无
注意事项	1. 函数内部先判断中断类型，并清除对应的中断标志，最后调用回调函数完成对应的中断处理 2. 该函数由CubeMX自动生成

4 串口发送中断回调函数：HAL_UART_TxCpltCallback

接口函数：HAL_UART_TxCpltCallback	
函数原型	void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
功能描述	回调函数，用于处理所有串口的发送中断，用户在该函数内编写实际的任务处理程序
入口参数	huart：串口句柄的地址
返回值	无
注意事项	1. 函数由串口中断通用处理函数HAL_UART_IRQHandler调用，完成所有串口的发送中断任务处理 2. 函数内部需要根据串口句柄的实例来判断是哪一个串口产生的发送中断 3. 函数由用户根据具体的处理任务编写

5 串口接收中断回调函数：HAL_UART_RxCpltCallback

接口函数：HAL_UART_RxCpltCallback	
函数原型	void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
功能描述	回调函数，用于处理所有串口的接收中断，用户在该函数内编写实际的任务处理程序
入口参数	huart：串口句柄的地址
返回值	无
注意事项	1. 函数由串口中断通用处理函数HAL_UART_IRQHandler调用，完成所有串口的接收中断任务处理 2. 函数内部需要根据串口句柄的实例来判断是哪一个串口产生的接收中断 3. 函数由用户根据具体的处理任务编写

6 串口中断使能函数：__HAL_UART_ENABLE_IT

接口函数：__HAL_UART_ENABLE_IT	
函数原型	__HAL_UART_ENABLE_IT(__HANDLE__, __INTERRUPT__)
功能描述	使能对应的串口中断类型
参数1	__HANDLE__：串口句柄的地址
参数2	__INTERRUPT__：串口中断类型，该参数几个常用的取值如下： UART_IT_TXE ：发送数据寄存器空中断 UART_IT_TC ：发送完成中断 UART_IT_RXNE：接收数据寄存器非空中断 UART_IT_IDLE ：线路空闲中断
返回值	无
注意事项	1. 该函数是宏函数，进行宏替换，不发生函数调用 2. 函数需要由用户调用，用于使能对应的串口中断类型

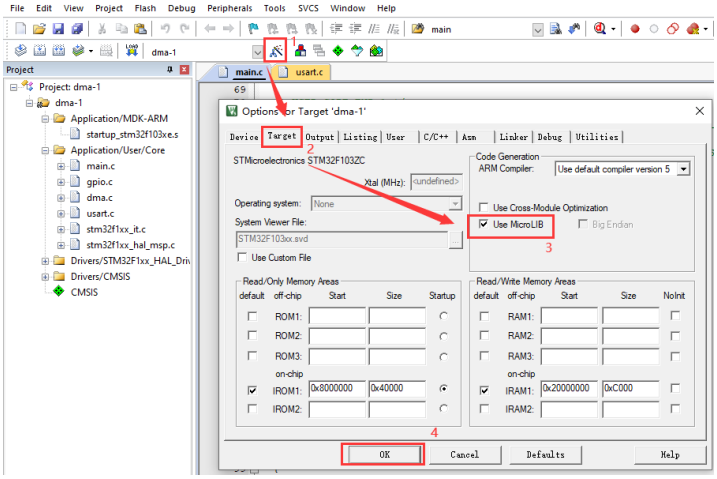
串口重定向：

在 C 语言中，printf()函数是将数据输出到屏幕，scanf()函数是将数据从键盘读入，在嵌入式 C 语言开发中需要重新定义 printf()函数和 scanf()函数，将标准的输入和输出重新定向到串口，才能实现串口输出或 LCD 输出，重定向 printf()函数和 scanf()函数也就是重写 fputc()函数和 fgetc()函数。重定向是指用户改写 C 语言的库函数，当链接器检查到用户编写了与 C 库函数同名的函数时，将优先使用用户编写的函数，从而实现对库函数的修改。

```
    uint8_t ch;
    uint8_t ch_r;
//重写这个函数,重定向 printf 函数到串口
    int fputc(int c, FILE * f)
    {
        ch=c;
```

```
        HAL_UART_Transmit(&huart1,&ch,1,1000);//发送串口
        return c;
    }
//重定向 scanf 函数到串口  意思就是说接受串口发过来的数据,fputc 函数重定义
    int fgetc(FILE * F)
    {
    HAL_UART_Receive (&huart1,&ch_r,1,0xffff);//接收
    return ch_r;
    }
```

特别提醒：在串口重定向的工程设计时，必须配置并勾选”Use Micro LIB”项。



四、实验内容

1、采用查询法编写串口程序，串行口波特率设置为 115200bps，数据字长 8 位，停止位 1 位，无校验。

(1)向串行口发送 1 个字符数据，开发板接收到数据后，将数据加一从串口发回。

(2)向串行口发送 1 个字符数据，开发板接收到数据后，判定数据为 Y 或者 y 跑马灯全亮、数据为 N 或者 n 跑马灯全灭，其他字符，跑马灯旋转。

(3)重定向 printf、scanf 函数到串口。

2、采用中断法编写串口程序，串行口波特率设置为 115200bps，数据字长 8 位，停止位 1 位，无校验。

(1)向串行口发送 1 个字符数据，开发板接收到数据后，将数据加一从串口发回。

(2)向串行口发送 1 个字符数据，开发板接收到数据后，判定数据为 Y 或者 y 跑马灯全亮、数据为 N 或者 n 跑马灯全灭，其他字符，将数据的 ASCII 码显示在数码管上。显示内容为 XXH。

3、构建发送和接收缓冲区，编写发送和接收单字节、双字节、四字节和字符串发送函数。将接收到的字符串从串口发回，并在数码管上显示你接收到的字符串。

4、采用中断法定义串口通讯协议，串行口波特率设置为 115200bps，数据字长 8 位，停止位 1 位，无校验。通过串行口向蜂鸣器、电机、跑马灯和数码管发送 5 字节命令，控制命令如下：

起始字节	数据字节 1	数据字节 2	校验字节	结束字节
0xAA	01：蜂鸣器	00：音乐 1 01：音乐 2	——	0xCC
	02：跑马灯	02：正向旋转 03：反向旋转		
0xBB	01：电机	01：正向旋转 02：反向旋转		0xCC
	02：数码管	03：显示 BB0203CC 04：显示 CC3020BB	——	

5、（**选做**）多机、多数据通信。采用中断法编写串口程序，串行口波特率设置为 115200bps，数据字长 8 位，停止位 1 位，无校验。在对一帧数据包包含多个有效数据进行编程，参考 0-ADD8 协议编写通信程序。对设备地址为 1 的开发板发送数码管显示数据，校验和正确后将数据显示在数码管上。

同步字	地址	数码管 0 数据	数码管 1 数据	数码管 n 数据	数码管 7 数据	校验 0-ADD8
0x55	0x01	0xXX	0xXX	……	0xXX	CRC

CRC=0-0x55-0x01-数码管 0 数据-……-数码管 7 数据。其结果取低 8 位。

五、思考题

1、你身边常见的串行口通信有哪些，列举 5 种以上？

2、你在实验中遇到的最大的困难是什么？如何解决？

实验五 DMA 实验

一、实验目的

1.学习 DMA 工作原理，掌握 DMA 初始化方法。

2.掌握 DMA 编程方法。

二、实验设备

1.硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

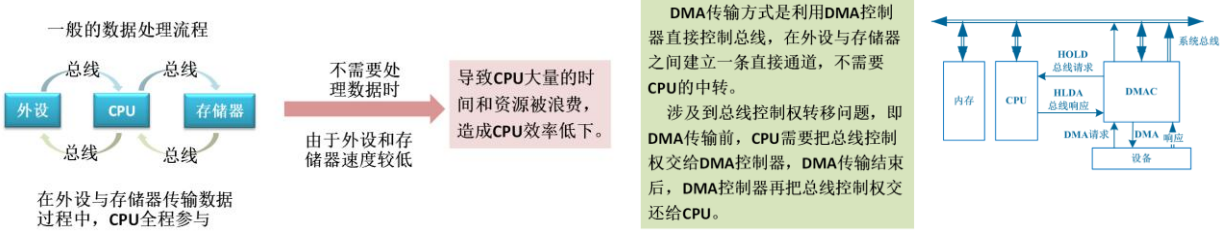
2.配套软件系统：

MDK5、CubeMX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验原理

1、DMA 概述

DMA，全称为：Direct Memory Access，即直接存储器访问。DMA 传输方式无需 CPU 直接控制传输,也没有中断处理方式那样保留现场和恢复现场的过程,通过硬件为 RAM 与 I/O 设备开辟一条直接传送数据的通路，能使 CPU 的效率大为提高。



DMA 数据传输的四个要素：①传输源：DMA 数据传输的来源②传输目标：DMA 数据传输的目的③传输数量：DMA 传输数据的数量④触发信号：启动一次 DMA 数据传输的动作。

STM32F103 微控制器具备两个 DMA 控制器:DMA1 和 DMA2(DMA2 仅存在大容量产品中)，DMA1 有 7 个通道，DMA2 有 5 通道。每个通道专门用来管理来自于一个或多个外设对存储器访问的请求。还有一个仲裁起来协调各个 DMA 请求的优先权。数据流（stream）是用于连接传输源和传输目标的数据通路，每个数据流可以配置为不同的传输源和传输目标，这些传输源和传输目标称为通道（Channel）。

STM32 最多有 2 个 DMA 控制器，从外设（TIMx、ADC、SPIx、I2Cx 和 USARTx）产生的 DMA 请求，通过逻辑或输入到 DMA 控制器，这就意味着同时只能有一个请求有效。外设的 DMA 请求，可以通过设置相应的外设寄存器中的控制位，被独立地开启或关闭。

DMA 的内部结构如下图所示。

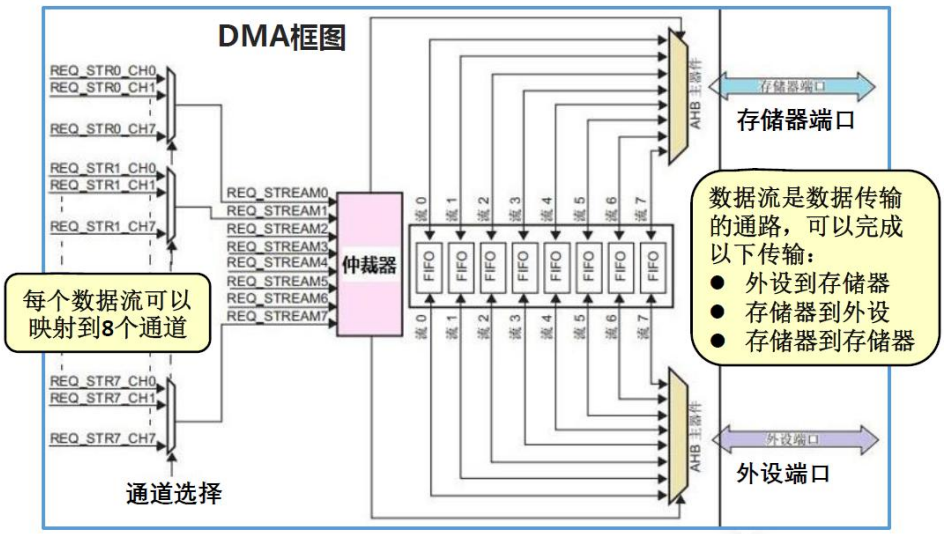


表 5.1 DMA1 各通道一览表

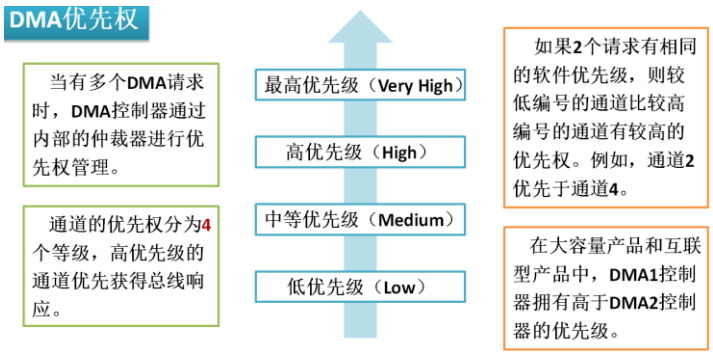
外设	通道1	通道2	通道3	通道4	通道5	通道6	通道7
ADC	ADC1						
SPI		SPI1_RX	SPI1_TX	SPI2_RX	SPI2_TX		
USART		USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
I^2C				I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
TIM1		TIM1_CH1	TIM1_CH2	TIM1_TX4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
TIM2	TIM2_CH3	TIM2_UP			TIM2_CH1		TIM2_CH2 TIM2_CH4
TIM3		TIM3_CH3	TIM3_CH4 TIM3_UP			TIM3_CH1 TIM3_TRIG	
TIM4	TIM4_CH1			TIM4_CH2	TIM4_CH3		TIM4_UP

DMA 数据传输方式：

普通模式 传输结束后（即要传输数据的数量达到零），将不再产生 DMA 操作。若开始新的 DMA 传输，需在关闭 DMA 通道情况下，重新启动 DMA 传输。

循环模式 可用于处理环形缓冲区和连续数据流（例如 ADC 扫描模式）。当激活循环模式后，每轮传输结束时，要传输的数据数量将自动用设置的初始值进行加载，并继续响应 DMA 请求。

DMA 优先权



DMA 中断请求

DMA中断请求

DMA的每个通道都可以在DMA传输过程中触发中断，可通过设置相应寄存器的不同位来打开这些中断。

DMA中断事件主要有：

- HT（Half Transfer，传输一半）
- TC（Transfer Complete，传输完成）
- TE（Transfer Error，传输错误）

分别对应三个中断标志：HTIF、TCIF、TEIF，每个中断标志都有允许控制位。

中断事件标志位	中断使能控制位	中断事件
HTIF	HTIE	传输过半
TCIF	TCIE	传输完成
TEIF	TEIE	传输错误

2、DMA 接口函数

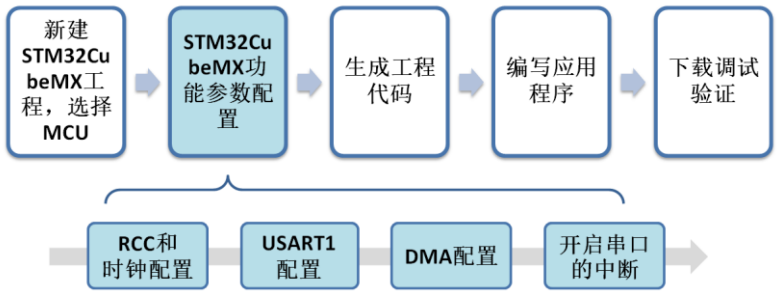
1 串口DMA方式发送函数：HAL_UART_Transmit_DMA	
函数原型	HAL_StatusTypeDef HAL_UART_Transmit_DMA (UART_HandleTypeDef *huart,uint8_t *pData, uint16_t Size)
功能描述	在DMA方式下发送一定数量的数据
入口参数1	huart：串口句柄的地址
入口参数2	pData：待发送数据的首地址
入口参数3	Size：待发送数据的个数
返回值	HAL状态值：HAL_OK表示发送成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用
注意事项	1. 该函数将启动DMA方式的串口数据发送 2. 完成指定数量的数据发送后，可以触发DMA中断，在中断中将调用发送中断回调函数HAL_UART_TxCpltCallback进行后续处理 3. 该函数由用户调用

2 串口DMA方式接收函数：HAL_UART_Receive_DMA	
函数原型	HAL_StatusTypeDef HAL_UART_Receive_DMA (UART_HandleTypeDef *huart,uint8_t *pData, uint16_t Size)
功能描述	在DMA方式下接收一定数量的数据
入口参数1	huart：串口句柄的地址
入口参数2	pData：待接收数据的首地址
入口参数3	Size：待接收数据的个数
返回值	HAL状态值：HAL_OK表示接收成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用
注意事项	1. 该函数将启动DMA方式的串口数据接收 2. 完成指定数量的数据接收后，可以触发DMA中断，在中断中将调用接收中断回调函数HAL_UART_RxCpltCallback进行后续处理 3. 该函数由用户调用

```
  //串口 DMA 方式接收函数
HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size);
  //串口 DMA 方式发送函数
HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size);
  //启动 DMA 发送数据函数
HAL_StatusTypeDef UART_Start_Receive_DMA(UART_HandleTypeDef *huart,
uint8_t *pData, uint16_t Size);
  // DMA 中断方式启动函数
HAL_StatusTypeDef HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t
SrcAddress, uint32_t DstAddress, uint32_t DataLength);
  // DMA 中断方式接收数据 回调 函数
static void UART_DMAReceiveCplt(DMA_HandleTypeDef *hdma);
  //中断方式 接收数据 回调 函数
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
  //DMA 方式 接收数据过半 回调 函数
```

```
static void UART_DMARxHalfCplt(DMA_HandleTypeDef *hdma);
  //中断方式 接收数据过半 回调 函数
__weak void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart);
  // DMA 中断方式发送数据 回调 函数
static void UART_DMATransmitCplt(DMA_HandleTypeDef *hdma);
  //中断方式发送数据 回调 函数
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart);
```

3、DMA 项目开发流程



四、实验内容

1、采用 DMA 方式编写串口程序，串行口波特率设置为 115200bps，数据字长 8 位，停止位 1 位，无校验。

(1)上位机向串行口发送一定长度的字符数据，开发板接收到数据后，将数据从串口发回。测试单次发送与循环发送，比较并记录实验效果。

(2) 上位机向串行口发送一定长度的字符数据，包括字母、数字、其他字符，开发板接收到数据后，数字字符滚动显示在数码管上，其他字符从串口发回。

2、采用 DMA 方式编写串口程序，串行口波特率设置为 115200bps，数据字长 8 位，停止位 1 位，无校验。实现简易“心跳包”程序。

(1)设计计数器 1，计数器 2，分别采集和记录按键 KB1、KB2 按下的次数，并将实时计数结果显示在数码管上。

(2)设计通信协议，将计数器 1、计数器 2 的计数结果封装成数据帧，以 DMA 方式向串行口发送，要求发送的数据与数码管上显示的数据同步。

五、思考题

1、USART2 以 DMA 方式发送和接收数据，应该使用那些 DMA 通道？

2、串行口发送数据 DMA 初始化时，为什么设置存储器地址自动加 1，外设地址不变呢？

实验六 定时器实验

一、实验目的

1. 学习定时器工作原理，掌握基本定时器、捕捉和比较的初始化方法。

2. 了解定时器相关 IO 口。

3. 掌握定时器编程方法。

二、实验设备

1. 硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

2. 配套软件系统：

MDK5、Cube MX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

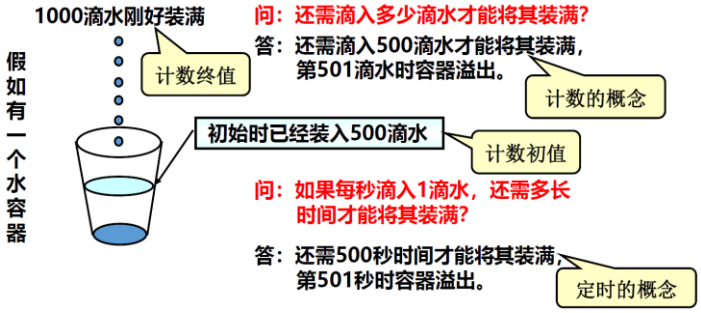
三、实验原理

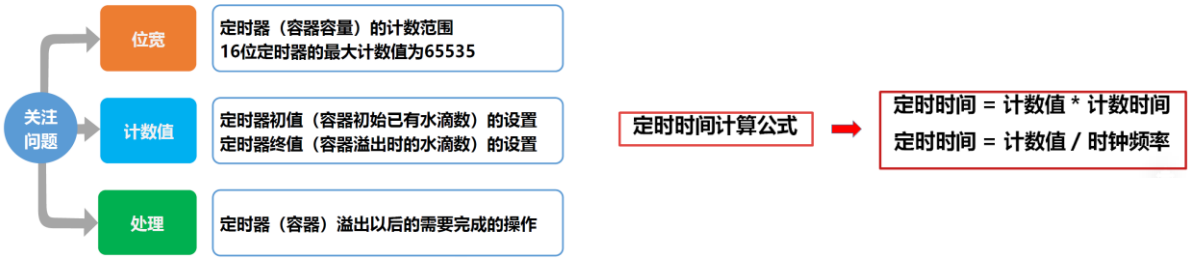
STM32F1 的定时器功能十分强大，有 TIME1 和 TIME8 等高级定时器，也有 TIME2~TIME5 等通用定时器，还有 TIME6 和 TIME7 等基本定时器。在《STM32 参考手册》里面，定时器的介绍占了 1/5 的篇幅，足见其重要性。

定时器是对周期固定的脉冲信号进行计数，如 MCU 内部的外设时钟(APB)。

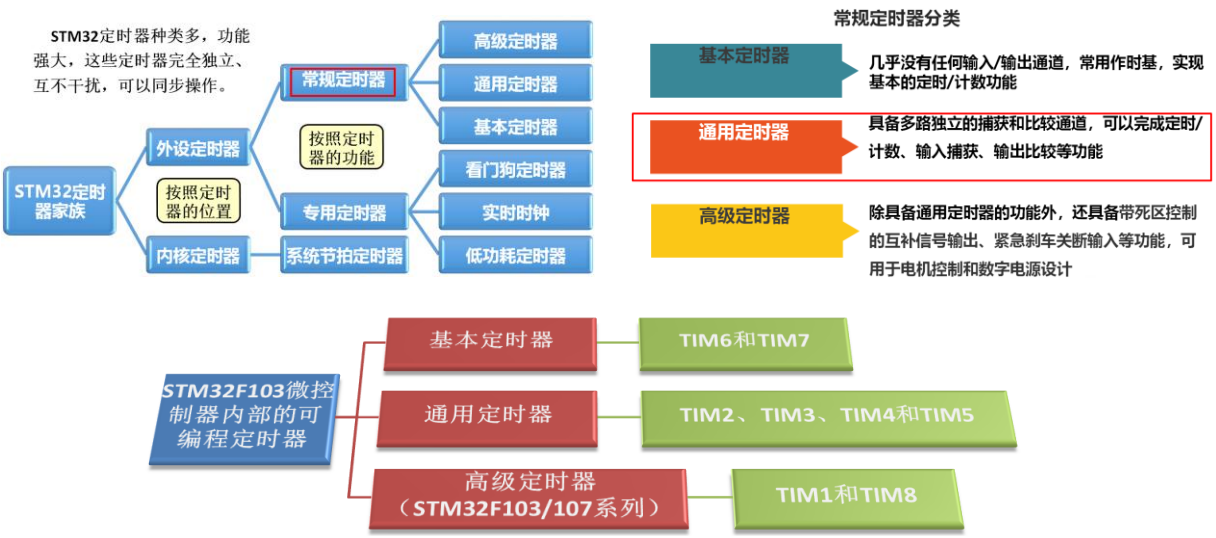
计数器是对周期不确定的脉冲信号进行计数，如 MCU 的 I/O 引脚所引入的外部脉冲。

定时器和计数本质上都是计数器，定时器是计数器的一种特例。





定时器模式的两个概念：送入定时器的周期性时钟信号的频率，计数单元记一次数所花费的时间，它是时钟频率的倒数，以此推算出定时时间。



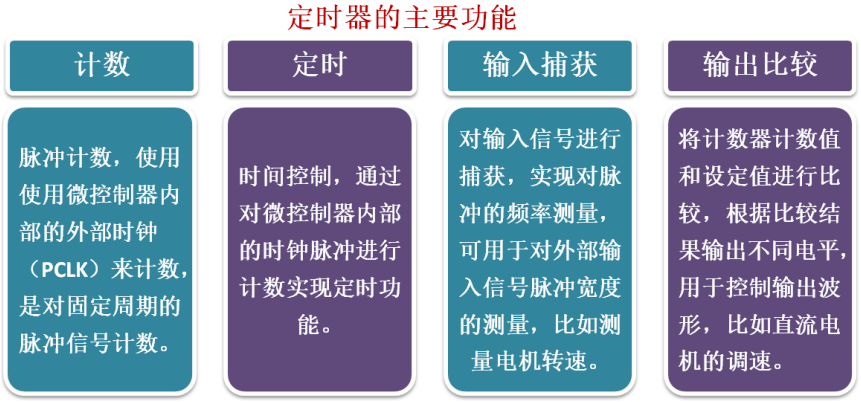
SysTick 定时器　SysTick 系统时钟位于 Cortex-M3 内核，是一个 24 位的递减计数器，主要用于：精确延时，在多任务操作系统中为系统提供时间基准（时基）；任务切换，为每个任务分配时间片。

WatchDog 看门狗　作用：当微控制器受到外部干扰或程序中出现不可预知的逻辑故障导致应用程序脱离正常的执行流程时（俗称程序跑飞），在一定的时间间隔内使系统复位，回到初始状态；看门狗设计是用来监视 MCU 程序运行状态的，是确保系统可靠稳定运行的一种有效措施。

STM32定时器分类比较表

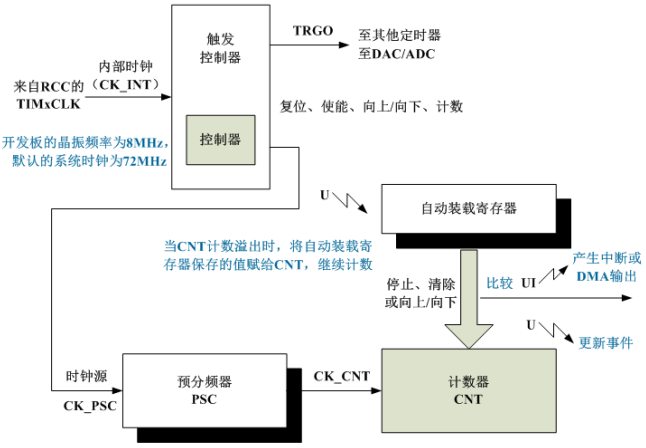
定时器	基本定时器 (TIM6、TIM7)	通用定时器 TIMx(x=2~5)	高级定时器 (TIM1、TIM8)
计数器类型	16位，向上	16位，向上、向下、向上/向下	16位，向上、向下、向上/向下
预分频系数	1～65535之间的任意数	1～65535之间的任意数	1～65535之间的任意数
输入/捕获通道	无	四个独立通道：输入捕获、输出比较、PWM生成、单脉冲模式输出	
产生中断/DMA	可以	可以	可以
刹车（电机控制）	无	无	可以

定时器的主要功能：

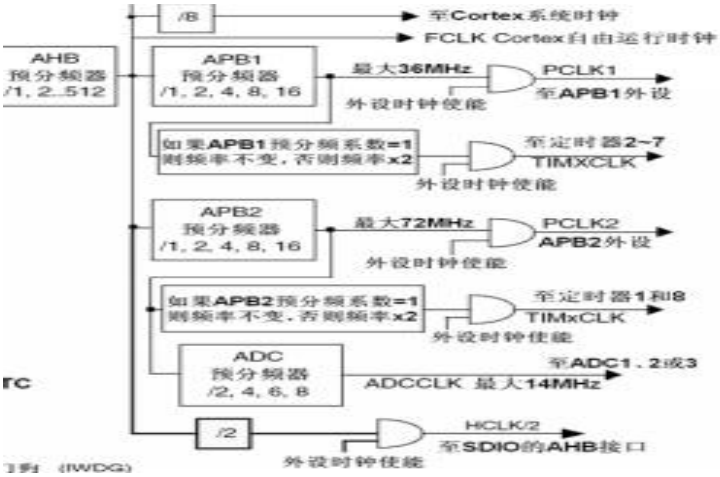


1、定时功能

STM32F103 系列微控制器的定时器功能十分强大，内部结构也比较复杂，STM32 通用定时器 TIMx（x=2，3，4，5）主要由时钟源、时钟单元、捕获和比较通道等构成，核心是可编程预分频驱动的 16 位自动装载计数器，具有定时、测量输入信号的脉冲长度（输入捕获）、输出所需波形（输出比较、产生 PWM、单脉冲输出等）等功能。



通用定时器内部简化图



当定时器使用内部时钟时，定时器的时钟源统称为 TIMxCLK。虽然在系统默认的配置中，TIMxCLK 的时钟频率都是 72MHz，但其时钟来源并不相同。定时器 TIM2~TIM7 挂接在 APB1 上，定时器 TIM1 和 TIM8 挂接在 APB2 上。

预分频器 PSC：可以以 1～65535 之间的任意数值对时钟源 CK_PSC 的时钟频率进行分频，输出 CK_CNT 脉冲供计数器 CNT 进行计数。

计数器 CNT：TIMxCNT 是一个 16 位的寄存器，计数范围为 1～65535，可以向上计数、向下计数或向下向上双向计数。要得到想要的计数值，需要对输入时钟频率进行分频。当计数值达到设定值时，便产生溢出事件，溢出时产生中断或 DMA 请求，然后再由自动装载寄存器进行重新加载或更新。计数器溢出中断属于软件中断，执行相应的定时器中断服务程序。

定时器的定时时间主要取决于定时周期和预分频因子，计算公式为：

定时时间=（ARR+1）×（预分频值 PSC+1）/输入时钟频率或

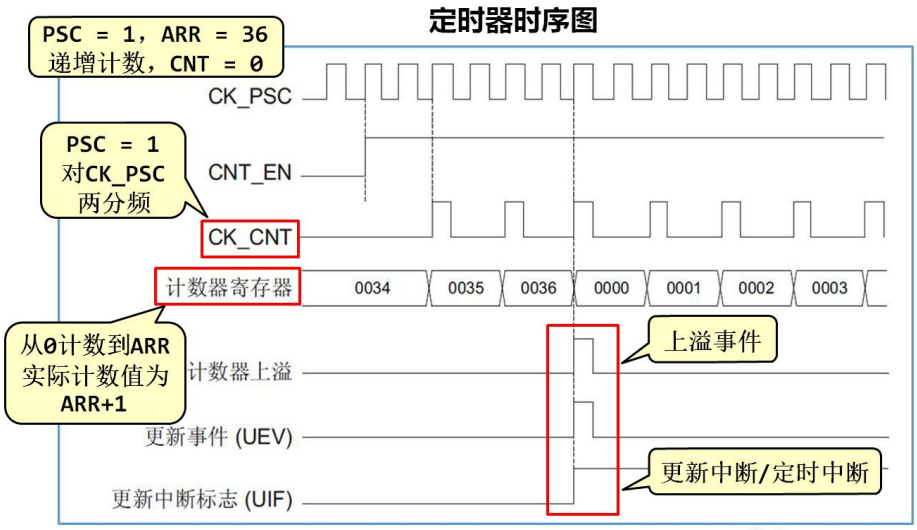
T=（TIM_Period +1）*（TIM_Prescaler +1）/TIMxCLK

这里 ARR+1 是因为计数器都是从 0 开始计数的。

例如：使用通用定时器定时 1s。假设系统时钟为 72MHz，通用定时器时钟 TIMxCLK 为 72MHz，设置如下：

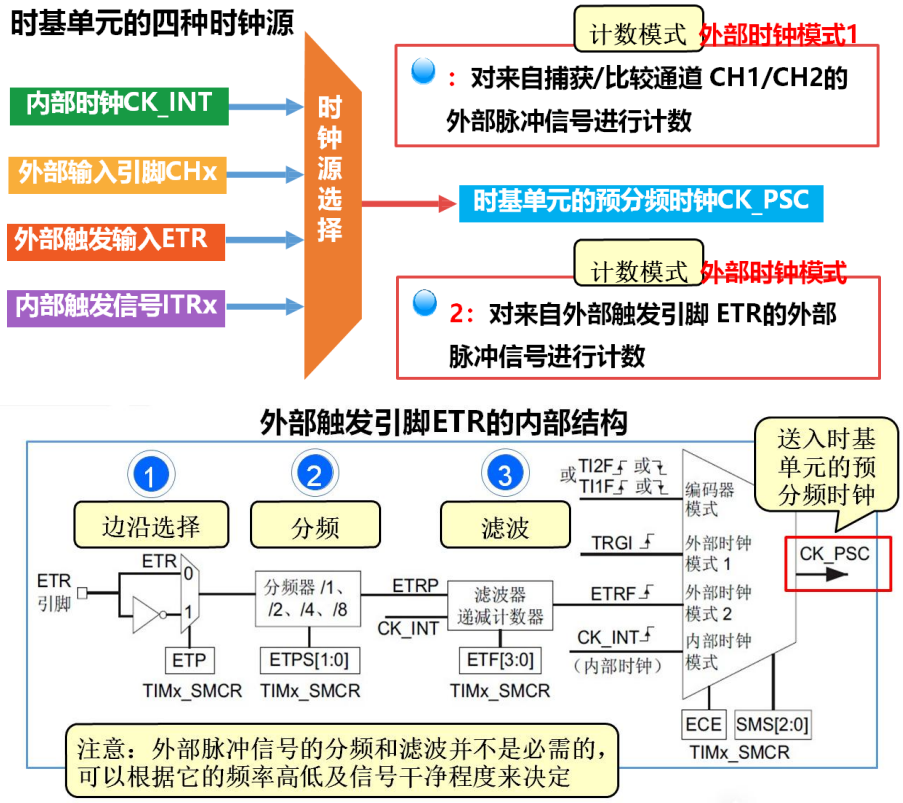
预分频系数 PSC=6000-1；ARR=2000-1；则，定时时间=2000×36000/72000000=1s

例如：使用通用定时器。假定分频 PSC 设置为 1，计数自动装入寄存器 ARR 设置为 1，定时器的溢出中断时序如图所示：



2、外部脉冲计数功能

如果将时钟源设置为外部引脚的脉冲，则可以进行对外部脉冲的计数功能。



3、PWM 输出功能

脉冲宽度调制（PWM）是一种对模拟信号电平进行数字编码的方法。广泛应用于电机控制、灯光的亮度调节、功率控制等领域。





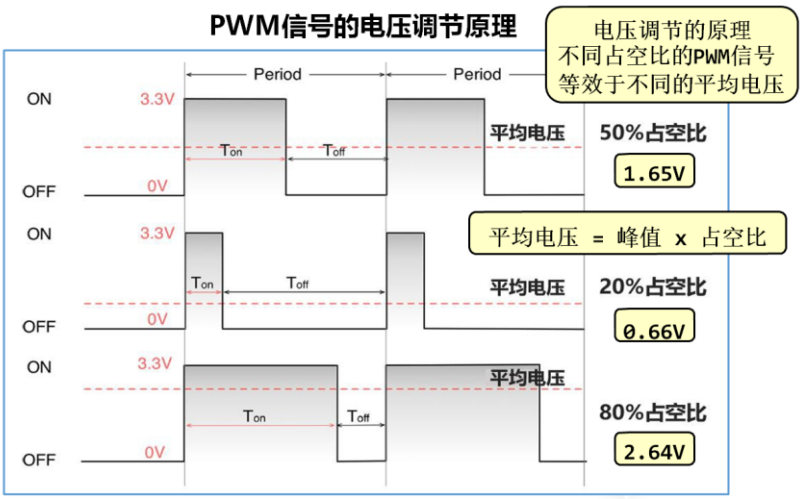


脉宽调制动画



脉宽调制的实质是修改高电平的持续时间

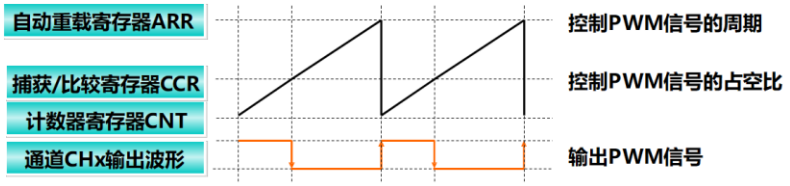
PWM 信号的两个基本参数：周期，一个完整 PWM 波形所持续的时间；占空比，高电平持续时间（Ton）与周期时间（Period）的比值。



工作原理

PWM输出的工作原理

- 初始输出高电平
- 匹配CCR时输出为低
- 匹配ARR时输出为高

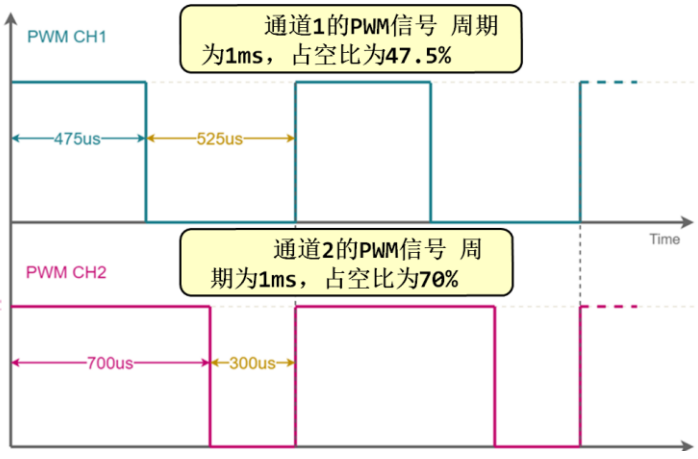


参数计算公式

```
Period(s) = ( ARR + 1 ) x ( PSC + 1 ) / TIMx_CLK
Duty      = ( CRR / ( ARR + 1 ) ) x 100%
```

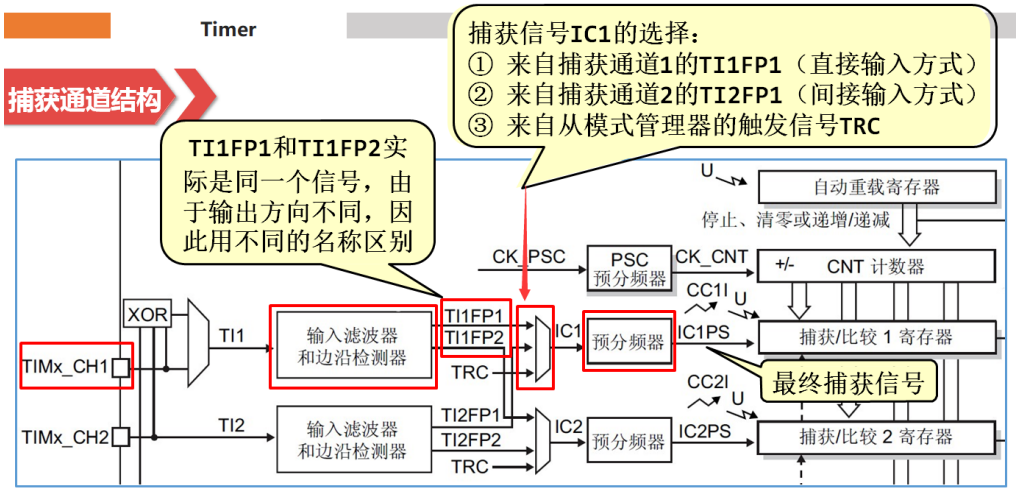
多通道输出

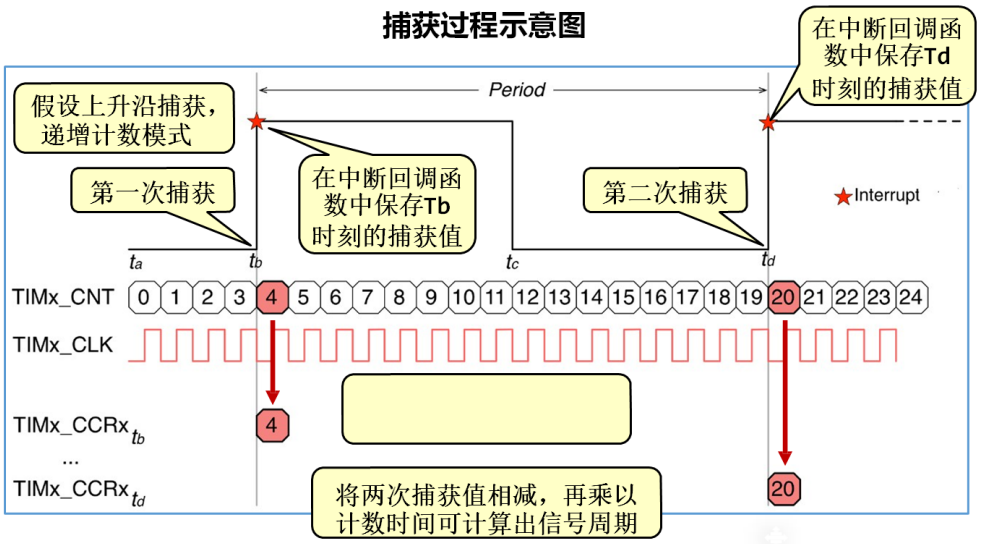
定时器的每个通道都可以输出PWM信号，对于同一个定时器而言，它的多个通道共享同一个自动重载寄存器，因此可以输出**占空比不同，但周期相同**的PWM信号。



4、输入捕获功能

在输入捕获模式下，当捕获单元捕捉到外部信号的有效边沿（上升沿/下降 沿/双边沿）时，将计数器的当前值锁存到捕获/比较寄存器 TIMx_CCR，供用户读取。用于测量信号的参数，比如周期和频率。





计算公式

信号参数计算公式

捕获差值　定时器时钟　预分频系数

```
Period(s) = Diff / TIMx_CLK x ( PSC + 1 )
Freq(Hz)  = TIMx_CLK / ( Diff x ( PSC + 1 ))
```

当待测信号不大于定时器的一个完整计数周期（从0到ARR）时。假设两次连续的捕获值分别为CCRx_1 和 CCRx_2，则捕获差值可以按照如下方法计算：

- **如果 CCRx_1 < CCRx_2：捕获差值 = CCRx_2 - CCRx_1**
- **如果 CCRx_1 > CCRx_2：捕获差值 = (ARR + 1 - CCRx_1) + CCRx_2**

注意：如果待测信号大于定时器的一个完整计数周期，则需要结合定时器的更新中断次数来计算捕获差值。

5、定时器设计操作流程

定时器的一般步骤可以总结为如下几个步骤：

- 定时器时钟使能
- 定时器参数初始化
- 开启中断并且初始化 NVIC（如果需要开启中断才需要这个步骤）
- 使能串口
- 编写中断处理函数

6. 定时器常用的库函数

```
HAL_StatusTypeDef HAL_TIM_Base_Init ( TIM_HandleTypeDef *htim )
    //时基单元初始化函数
HAL_StatusTypeDef HAL_TIM_Base_Start (TIM_HandleTypeDef *htim)
    //轮询模式启动函数
HAL_StatusTypeDef HAL_TIM_Base_Start_IT (TIM_HandleTypeDef *htim)
   //中断模式启动函数
void HAL_TIM_IRQHandler(TIM_HandleTypeDef *htim)
  //定时器中断通用处理函数
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
  /*定时器更新中断回调函数
   1.该函数由定时器中断通用处理函数 HAL_TIM_IRQHandler 调用，完成所 有定时器的更新中断的任务处理
   2.函数内部需要根据定时器句柄的实例来判断是哪一个定时器产生的本次更 新中断
   3.函数由用户根据具体的处理任务编写  */
HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)
    //定时器 PWM 输出启动函数
_HAL_TIM_SET_COMPARE(_HANDLE_,_CHANNEL_,_COMPARE_)
   //定时器比较/捕获寄存器设置函数
uint32_t      HAL_TIM_IC_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel)
   //输入捕获启动函数
   HAL_StatusTypeDef HAL_TIM_IC_Stop_IT(TIM_HandleTypeDef *htim, uint32_t Channel)
```

```
   //输入捕获停止函数
void HAL_TIM_IC_CaptureCallback (TIM_HandleTypeDef *htim)
   //输入捕获中断回调函数
uint32_t      HAL_TIM_IC_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel)
   //捕获值读取函数
_HAL_TIM_SET_CAPTUREPOLARITY(_HANDLE_,_CHANNEL__,_POLARITY__)
  //捕获边沿设置函数
```

四、实验内容

1、电子时钟设计。时钟频率为 72Mhz，采用定时器 3。

（1）实现 1 秒延时，并控制 LED0 闪烁.

（2）在数码管上显示时钟，格式为“12-00-00”，设置定时器 1 秒延时，并实时更新时钟显示。

（3）设置 KB1、KB2、KB3 分别对时钟的小时、分、秒进行调整（KB1 按下，小时+1，KB2 按下，分钟+1，KB3 按下，秒钟+1）

2、PWM 输出设计。时钟频率为 72Mhz，采用定时器，PWM 输出。

（1）设计 0.1 毫秒延时，设计占空比 50%，全彩灯 RGB_R.

（2）设置 KB1 调整占空比，5 级调整，循环按键。

（3）设计全彩灯 RGB_R 呼吸渐变效果。

3、外部脉冲计数设计。

设计计数器，对外部脉冲（开发板上 PIC 单片机提供的脉冲信号）进行计数，并在数码管上实时显示计数值。

4、用跳线短接 PB1 和（开发板上 PIC 单片机提供的脉冲信号 Test 端），PB1 可作为 TIM3_CH4 的信号捕捉脚，测量 PIC 信号发生器脉冲的频率和周期，并通过串行口返回测量的值。

五、思考题

1、采用 72Mhz 时钟，单个定时器最大定时时间为 $2^{16}X2^{16}/72000000=59.6$ 秒，如果要实现更长时间的延时，可以采取什么办法？

2、将汽车的速度转换为光电编码输出脉冲，脉冲频率和汽车速度成正比，思考如何用定时器实时采样汽车速度？

实验七 ADC 实验

一、实验目的

1. 学习 ADC 工作原理，掌握 ADC 初始化方法。

2. 了解 ADC 相关 IO 口。

3. 掌握 ADC 编程方法。

二、实验设备

1. 硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

2. 配套软件系统：

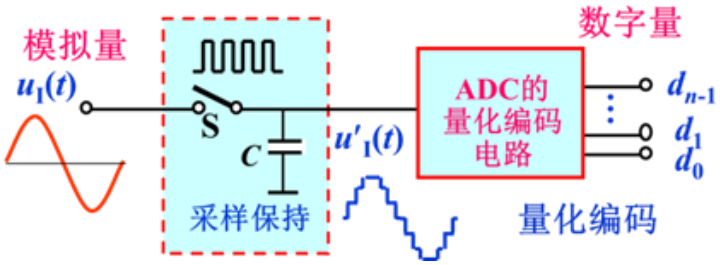
MDK5、CubeMX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验原理

STM32 拥有 1~3 个 ADC（STM32F101/102 系列只有 1 个 ADC），这些 ADC 可以独立使用，也可以使用双重模式（提高采样率）。STM32 的 ADC 是 12 位逐次逼近型的模拟数字转换器。它有 18 个通道，可测量 16 个外部和 2 个内部信号源。各通道的 A/D 转换可以单次、连续、扫描或间断模式执行。ADC 的结果可以左对齐或右对齐方式存储在 16 位数据寄存器中。模拟看门狗特性允许应用程序检测输入电压是否超出用户定义的高/低阀值。

STM32F103 系列最少都拥有 2 个 ADC，我们选择的 STM32F103ZET 包含有 3 个 ADC。STM32 的 ADC 最大的转换速率为 1Mhz，也就是转换时间为 1us（在 ADCCLK=14M，采样周期为 1.5 个 ADC 时钟下得到），**不要让 ADC 的时钟超过 14M**，否则将导致结果准确度下降。

ADC（Analog to Digital Converter）即模数转换器，用来将模拟信号转换为数字信号，A/D 转换器对输入模拟量微小变化的分辨能力，通常用二进制数的有效位表示，在最大输入电压一定时，位数越多，量化单位越小，误差越小，分辨率越高。STM32F103 采用 12 位转换，16 位寄存器存储。



位数	级数	分辨率（精度）	说明
8	$2^8=256$	39.1 mV	8位、12位ADC常用于工控领域、语音处理领域
10	$2^{10}=1024$	9.77 mV	
12	$2^{12}=4096$	2.44 mV	
14	$2^{14}=16384$	0.61 mV	
16	$2^{16}=65536$	0.15 mV	16位或更高的ADC常用于医学领域中微弱生理信号的采集

A/D 转换的主要技术参数：

转换时间是指从转换控制信号到来到 A/D 转换器输出端得到稳定的数字量所需要的时间。转换时间的倒数即为转换速率。转换时间与 A/D 转换器类型有关：

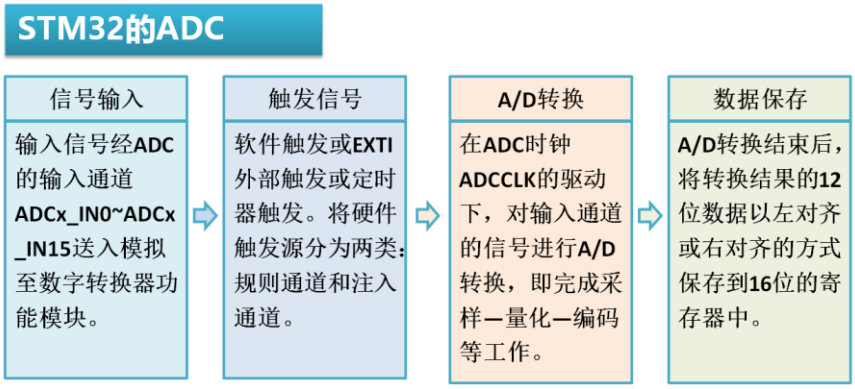
双积分型 A/D 转换器的转换时间一般在几十毫秒数量级，属于低速 A/D 转换器；

逐次比较型在几十微秒，属于中速 A/D 转换器；

并行比较型一般在几十纳秒。

转换误差 由于 A/D 转换器的有限分辨率而引起的误差，反映 A/D 转换器实际输出数字量和理想输出之间的差异。

STM32 集成有三个 12 位 ADC 逐次逼近型的 A/D 转换器，转换结果是一个 12 位的二进制数，可以以左对齐或右对齐两种方式存储在 16 位数据寄存器中，多达 18 个通道，可实现 16 个外部模拟输入通道和 2 个内部信号源的 A/D 转换，各通道的 A/D 转换可采用单次、连续、扫描或间断模式执行.



ADC 的引脚：

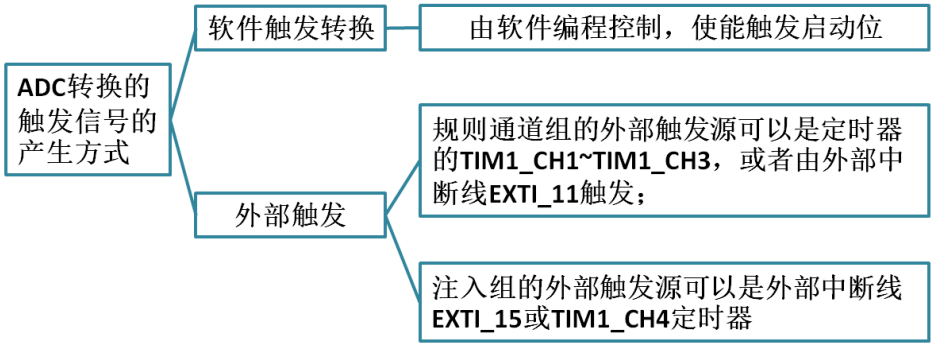
通道	ADC1（复用引脚）	ADC2（复用引脚）	ADC3（复用引脚）
通道0(ADC_Channel_0)	PA0	PA0	PA0
通道1(ADC_Channel_1)	PA1	PA1	PA1
通道2(ADC_Channel_2)	PA2	PA2	PA2
通道3(ADC_Channel_3)	PA3	PA3	PA3
通道4(ADC_Channel_4)	PA4	PA4	PF6
通道5(ADC_Channel_5)	PA5	PA5	PF7
通道6(ADC_Channel_6)	PA6	PA6	PF8
通道7(ADC_Channel_7)	PA7	PA7	PF9
通道8(ADC_Channel_8)	PB0	PB0	PF10
通道9(ADC_Channel_9)	PB1	PB1	
通道10(ADC_Channel_10)	PC0	PC0	PC0
通道11(ADC_Channel_11)	PC1	PC1	PC1
通道12(ADC_Channel_12)	PC2	PC2	PC2
通道13(ADC_Channel_13)	PC3	PC3	PC3
通道14(ADC_Channel_14)	PC4	PC4	
通道15(ADC_Channel_15)	PC5	PC5	
通道16(ADC_Channel_16)	温度传感器		
通道17(ADC_Channel_17)	内部参考电压		

ADC 通道选择：按组进行转换的模式，可以由程序设置实现对多个模拟通道自动地进行逐个采样转换，**STM32 分为两种组转换模式：规则组和注入组。**

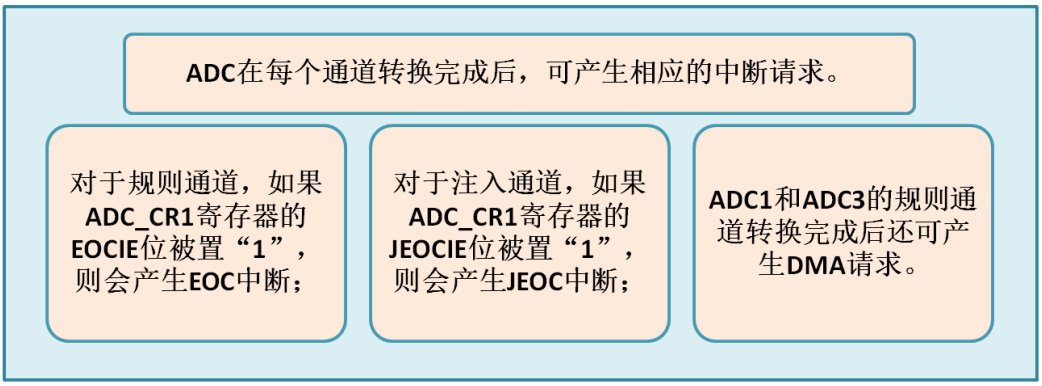
规则组 规则通道组最多允许 16 个规则通道进行转换，可通过编程来设定规则通道的数量 n，n 最多可设定为 16，转换结果保存在同一个 16 位的规则通道数据寄存器 ADC_DR 中，同时，EOC 标志被置位，产生相应的中断或 DMA 请求。

注入组 最多允许 4 个通道进行转换，并且对应有 4 个注入通道寄存器用来存放注入通道的转换结果，因此注入通道组没有 DMA 请求，注入通道组的触发转换序列和规则通道一样，转换结果保存数据寄存器 ADC_JDRx 中，同时产生 ADC 注入转换结束事件，即 JEOC 标志被置位，产生相应的中断，规则通道组转换好比是程序的正常执行，注入通道组的转换则好比是一个中断处理程序 。

ADC 转换触发信号产生方式：



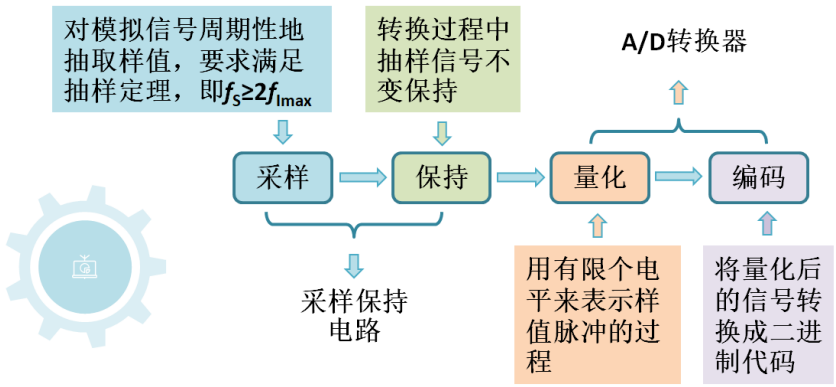
ADC 转换的中断标志：



ADC 中断事件主要有三个，ADC_IT_EOC 中断针对规则通道，ADC_IT_JEOC 中断针对注入通道

中断事件	事件标志	使能控制位
规则组转换结束中断 ADC_IT_EOC	EOC中断 （End of Coversion）	EOCIE
注入组转换结束中断 ADC_IT_JEOC	JEOC中断 （End of injected Coversion）	JEOCIE
模拟看门狗中断 ADC_IT_AWD	AWD中断 （Analog WatchDOG）	AWDIE

A/D 转换过程：



STM32 将 ADC 的转换分为 2 个通道组：规则通道组和注入通道组。规则通道相当于你正常运行的程序，而注入通道呢，就相当于中断。在你程序正常执行的时候，中断是可以打断你的执行的。同这个类似，注入通道的转换可以打断规则通道的转换，在注入通道被转换完成之后，规则通道才得以继续转换。

1、ADC 在嵌入式开发板 IO 口配置，见表。

序号	基本端口	端口模式	基本功能/扩展功能	备注
1	PA0	浮空输入	电压基准（2.5V）	
2	PA2	浮空输入	光敏电阻	
3	PA3	浮空输入	电位器	
4	PA1	浮空输入	热敏电阻	

2、下面我们介绍 ADC 的 HAL 库函数

类型		函数及功能描述
引脚功能操作函数	轮询方式	HAL_ADC_Start(ADC_HandleTypeDef* hadc); 功能描述：启动ADC转换
		HAL_ADC_Stop(ADC_HandleTypeDef* hadc); 功能描述：停止ADC转换
		HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout); 功能描述：等待转换结束，采用超时管理机制
	中断方式	HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc); 功能描述：中断方式下启动ADC转换
		HAL_ADC_Stop_IT(ADC_HandleTypeDef* hadc); 功能描述：中断方式下停止ADC转换
	DMA方式	HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length); 功能描述：DMA方式下启动ADC转换
		HAL_ADC_Stop_DMA(ADC_HandleTypeDef* hadc); 功能描述：DMA方式下停止ADC转换
		uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc); 功能描述：轮询或中断方式下获取ADC转换值
中断处理函数（回调函数）		void HAL_ADC_IRQHandler(ADC_HandleTypeDef* hadc); 功能描述：ADC中断处理函数
		void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc); 功能描述：ADC中断回调函数，用户在该函数内编写实际的中断处理程序
		void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef* hadc); 功能描述：ADC转换一半时调用的中断回调函数
外设配置函数		HAL_ADC_ConfigChannel(ADC_HandleTypeDef* hadc, ADC_ChannelConfTypeDef* sConfig); 功能描述：ADC配置函数
外设状态函数		uint32_t HAL_ADC_GetState(ADC_HandleTypeDef* hadc); 功能描述：获取ADC转换状态函数
		uint32_t HAL_ADC_GetError(ADC_HandleTypeDef *hadc); 功能描述：获取ADC错误函数

开启 ADC 时钟之后，将 ADC 的全部寄存器重设为缺省值之后我们就可以通过 RCC_CFGR 设置 ADC 的分频因子。**分频因子要确保 ADC 的时钟（ADCCLK）不要超过 14Mhz**。这个我们设置分频因子位 6，时钟为 72/6=12MHz。

3、**ADC 的 HAL 库应用实例**

STM32 芯片内部有一个温度传感器，测量范围为：-40℃~125℃，测量精度范围为±1.5℃，用来测量芯片内部的温度，连接在 ADC1_IN16 的输入通道上， 本实例将采集到的温度数据通过串口发送到 PC 机.

Reading the temperature

To use the sensor:

1. Select the ADCx_IN16 input channel.
2. Select a sample time of 17.1 µs
3. Set the TSVREFE bit in the *ADC control register 2 (ADC_CR2)* to wake up the temperature sensor from power down mode.
4. Start the ADC conversion by setting the ADON bit (or by external trigger).
5. Read the resulting V_{SENSE} data in the ADC data register
6. Obtain the temperature using the following formula:

Temperature (in °C) = {(V_{25} - V_{SENSE}) / Avg_Slope} + 25.

Where,

V_{25} = V_{SENSE} value for 25° C and

Avg_Slope = Average Slope for curve between Temperature vs. V_{SENSE} (given in mV/° C or µV/ °C).

Refer to the Electrical characteristics section for the actual values of V_{25} and Avg_Slope.

CubeMX 配置：

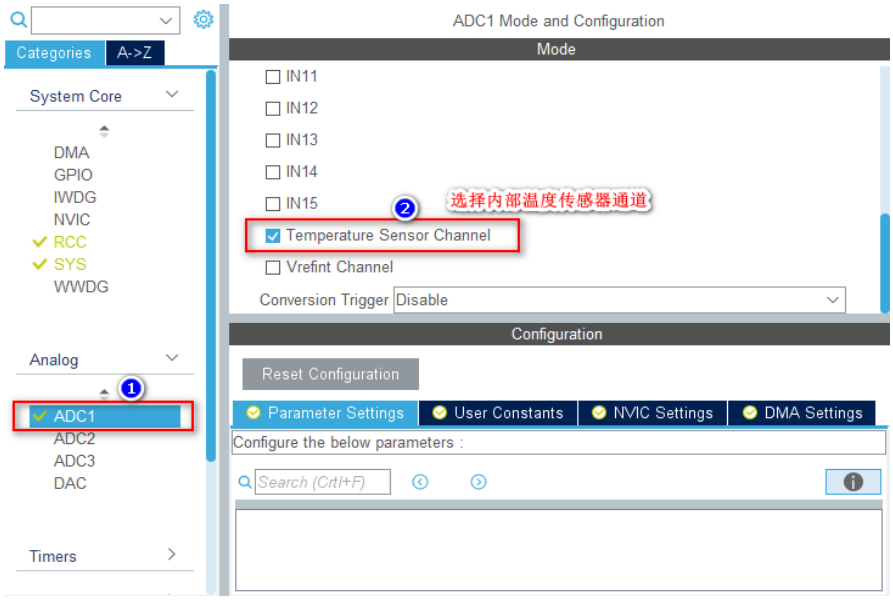
RCC-外部晶振

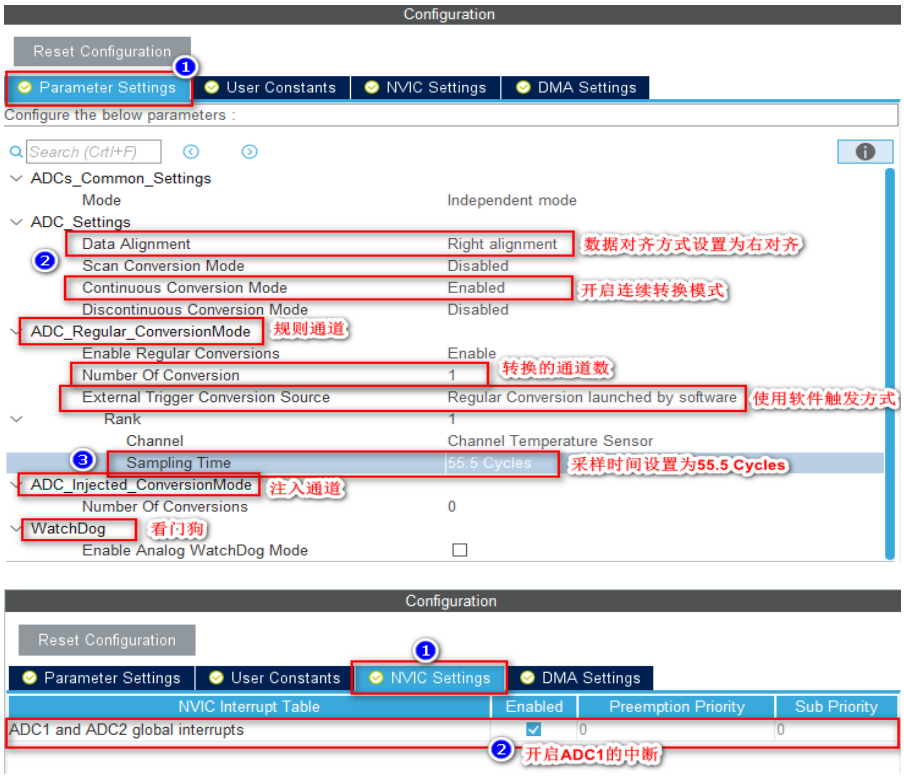
SYS-Serial Wire

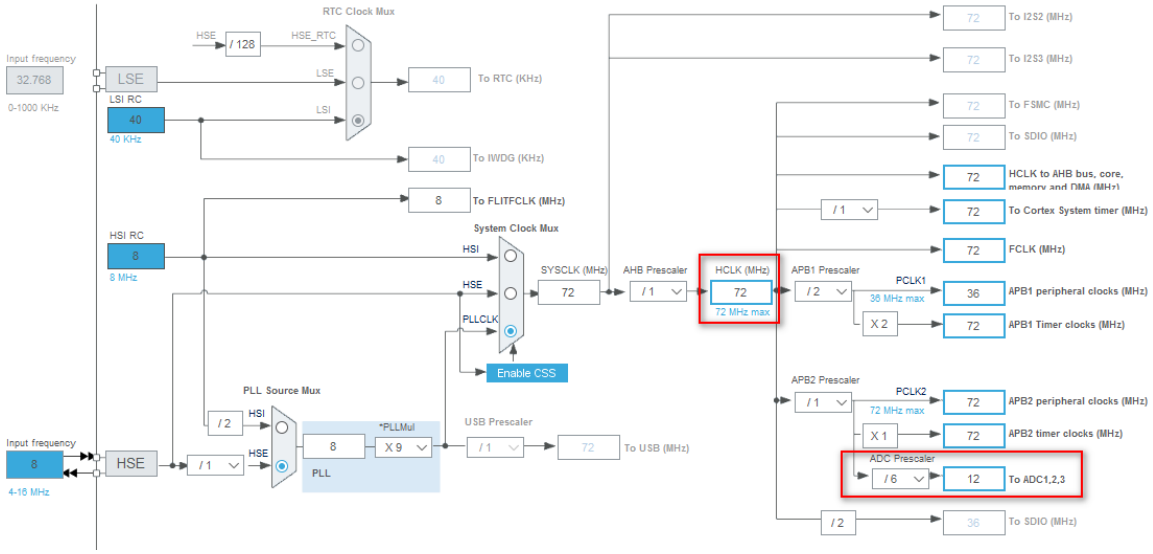
ADC1-第 16 通道（Temperatures Sensor Channel），打开全局中断

时钟树-系统时钟 72MHZ-ADC 时钟 12MHZ

USART1-异步模式







DMA 方式程序设计：

HAL_ADCEx_Calibration_Start(&hadc1); /* ADC1 校准*/

HAL_ADC_Start_DMA(&hadc1,(uint32_t*)&ADC_Converted_Value,sizeof(&ADC_Converted_Value)); /*启动 ADC1 的 DMA 转换 */

MCU_Temperature = (V25-ADC_Converted_Value*3.3/4095)/Avg_Slope+25; //计算

printf("\r\n The MCU temperature = %3d \r\n ",MCU_Temperature); //输出

查询方式程序设计：

HAL_ADC_Start(&hadc1);//启动 ADC 装换

```
HAL_ADC_PollForConversion(&hadc1, 50);
//等待转换结束，第二个参数为超时时间，单位为 ms
//判断转换完成标志位是否设置，HAL_ADC_STATE_REG_EOC 为转换完成标志位
if(HAL_IS_BIT_SET(HAL_ADC_GetState(&hadc1), HAL_ADC_STATE_REG_EOC))
{  ADC_Converted_Value = HAL_ADC_GetValue(&hadc1); //读取 ADC 转换数据，数据为 12 位
   printf("MCU Temperature : %.1f 度\r\n",( (V25-ADC_Converted_Value*3.3/4095)/Avg_Slope+25);  //计算温度
 }
```

四、实验内容

1、利用 ADC1 通道 3 采样 PA3 的电压值,将 ADC 采样值和转换的电压值通过串行口返回，调节电位器，观察运行结果。

2、编写程序，利用光敏电阻设计光电开关，控制 LED1 点亮和熄灭。

3、设计一个室内温控系统，PA3 采样电压表示采集的温度，阈值上限为 2.5V，下限为 1V，采样电压显示在数码管上。

（1）当采样电压低于 1V（温度过低），蜂鸣器鸣叫（模拟声音报警），LED1 闪烁（模拟光报警），全彩灯打开（模拟加热）。

（2）当采样电压位于 1V~2.5V 时，正常，蜂鸣器不叫，发光二极管不亮，电机不转。

（3）当采样电压高于 2.5V 时（温度过高），蜂鸣器鸣叫（模拟声音报警），LED1 闪烁（模拟光报警），电机旋转（模拟风扇降温）。

4、（选做）使用温度传感器 DS18B20，重新设计室内温控系统。

五、思考题

1、STM32ADC 操作模式有哪些？

2、STM32CPU 温度数值如何计算？

实验八 嵌入式操作系统应用实验

一、实验目的

1. 了解操作系统的基本概念功能
2. 理解操作系统的任务、任务组成及优先级
3. 理解操作系统的时钟和中断
4. 了解理解操作系统的多任务应用

二、实验设备

1.硬件设备：

实验板（1 套）、USB 数据线（1 根）和计算机（一台）。

2.配套软件系统：

MDK5、CubeMX、FLYMCU.exe（STM32 芯片下载程序）、sscom5.13.1（串行口调试程序）和 USBtoUARTDriver（USB 驱动程序）。

三、实验原理

嵌入式开发中，基于操作系统的开发方式不仅能有效提高开发效率，而且能够提高系统的稳定性和可靠性，本实验采用的操作系统，是一种可移植、可剪裁、可固化的基本优先级的抢占式多任务实时内核。通过本实验，学习嵌入式操作系统的应用开发方法及程序实现过程，为后续综合设计奠定基础。

嵌入式系统的开发方式：轮询方式、前后台系统方式、多任务系统（或者基于嵌入式操作系统的方式）。

计算机系统的**分层设计思想：**把一个庞大而复杂的问题分解为若干个较小且易于处理的局部小问题；局部小问题的功能部件的叠加完成整个系统功能。分层设计思想应用在了计算机的各个领域，比如计算机系统的层次结构、计算机网络的分层结构等。

轮询方式：CPU 通过程序不断查询 I/O 设备是否准备就绪，效率较低。

前后台系统方式：通常由一个大循环和若干中断组成，大循环是后台，一般由 main()函数中的 while(1)死循环构成，中断是前台，由各种中断服务程序组成。前后台方式程序流程比较简单，常应用于单片机裸机应用程序的开发。

多任务系统/基于嵌入式操作系统的方式：引入操作系统层，由操作系统对嵌入式平台的硬件资源进行统一管理和分配，这种方式实时性好，开发效率高。

嵌入式操作系统（Embedded Operating System，EOS）是指用于嵌入式系统的操作系统。一般会有一个**内核（Kernel），内核是指系统的一个组件，包含了操作系统的最基本、最主要功能，比如任务管理、存储管理、I/O 设备管理等**。

常见的嵌入式操作系统如图 8-1 所示：

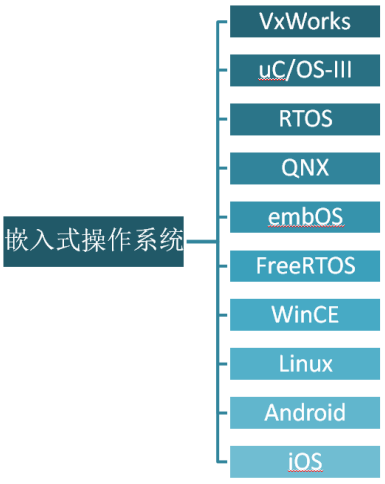


图 8-1　常见操作系统

相比 PC 机操作系统，**嵌入式操作系统**具有以下的**特点**：

可剪裁性　嵌入式系统应用环境差异很大，功能要求也各不相同，所以能够根据实际需求删减不需要的功能；

微型化　嵌入式系统体积小，系统资源受限，要求嵌入式操作系统本身不能占用过多的存储空间和资源；

实时性和可靠性　对实时性要求较高，且要求具备一定的抗干扰能力，可靠性高；

易移植　为满足多种硬件平台的统一性，要求嵌入式操作系统具有一定的可移植性。

1、 uC/OSⅢ嵌入式操作系统

uC/OSⅢ嵌入式操作系统包含了任务调度、任务管理、时间管理、内存管理、任务间的通信和同步等基本功能，具有一个抢占式多任务处理内核，意味着 uC/OSⅢ总是执行优先级最高、最重要的任务。该系统对任务数无限制，同时 uC/OSⅢ允许多个任务具有相同的优先级，对不同优先级的任务采用基于优先级的可抢占式调度法执行优先级最高的任务，对同等优先级执行时间轮转调度法。

（1）uC/OSⅢ的任务分为系统任务和用户任务。

系统任务　由操作系统提供的任务，为应用程序提供服务或为系统本身提供服务。

用户任务　由用户编写的实现应用程序功能的任务。

uC/OS-III 中每个任务都是一个无限循环，每个任务都有五种状态：休眠态、就绪态、运行态、挂起态、中断服务态。任务就是程序实体，uC/OS-III 通过管理和调度这些任务来实现应用程序功能。 任务可看作是一个简单的程序，每个任务被赋予一定的优先级，有对应的 CPU 寄存器和堆栈空间，以保证任务运行所需的系统资源。

（2）任务控制块　用来记录与任务相关的信息的数据结构，如任务堆栈指针、任务当前状态、任务优先级等。每一个任务都有一个唯一的优先级别，通常也用任务的优先级作为这个任务的标识。记录了任务的全部静态和动态信息，每个任务都必须有各自的任务控制块。系统需要为每个任务创建一个数据结构，称作任务控制块（Task Control Block，TCB）。多任务操作系统中，每个任务都有名字。任务控制块就相当于一个任务的身份证。

任务控制块由用户创建，OS_TCB 是一个结构体。

```
OS_TCB StartTaskTCB;
```

（3）**任务堆栈**

作用：任务切换时用来保存任务的运行环境。

特点：每个任务都有自己的专用堆栈，彼此之间不能互用。

堆栈：内存中按照“后进先出（LIFO）”的原则组织的一块连续的存储空间。

任务堆栈的创建：

```
#define START_STK_SIZE  512
//堆栈大小
CPU_STK START_TASK_STK[START_STK_SIZE];
 //定义一个数组来作为任务堆栈
```

（4）任务函数

由用户编写的任务处理代码，是应用功能的具体实现。uC/OS-III 应用程序由多个任务组成，任务的一般写法如下：

```
void XXX_Task(void *p_arg)
{
   while(1)
   {
      … … //任务处理过程
    }
}
```

（5）uC/OS-III 任务优先级

嵌入式操作系统支持多任务。每个任务都具有优先级，任务越重要，赋予的优先级应越高。uC/OS-III 对优先级的数量没有限制，任务优先级数由宏 OS_CFG_PRIO_MAX 来配置，uC/OS-III 将最低优先级只分配给空闲任务；任务就绪表根据任务优先级顺序存放已经就绪的任务。

（6）uC/OS-III 任务调度和切换

任务调度协调任务对计算机系统资源的争夺使用，直接影响着系统的实时性能。

任务调度方式

基于优先级的抢占式调度 内核总是将 CPU 分配给处于就绪态的优先级最高的任务运行，保证重要的突发事件及时得到处理。

不可抢占式调度 任务一旦获得 CPU 就独占其运行，除非由于某种原因使它放弃 CPU 的使用权。

时间片轮转调度 让优先级相同的处于就绪状态的任务按时间片轮流使用 CPU，以防止同优先级的某一任务长时间独占 CPU。

大多数嵌入式操作系统采用基于优先级的抢占式调度与时间片轮转调度相结合的调度机制；uC/OS-III 是一个抢占式多任务处理内核，同时又支持同优先级任务，对不同优先级的任务采用基于优先级的可抢占式调度法，对相同优先级的任务则采用时间片轮转调度法；uC/OS-III 的任务调度由任务调度器完成，任务调度器有两种：任务级调度器和中断级调度器。

任务切换：当 uC/OS-III 需要切换到另外一个任务时，保存当前任务的现场（主要是 CPU 各寄存器的值）到当前任务的堆栈中，然后恢复新的现场（新任务的 CPU 各寄存器的值），并且执行新任务。

（7）uC/OS-III 时钟

时钟节拍（操作系统的时基）实现时间的管理（延时、超时确认等）；STM32 采用 Cortex-M3 内核提供的 SysTick 定时器产生系统时钟节拍，是一个周期为毫秒级的系统时钟；uC/OS-III 允许用户通过配置宏的值来设定系统时钟节拍的频率，范围为 10~1000Hz。

uC/OS-III 时间管理——延时函数，按节拍延时函数 **OSTimeDly()**

```
void  OSTimeDly (OS_TICK  dly, OS_OPT  opt, OS_ERR  *p_err);
```

dly 延时的时间节拍数，注意，节拍数必须在 0~65535 之间。

opt 指定延时所使用的选项，共有四种：OS_OPT_TIME_DLY（相对延时模式）、OS_OPT_TIME_TIMEOUT、OS_OPT_TIME_PERIODIC（周期性延时模式）OS_OPT_TIME_MATCH（绝对定时模式）。

p_err 指向调用该函数后返回的错误码。

举例：如果时基频率设置为 1000Hz，而任务每次执行延时大约 2 毫秒，则使用：

```
OSTimeDly（2，OS_OPT_TIME_DLY，&err);
```

按时分秒毫秒延时函数 OSTimeDlyHMSM()

```
void OSTimeDlyHMSM (CPU_INT16U   hours,  //设置延时的小时数
                    CPU_INT16U   minutes,  //设置延时的分钟数
                    CPU_INT16U   seconds,  //设置延时的秒数
                    CPU_INT32U   milli,  //设置延时的毫秒数
                    OS_OPT       opt,  //选项，此函数只在相对延时模式下运行
                    OS_ERR       *p_err)
```

举例： OSTimeDlyHMSM(0,0,0,100,OS_OPT_TIME_HMSM_STRICT,&err);

（8）uC/OS-III 临界段 临界段是指那些必须完整连续运行、不可被打断的代码段。

保护临界段代码：关中断，当宏 OS_CFG_ISR_POST_DEFERRED_EN 为“0”时，uC/OS-III 使用关中断的方式。锁定调度器，当宏 OS_CFG_ISR_POST_DEFERRED_EN 为“1”时，采用给调度器上锁的方式。

（9）uC/OS-III 中断

uC/OS-III 支持中断，也允许中断嵌套，最多支持 250 级的中断嵌套；uC/OS-III 为了记录中断嵌套的层数，定义了一个全局变量 OSIntNestingCtr。在进入中断服务函数时要调用 OSIntEnter()，将中断嵌套计数 OSIntNestingCtr 加 1，OSIntEnter()定义在 os_core.c 文件中。

（10）uC/OS-III 移植与应用

所谓移植就是对一个实时的内核进行修改，使其能在其他的微处理器或者微控制器上运行。用高级语言编写的源程序经过不同的编译器编译后，会产生不同的机器代码，运行于各类不同指令集的 CPU 上，这就是跨平台的概念。

登录 Micrium 公司官网，在“Downloads”中单击“STMicroelectronics”，列出了针对不同处理器平台的移植例程工程，如 ST 公司的 STM32 系列微控制器。

2、 应用举例

要求：基于 uC/OSIII嵌入式操作系统，设计多任务在操作系统中执行.

任务 1：led0 闪烁；

任务 2：led1 以不同频率闪烁；

任务 3：串口 1 与上位机进行数据通信，发送实时计算结果。

操作步骤：

(1)配置嵌入式操作系统 uC/OSIII。

(2)任务创建

<1>编写任务函数，系统管理的 1 个开始任务函数、3 个用户任务函数。

```
//开始任务函数
void start_task(void *p_arg)
{   OS_ERR err;
    CPU_SR_ALLOC();
    p_arg = p_arg;
    BSP_Init();                         /* Initialize BSP functions */
    //CPU_Init();
    //Mem_Init();             /* Initialize Memory Management Module */
#if OS_CFG_STAT_TASK_EN > 0u
   OSStatTaskCPUUsageInit(&err);       //统计任务
#endif
#ifdef CPU_CFG_INT_DIS_MEAS_EN         //如果使能了测量中断关闭时间
    CPU_IntDisMeasMaxCurReset();
#endif
#if OS_CFG_SCHED_ROUND_ROBIN_EN        //当使用时间片轮转的时候
 //使能时间片轮转调度功能,时间片长度为 1 个系统时钟节拍，既 1*5=5ms
OSSchedRoundRobinCfg(DEF_ENABLED,1,&err);
#endif
```

```
}
/* led0 任务函数 */
void led0_task(void *p_arg)
{   OS_ERR err;
    p_arg = p_arg;
    while(1)
    {   HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET);
        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
        HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET);
        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    }
}
/* led1 任务函数 */
void led1_task(void *p_arg)
{  OS_ERR err;
    p_arg = p_arg;

    while(1)
    {   HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
        OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
        OSTimeDly(2000, OS_OPT_TIME_DLY, &err);
      }
    }
/* 串口输出测试任务 */
void print_task(void *p_arg)
{   uint32_t num = 0;
    OS_ERR err;
    p_arg = p_arg;
    CPU_SR_ALLOC();
    while(1)
    {   num++;
        OS_CRITICAL_ENTER();   //进入临界区
        printf("num = %d\r\n",num);
        OS_CRITICAL_EXIT();    //退出临界区
        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
    }
  }
```

<2>设置任务优先级

```
#define START_TASK_PRIO      3
#define LED0_TASK_PRIO       4
#define LED1_TASK_PRIO       5
#define PRINT_TASK_PRIO     6
```

<3>设置任务堆栈大小

```
#define START_STK_SIZE       512
#define LED0_STK_SIZE        128
#define LED1_STK_SIZE        128
#define PRINT_STK_SIZE      128
```

<4>任务控制块申明

```
OS_TCB StartTaskTCB;
OS_TCB Led0TaskTCB;
OS_TCB Led1TaskTCB;
OS_TCB PrintTaskTCB;
```

<5>申请任务堆栈区

```
CPU_STK START_TASK_STK[START_STK_SIZE];
CPU_STK LED0_TASK_STK[LED0_STK_SIZE];
CPU_STK LED1_TASK_STK[LED1_STK_SIZE];
CPU_STK PRINT_TASK_STK[PRINT_STK_SIZE];
```

<6>在 main 中创建开始函数任务

```
/* USER CODE BEGIN 2 */
OSInit(&err);     //初始化 UCOSIII
OS_CRITICAL_ENTER();//进入临界区
//创建开始任务
   OSTaskCreate((OS_TCB   * )&StartTaskTCB,       //任务控制块
       (CPU_CHAR* )"start task",      //任务名字
       (OS_TASK_PTR)start_task,       //任务函数
       (void       * )0,                  //传递给任务函数的参数
       (OS_PRIO      )START_TASK_PRIO,     //任务优先级
       (CPU_STK * )&START_TASK_STK[0],    //任务堆栈基地址
       (CPU_STK_SIZE)START_STK_SIZE/10,   //任务堆栈深度限位
       (CPU_STK_SIZE)START_STK_SIZE,      //任务堆栈大小
       (OS_MSG_QTY)0,//任务内部消息队列能够接收的最大消息数目,为 0 时禁止接收消息
       (OS_TICK      )0,   //当使能时间片轮转时的时间片长度，为 0 时为默认长度，
       (void       * )0,   //用户补充的存储区
       (OS_OPT     )OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR, //任务选项
       (OS_ERR     * )&err);                //存放该函数错误时的返回值
OS_CRITICAL_EXIT();  //退出临界区
OSStart(&err); //启动多任务系统，控制权交给 uC/OS-III
  /* USER CODE END 2 */
```

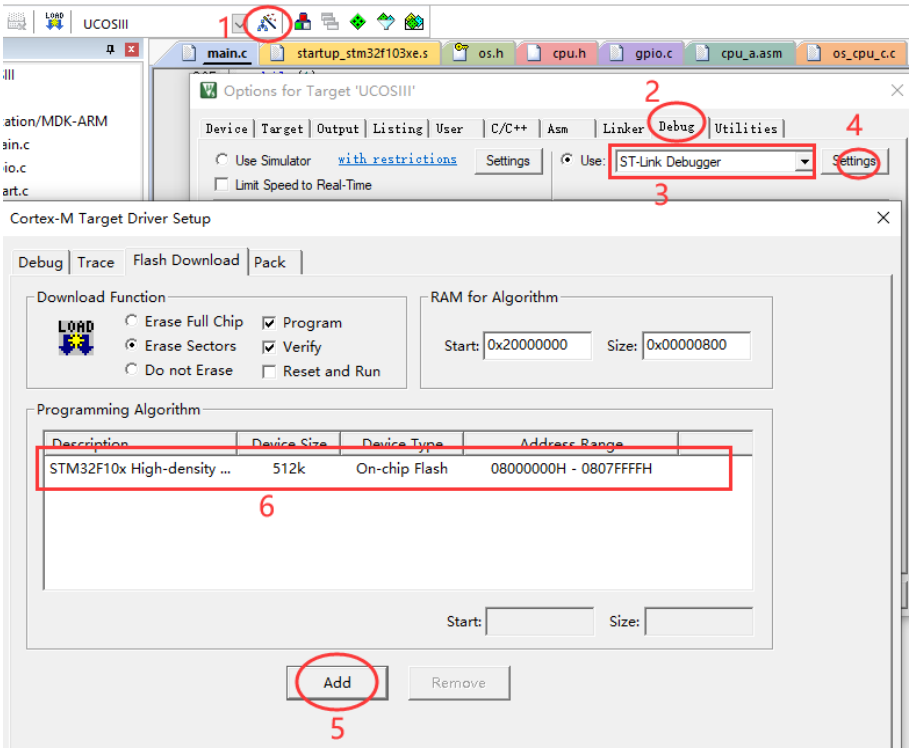
<7>在开始任务函数中最后创建用户函数任务

```
OS_CRITICAL_ENTER(); //进入临界区
/* 创建 LED0 任务 */
OSTaskCreate((OS_TCB * )&Led0TaskTCB,
             (CPU_CHAR   * )"led0 task",
             (OS_TASK_PTR )led0_task,
             (void       * )0,
             (OS_PRIO   )LED0_TASK_PRIO,
             (CPU_STK   * )&LED0_TASK_STK[0],
             (CPU_STK_SIZE)LED0_STK_SIZE/10,
             (CPU_STK_SIZE)LED0_STK_SIZE,
             (OS_MSG_QTY  )0,
             (OS_TICK   )0,
             (void    * )0,
             (OS_OPT      )OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR,
             (OS_ERR  * )&err);
/* 创建 LED1 任务 */
……
/* 创建串口通信任务 */
```

```
……
OS_TaskSuspend((OS_TCB*)&StartTaskTCB,&err);      //挂起开始任务
OS_CRITICAL_EXIT();  //进入临界区
```

(3)测试

由于操作系统较大，代码量较多，如果程序下载失败，查看 MDK 环境配置中选项，如果没有则需要按如下步骤设置添加。



四、实验内容

1. 配置嵌入式操作系统 uC/OSIII，完成实例程序的设计。
2. 创建任务 4：在数码管上显示自己的学号。
3. 船舰任务 5：蜂鸣器播放生日快乐音乐。
4. 创建任务 6：驱动电机正方向旋转。

五、思考题

1. uC/OSIII的内核包括哪几部分？
2. uC/OSIII在创建任务时需要完成那些工作？
3. 通过实验你体验到了嵌入式操作系统哪些优势？